

3. Spill

PO 1.4 Exploding JOIN – Must run below code to get Spill

```
%scala
// Create a large partition by mismanaging shuffle partitions
spark.conf.set("spark.sql.shuffle.partitions", 60)

// The location of our non-skewed set of transactions
val trxPath = "wasbs://spark-ui-simulator@dbacademy.blob.core.windows.net/global-sales/transactions/2011-to-2018_int-only_60gb.delta"

spark.read.load(trxPath).orderBy("trx_id").write.format("noop").mode("overwrite").save()

//spark.read.format("delta").load(trxPath).orderBy("trx_id").write.format("noop").mode("overwrite").save()
```



uc-mott1-ws 14.3

Low Shuffle Merge

Use DBR 8.2+ or the most recent Photon image: enable by ...
spark.databricks.delta.merge.enableLowShuffle true

Performance Optimization with Spark and Delta Lake

Note: DLT tables are auto optimized

PO 1.2 Data skipping
OPTIMIZE hive_metastore.dbacademy.flights ZORDER BY id, FlightNum;

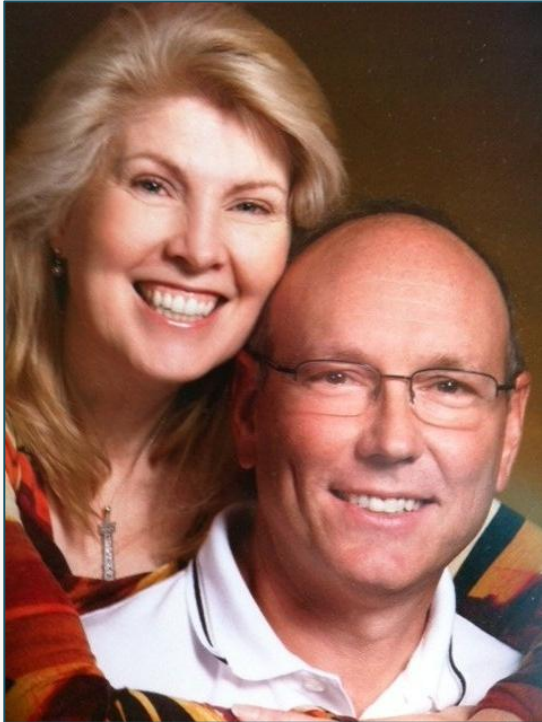
performance-optimization-with-delta-lake-and-photon

Name	Type	Owner
Includes	Folder	dbsp
Pipeline	Folder	dbsp
Solutions	Folder	dbsp
PO 1.0 - Module Introduction	Notebook	dbsp
PO 1.1 - File Explosion	Notebook	dbsp
PO 1.2 - Data Skipping	Notebook	dbsp
PO 1.3 - Shuffle	Notebook	dbsp
PO 1.4 - User-Defined Functions	Notebook	dbsp
PO 1.5L - Exploding Join	Notebook	dbsp





Your Instructor



Mark Ott Staff Technical Instructor

Cincinnati Ohio

Previous work: Teradata; ThoughtSpot
University of Cincinnati

Fun fact:

I'm a twin

Meet my bro, Mike



/in/mark-ott-610378b/



Images available for 7-days for 2-day or 4-day class
And 3-days for paid 4-hour class

Databricks Setup

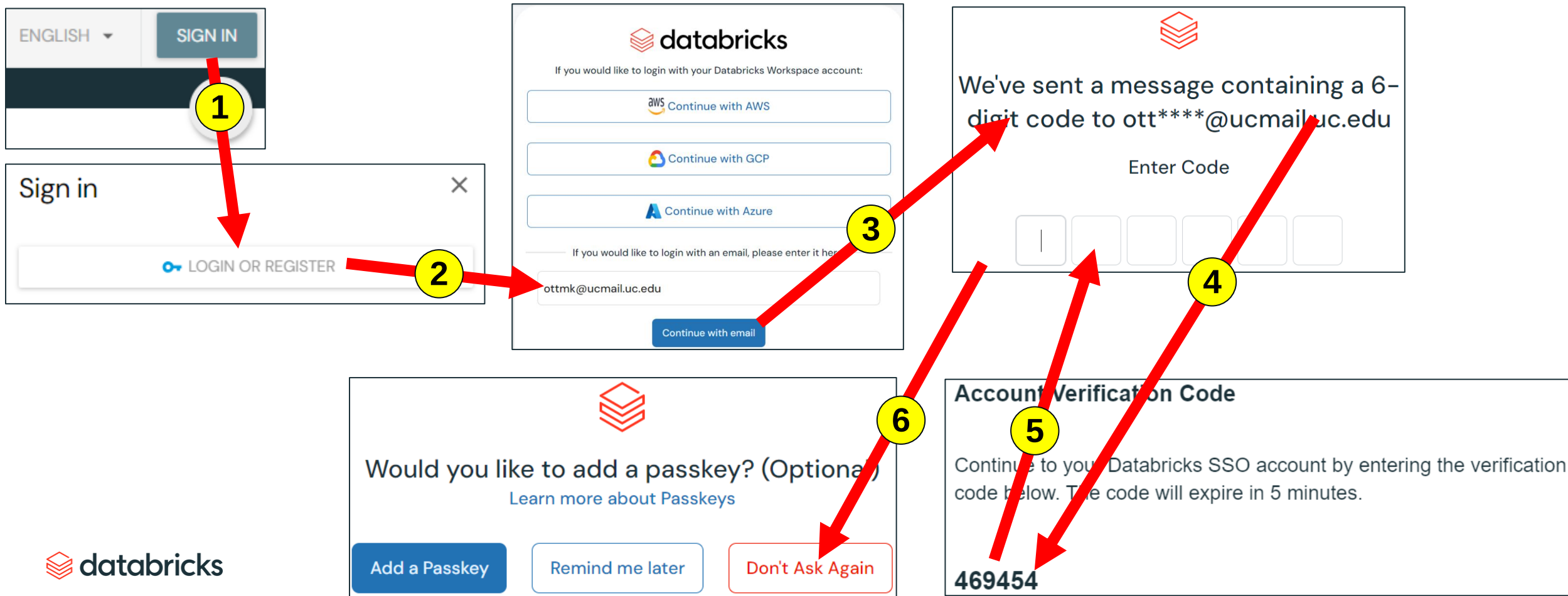
Things to try if can't connect:

- 1) No VPNs or firewalls
- 2) Don't use Edge/Safari for web browser
- 3) Disable pop-up blockers
- 4) Enable 3rd party cookies
- 5) Clear cache and try again
- 6) Try Incognito/Private browser

Step 1a: Login to Academy first

Login to Databricks Academy (via credentials you set up per email sent to you prior to class)

<https://customer-academy.databricks.com> or <https://partner-academy.databricks.com>



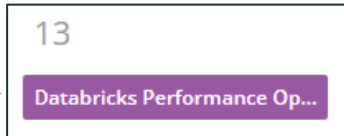
Step 1b: Get Spark Environment (1 of 2)

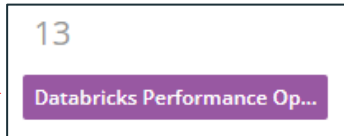
In upper-left hand corner, click 'hamburger' icon



- Then click on 'My Calendar' → 



- Click today's class link → 



- Click on the Class name in Orange (wait for it) → 



- In window, click on EVENTS tab → 

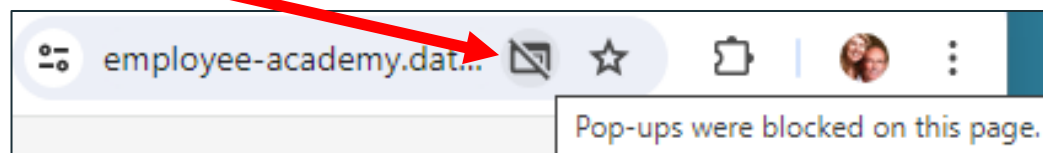
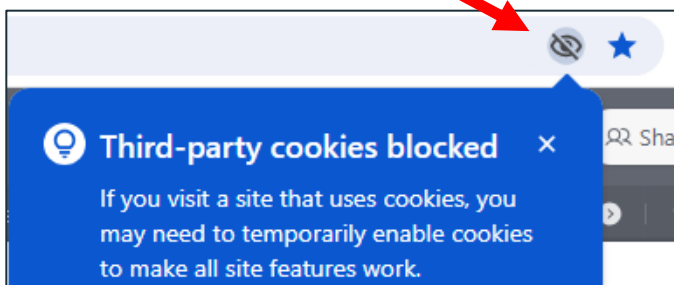


- Scroll down, then click on 'ILT' Lab Link → 



Click on icon to fix

If nothing happens, may have **pop-up blocker**/3rd party cookie issue. Here's fix



Step 1c: Get Spark Environment (2 of 2)



• As your lab spins up you will see this icon

• If get 'Sign In' window, click Blue button

→ 

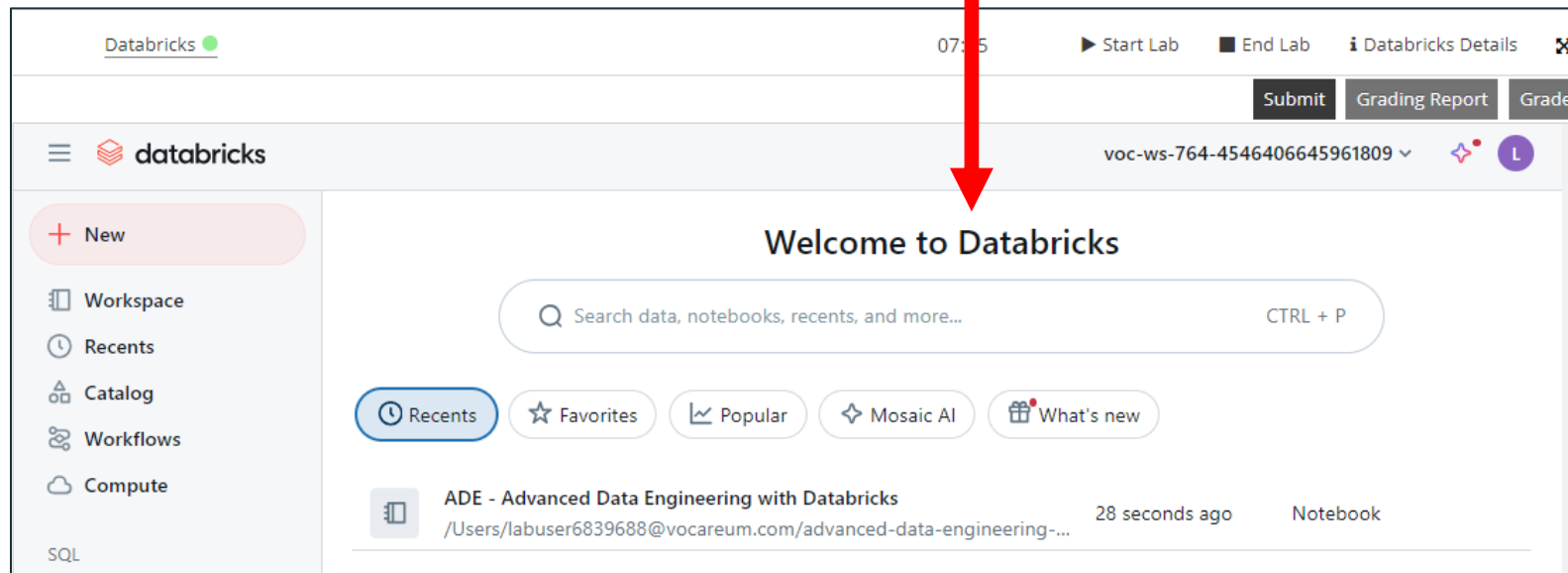
→ 

• In upper left-hand corner, Yellow dot turns Green dot

→ 

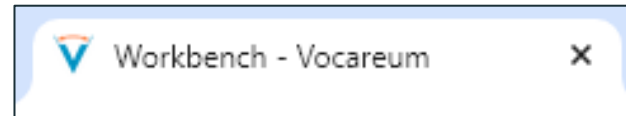


• Once environment is spun up, it will look like this

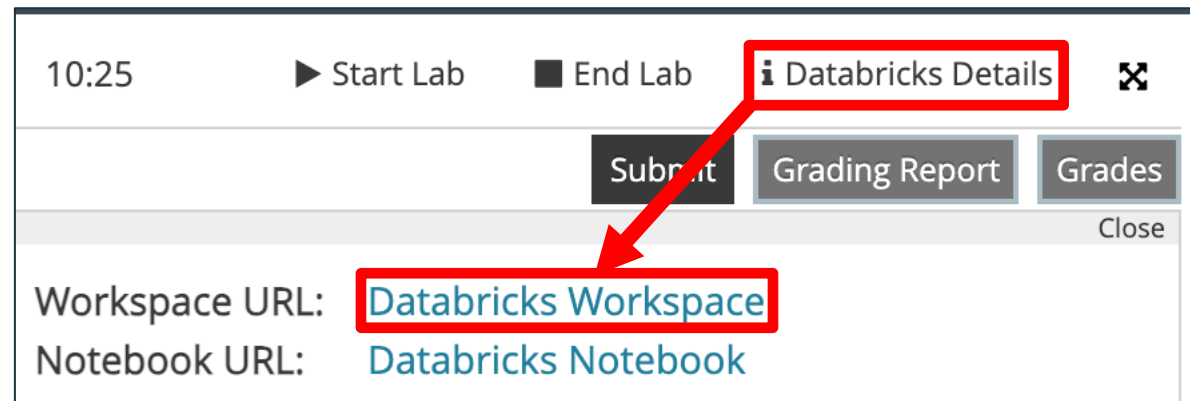


Step 2: Troubleshooting 'No workspace found'

If students are seeing **No workspace found**, go to below tab:



expand the **Databricks Details** pane,
and click on **Databricks Workspace** to remedy



Step 3: Ensure Cluster is Starting/Started



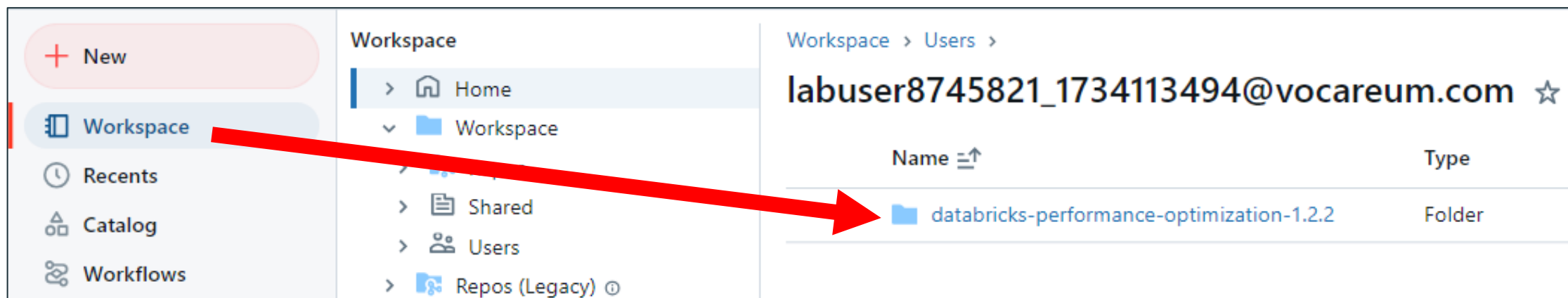
Compute

All-purpose compute Job compute SQL warehouses Vector Search Pools

Filter compute you ... Created by ☐ Only pinned Create with Personal Compute ☐ ⓘ

State	Name	Policy	Runtime	Active ...	Active c...	Active ...	Source	Creator	Notebo...
	labuser7177518@vocareum...	-	14.3	-	-	2	API	407ea82a-42...	-

Step 4: Ensure see Notebook folder



The screenshot shows the Databricks workspace interface. On the left, the 'Workspace' sidebar is visible with options: New, Workspace, Recents, Catalog, and Workflows. The 'Workspace' section is expanded, showing a tree view with 'Home', 'Workspace', 'Shared', 'Users', and 'Repos (Legacy)'. A red arrow points from the 'Workspace' sidebar to the 'databricks-performance-optimization-1.2.2' folder in the 'Users' view. The 'Users' view shows the user 'labuser8745821_1734113494@vocareum.com' and a table of folders.

Name	Type
databricks-performance-optimization-1.2.2	Folder

PO 1.0 - Module Introduction	Notebook
PO 1.1 - File Explosion	Notebook
PO 1.2L - Data Skipping and Liq...	Notebook
PO 1.3 - Shuffle	Notebook
PO 1.4L - Exploding Join	Notebook
PO 1.5 - User-Defined Functions	Notebook

Agenda – Perf Opt w/Delta Lake+Photon

Performance Optimization with Spark and Delta Lake

Lecture: Spark Architecture

Lecture: Designing the Foundation

Lecture: Code Optimization

Lecture/Demo: Shuffles

Lecture/Demo: Spill

Lecture/Demo: Skew

Lecture/Demo: Serialization

Lecture: Fine-Tuning – Choosing the Right Cluster

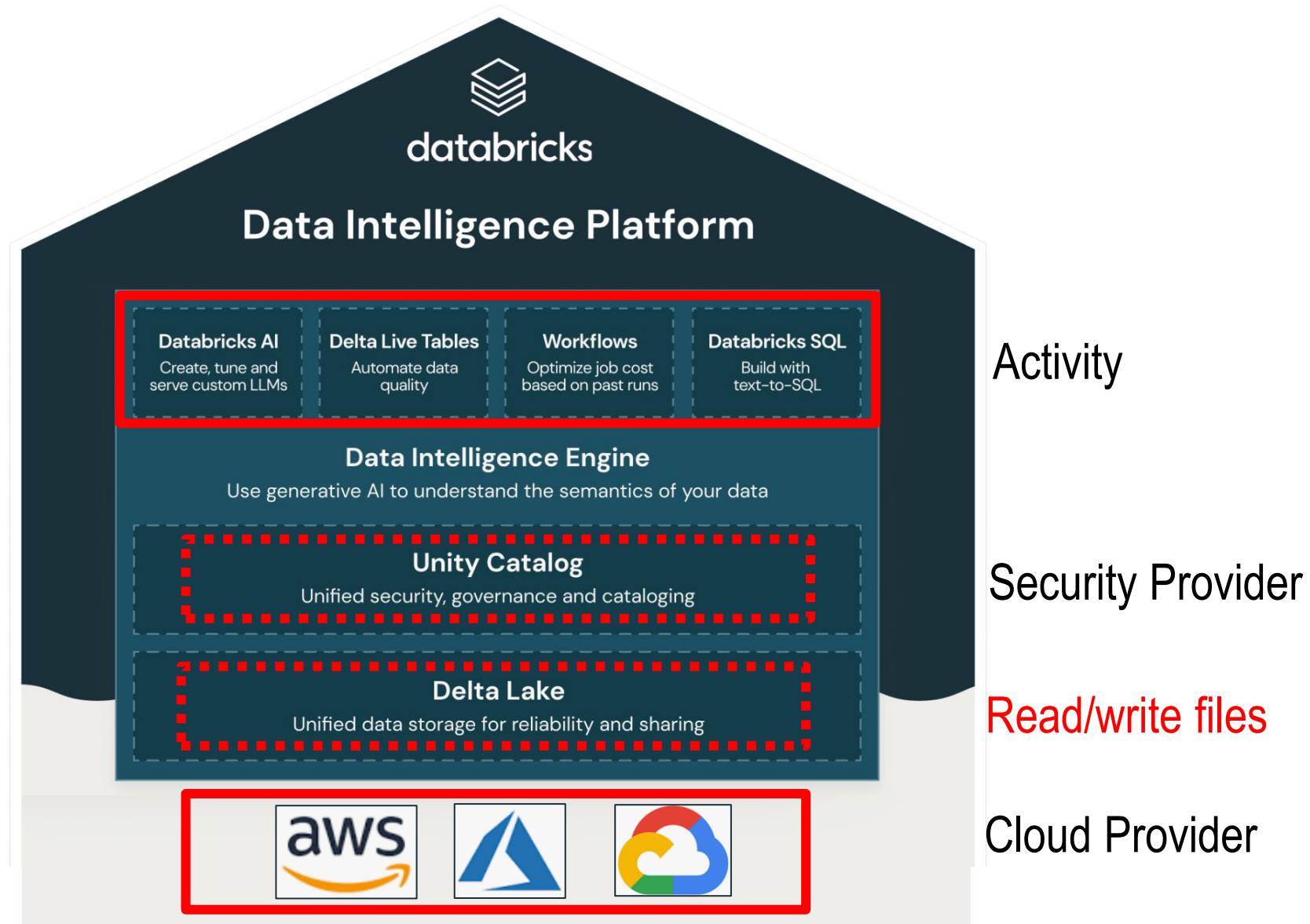


Spark Architecture





Bonus: Data Intelligence Platform



What is a Cluster?

An Engine to run your Queries

Collection of Virtual Machine (VM) instances you rent from Cloud provider

General purpose (Delta cache accelerated)	
m5d.large	8 GB Memory, 2 Cores
m5d.xlarge	16 GB Memory, 4 Cores
m5d.2xlarge	32 GB Memory, 8 Cores

Cluster consists of:

- 1 Driver node
- 0 – n Worker nodes

Summary	
4 Workers	122 GB Memory 16 Cores
1 Driver	30.5 GB Memory, 4 Cores
Runtime	13.3.x-scala2.12
Unity Catalog Photon i3.xlarge 10 DBU/h	

Note: Need Spark software too

3-Node Cluster

Worker Node A

Executor A (JVM)

Core 1
(thread)

Memory
Spark cache here

Core 2
(thread)

Local Storage
Disk cache here

Worker Node B

Executor B (JVM)

Core 3
(thread)

Memory
Spark cache here

Core 4
(thread)

Local Storage
Disk cache here

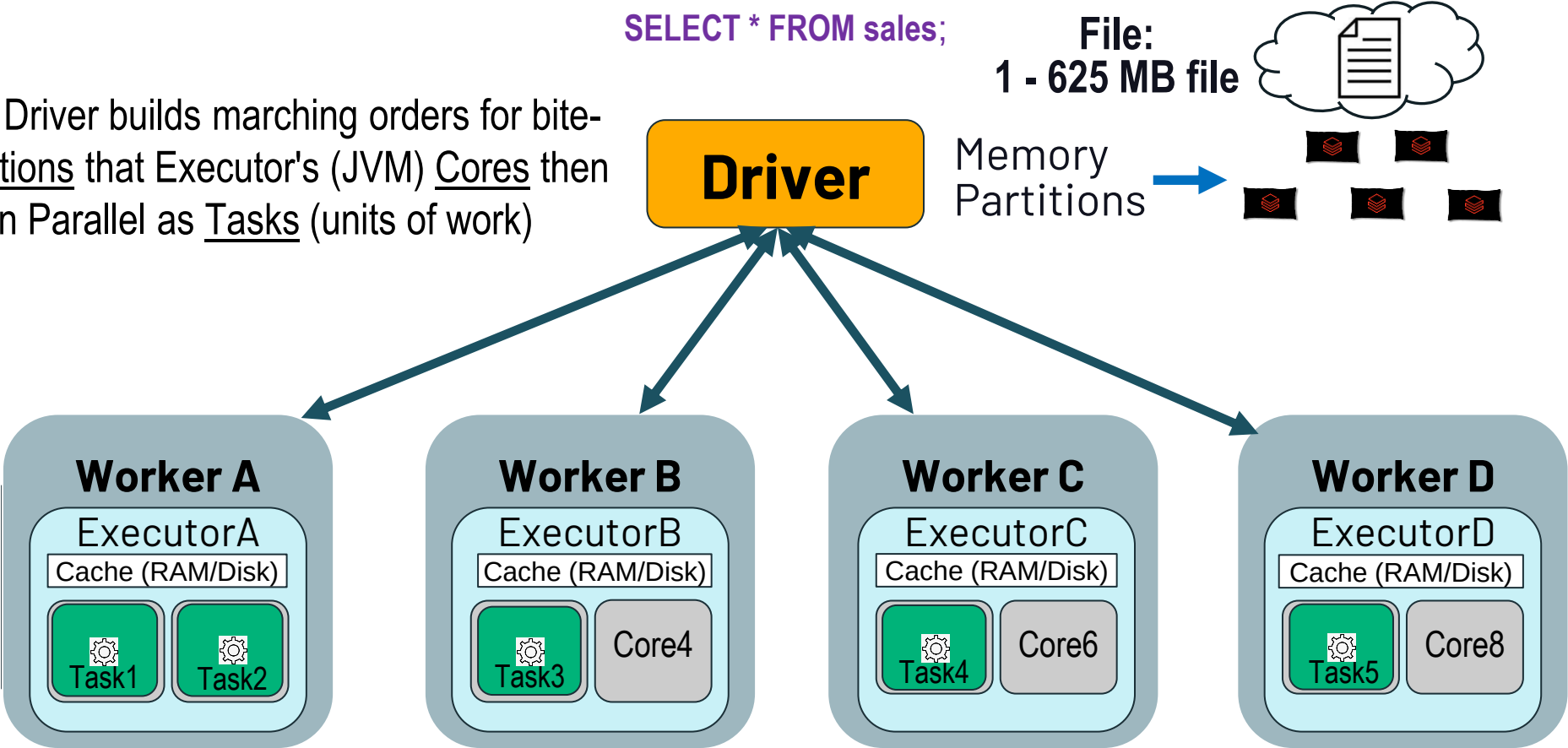
Driver
Node



Spark Cluster Execution

Task = Running a Memory Partition on Core within Executor

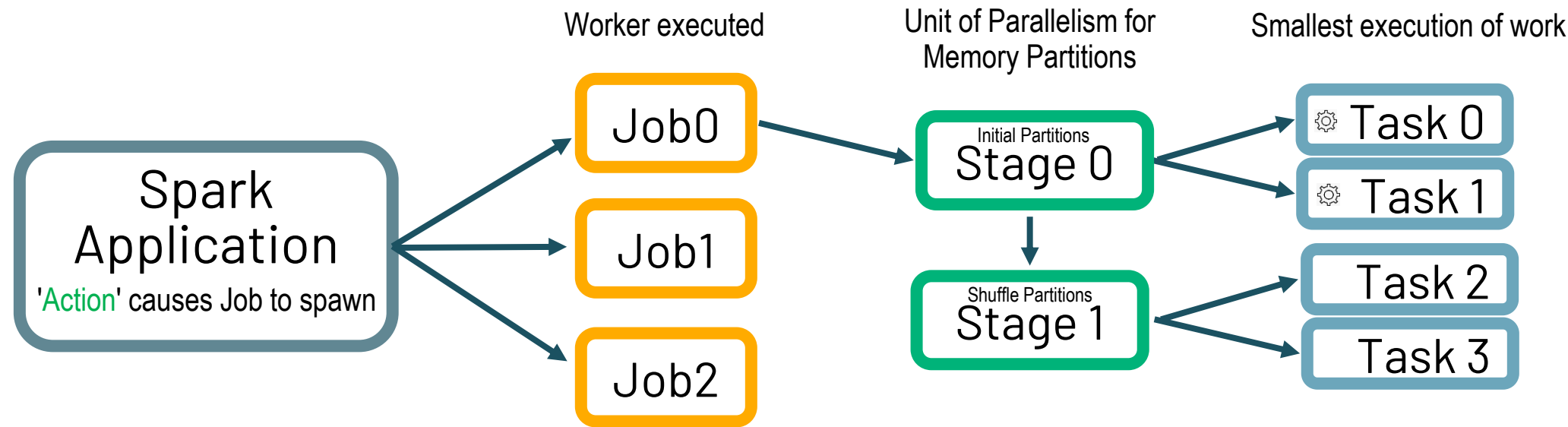
Client runs query. Driver builds marching orders for bite-size Memory Partitions that Executor's (JVM) Cores then creates and runs in Parallel as Tasks (units of work)



<https://databricks.atlassian.net/wiki/spaces/UN/pages/39034498/Worker+disk+layout+http+go+workerdisks>



Action → Job(s) – Stage(s) – Task(s)



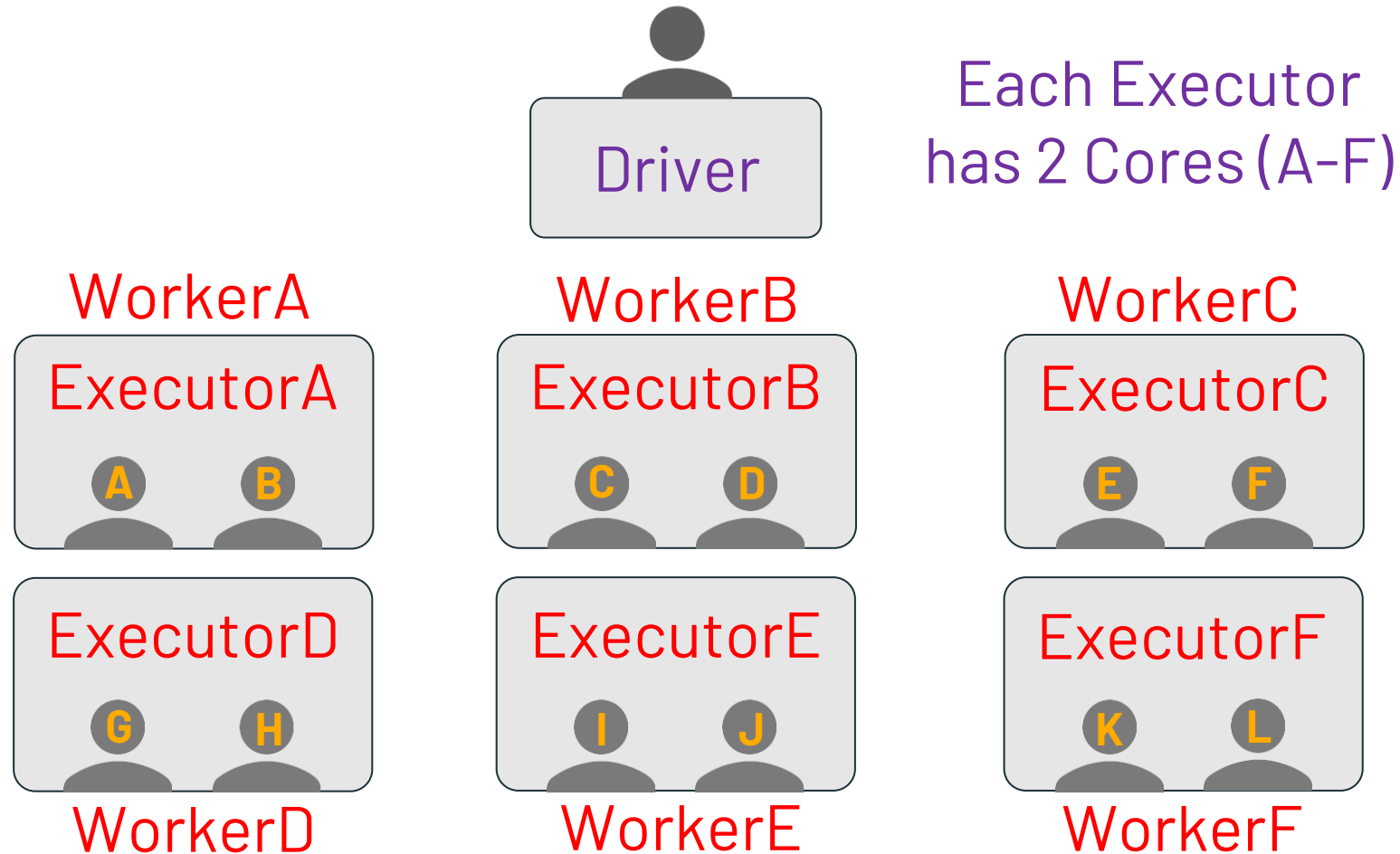
```
trafficDF1 = (eventsDF.groupBy(col("traffic_source"), window(col("createdAt"), "1 hour"))
    .agg(approx_count_distinct("user_id").alias("active_users"))
    .select("traffic_source", "active_users", hour(col("window.start")).alias("hour"))
    .orderBy("hour", "active_users"))
display(trafficDF1)
```

Spark UI

<u>Job Id (Job Group)</u> ▼	<u>Stages:</u> Succeeded/Total	<u>Tasks (for all stages):</u> Succeeded/Total
194 (3814184425400510911_5162481	3/3	10/10

An **Action** spawns Job(s)

Use Case: Assume following: Cluster (Driver, 6 Worker/Executors, 12 Cores)





Before we Begin:

What are Actions and Transformations?

- An **Action** spawns one or more **Jobs**
- A **Job** occurs when the Driver hands off the heavy lifting to Worker(s) to process
- A **Transformation** are functions that return another **RDD**
- **Transformations** are lazy. They do not spawn a Job until **Action** occurs

Common Actions: (Python/SQL)

display(), count(), collect(), show(), write()

CREATE TABLE, SELECT, UPDATE, INSERT

Common Transformations: (Python/SQL)

select(), drop(), limit(), filter(), withColumn()

CREATE VIEW



RDD = Resilient Distributed Dataset
Basic object that Spark manipulates, just an Array of data →

```
rdd1 = sc.textFile("dbfs:/FileStore/tables/input.txt")  
rdd1.take(3)
```

```
Out[7]: ['id,firstname,age,city,profession,salary',  
        '100,Brianna,32,SanFernando,doctor,1540',  
        '101,Mariele,18,Cairo,developer,5800']
```

Scenario 1: Filter out brown rows from DataFrame



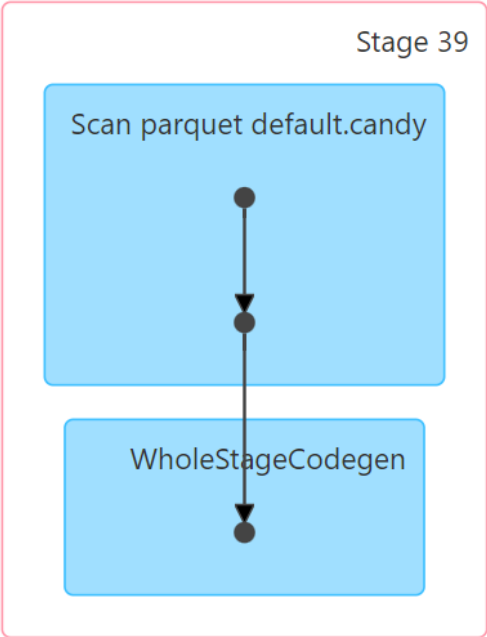
```
1 df = spark.table("candy")
2 df.filter("color != 'brown']").collect()
```

▼ (1) Spark Jobs

▼ Job 25 [View](#) (Stages: 1/1)

Stage 39: 4/4 ⓘ

Tasks

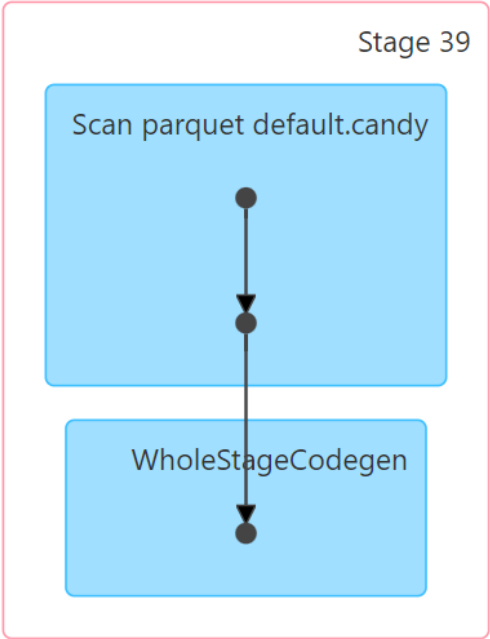
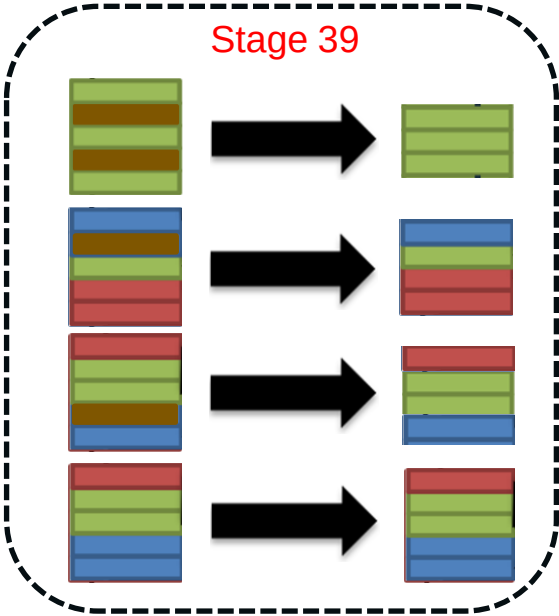


1 Job, 1 Stage, 4 Tasks (Partitions)

Description	Submitted	Duration	Tasks: Succeeded/Total
df = spark.table("candy") df.filter("color != '... collect at <command-1951720335359827>:2	2022/02/25 10:59:24	50 ms	4/4

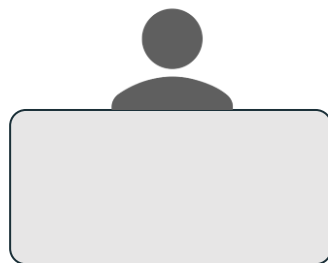
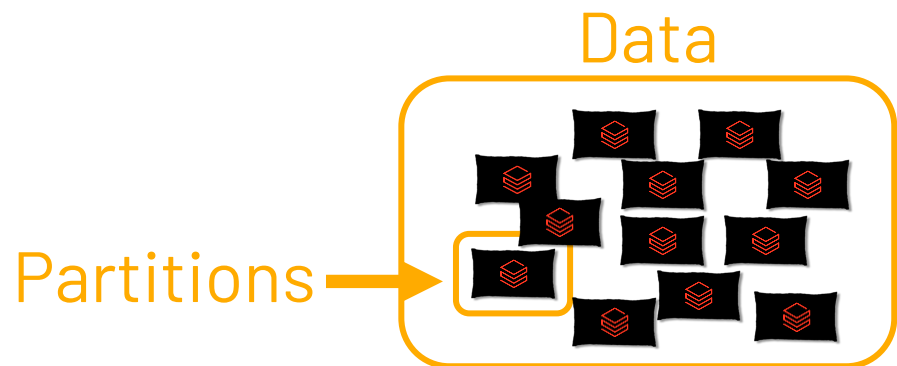
filter() is Narrow transformation (Process everything in same partition)

```
df.filter("color != 'brown']").collect()
```

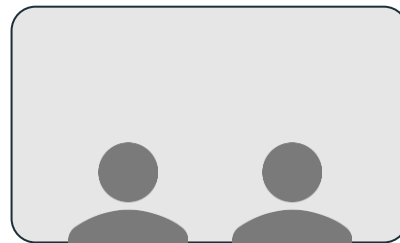
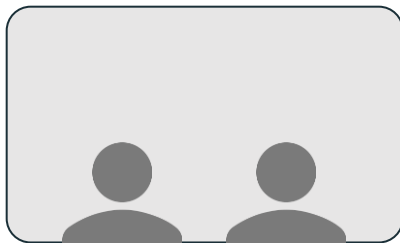
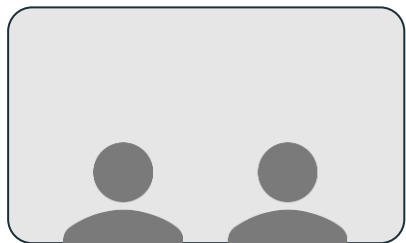
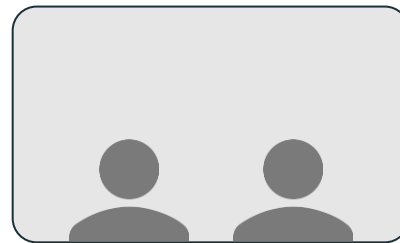
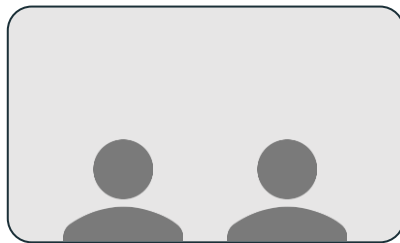
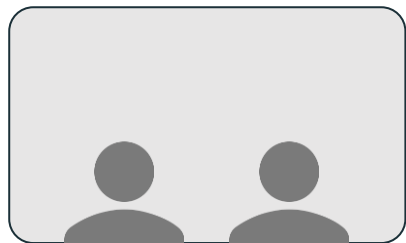


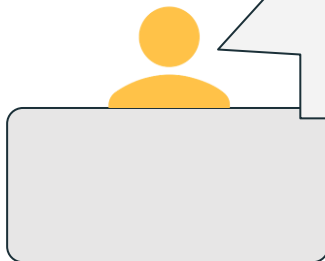
Start with 4 Partitions, end with 4 Partitions

Description	Submitted	Duration	Tasks: Succeeded/Total
df = spark.table("candy") df.filter("color != '... collect at <command-1951720335359827>:2	2022/02/25 10:59:24	50 ms	4/4



Driver assigns Tasks that create file(s)
roughly equal sized Partitions
(XML, gzip not splittable)

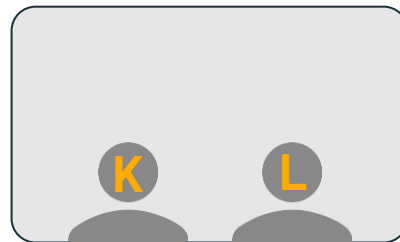
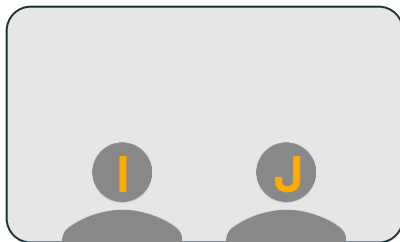
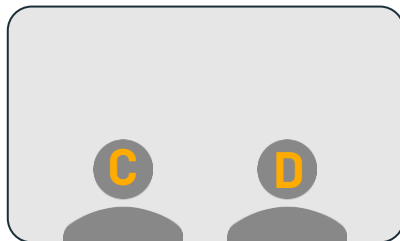
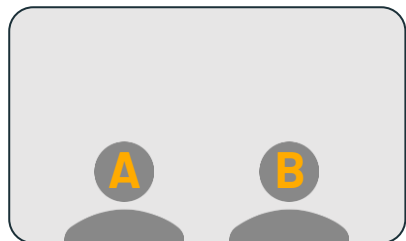




Schedule work

Core A get Partition # 0
Core B get Partition # 1
Core C get Partition # 2

...

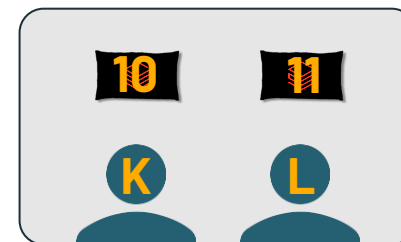
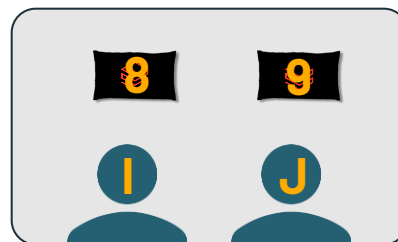
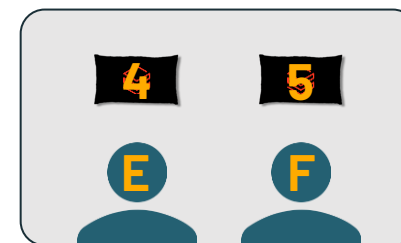
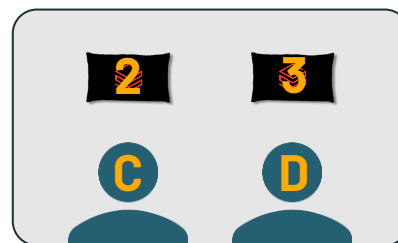
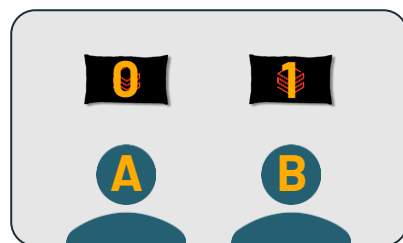


```
df.filter("color != 'brown'").collect()
```

Opps, we still have 4 more Partitions to work on once get 12 current Partitions processed. This will require 2nd iteration. More on that later

12 13
14 15

Eliminate the brown rows and return result set to me



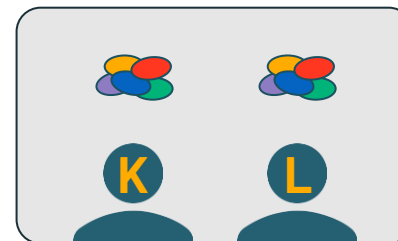
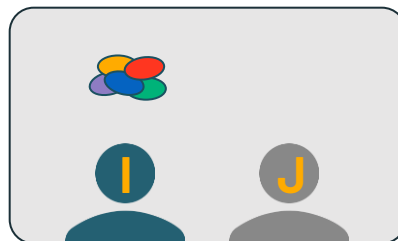
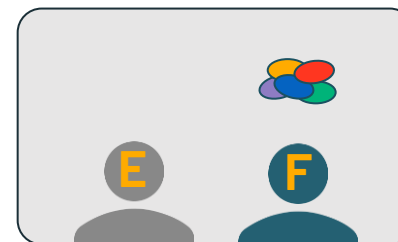
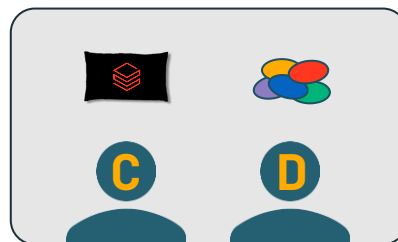
Cores idle so I can Schedule more work

Core A, E, H, J, get Partitions 12, 13, 14, 15

12 13
14 15

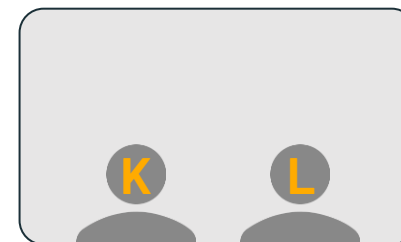
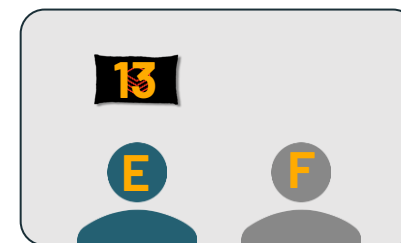
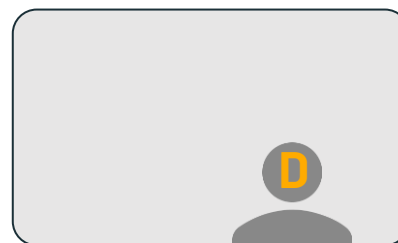
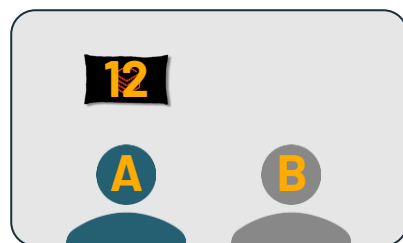
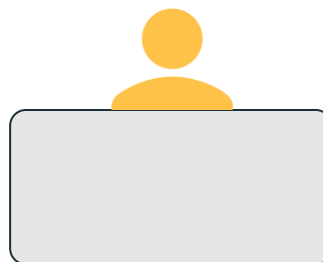
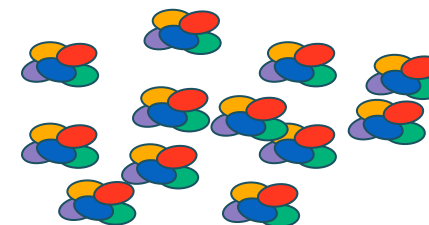


Intermediate
result sets
returned by
Core A,E,H,J



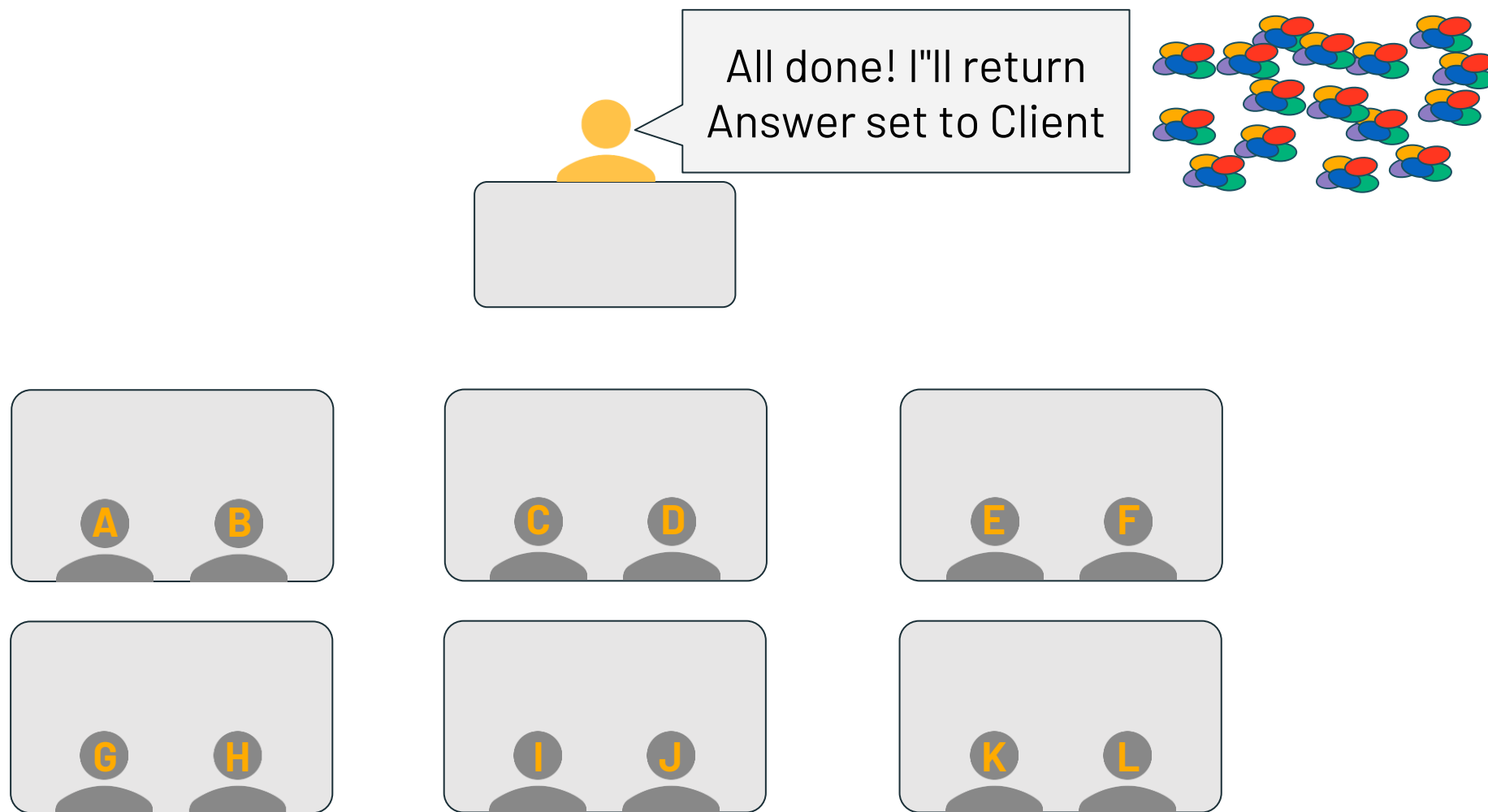
```
df.filter("color != 'brown'").collect()
```

Intermediate result sets returned by
other 12 Cores



```
df.filter("color != 'brown']").collect()
```

Cores A, E, H, J finish their 2nd iteration.
All 16 result sets merged



Scenario 2: Count total rows by Color in DataFrame



```
6 df = spark.table("candy")
7 df.groupBy("color").count().collect()
```

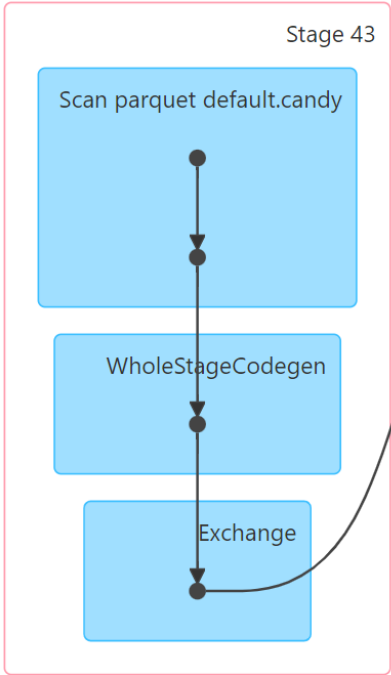
▼ (1) Spark Jobs

▼ Job 28 View (Stages: 2/2)

Stage 43: 4/4 ⓘ

Stage 44: 200/200 ⓘ

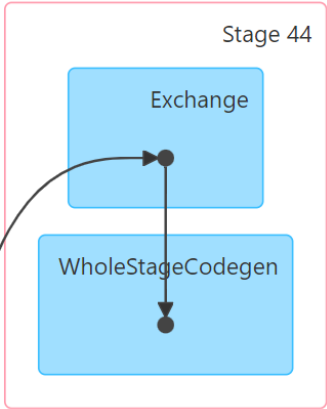
Local count



Shuffle Write

568.0 B

Global count



Shuffle Read

568.0 B

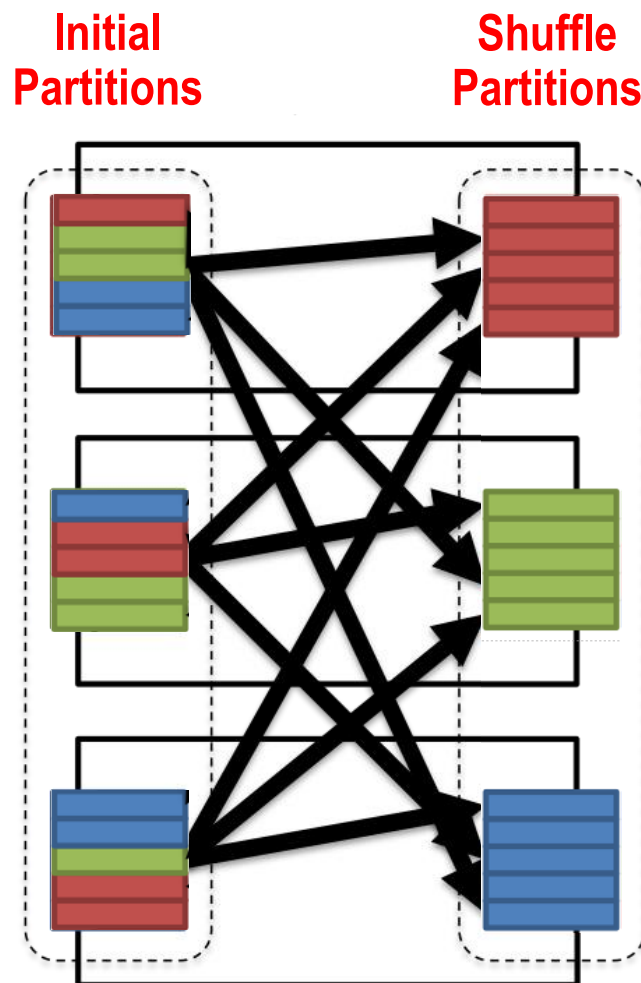
Start with 4 Partitions, end with variable # Shuffle Partitions

Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
# Scenario 02: Count all rows # 'count' is Wide... collect at <command-1951720335359828>:7 +details	2022/02/25 11:44:06	2 s	200/200	2		568.0 B	
			Shuffle Partitions				
# Scenario 02: Count all rows # 'count' is Wide... collect at <command-1951720335359828>:7	2022/02/25 11:44:06	0.1 s	4/4	1			568.0 B
			Initial Partitions				

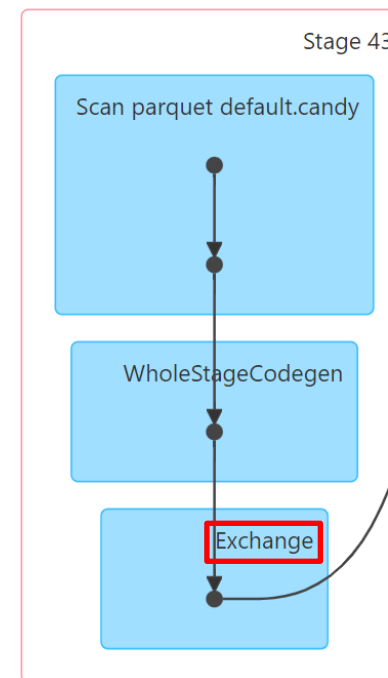
groupBy() is Wide transformation (Need create/distribute new Partitions)

Every time have a wide transformation, new Stage is spawned

Then new Shuffle Partitions are created and executed in that new Stage



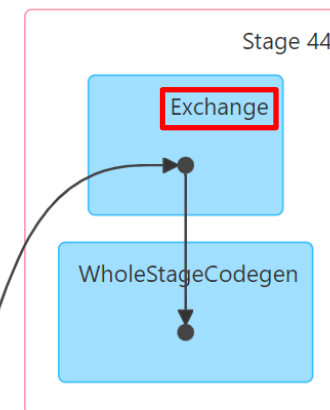
Local count



Shuffle Write

568.0 B

Global count

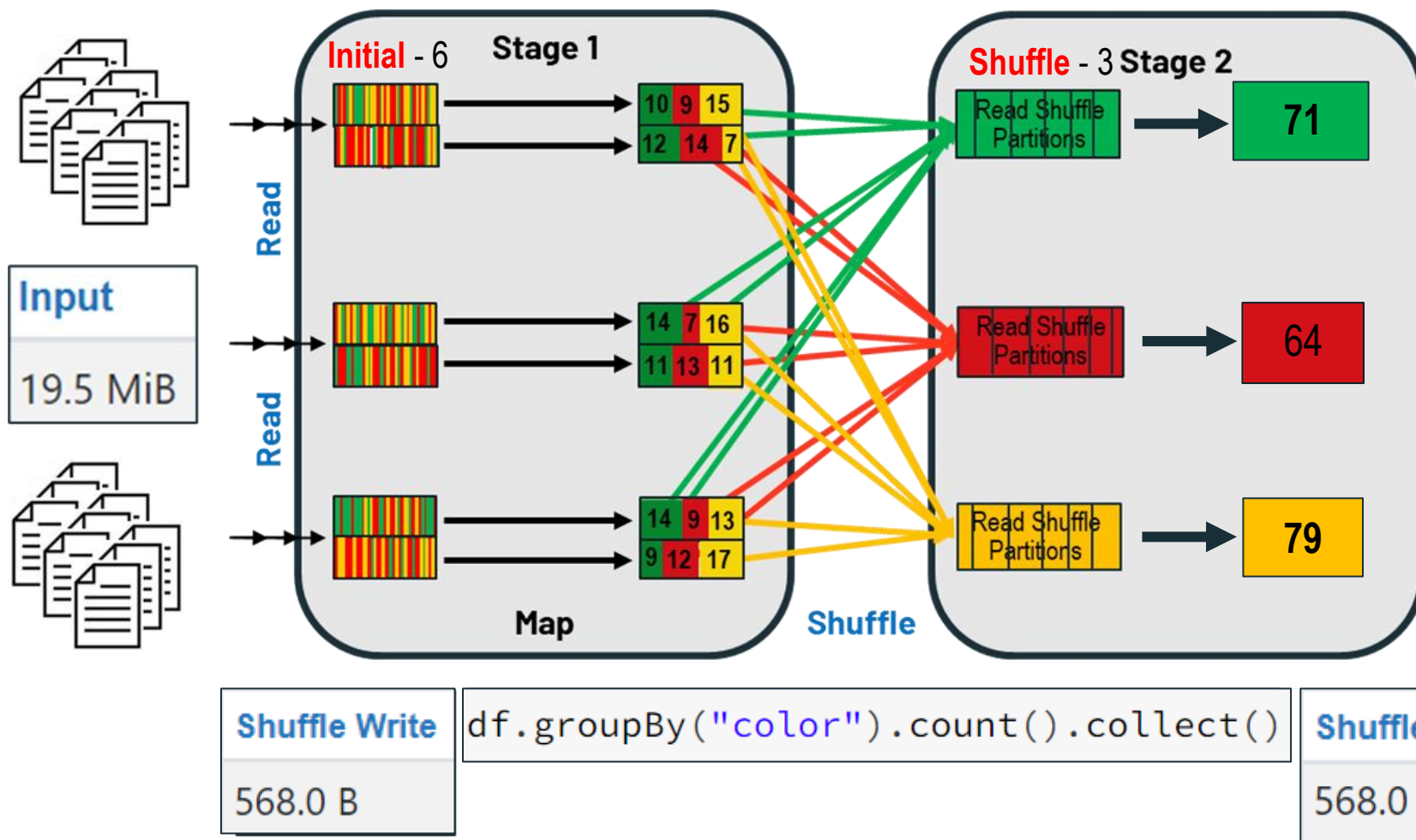


Shuffle Read

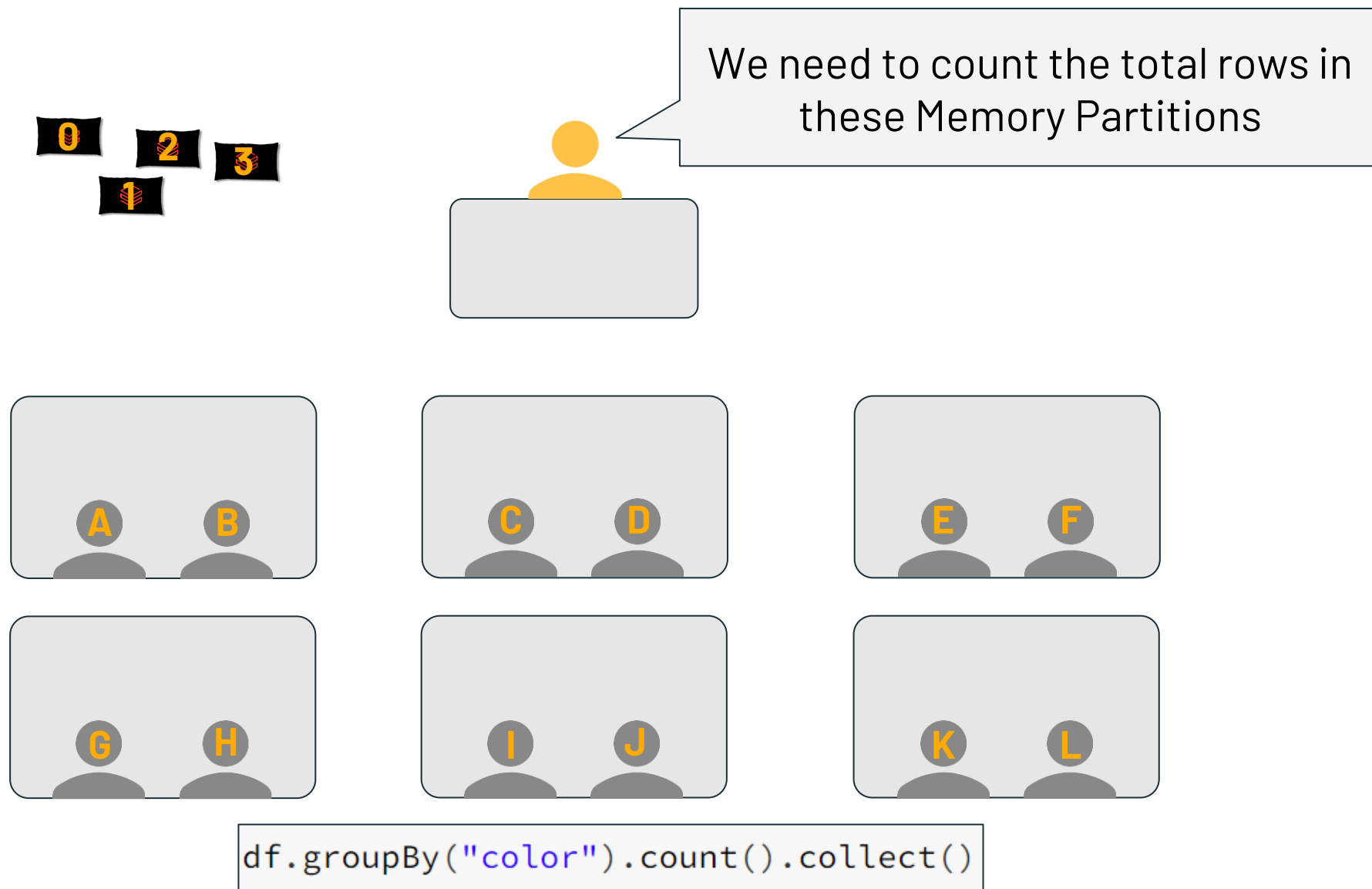
568.0 B

Before we Begin: What is a Shuffle/Exchange?

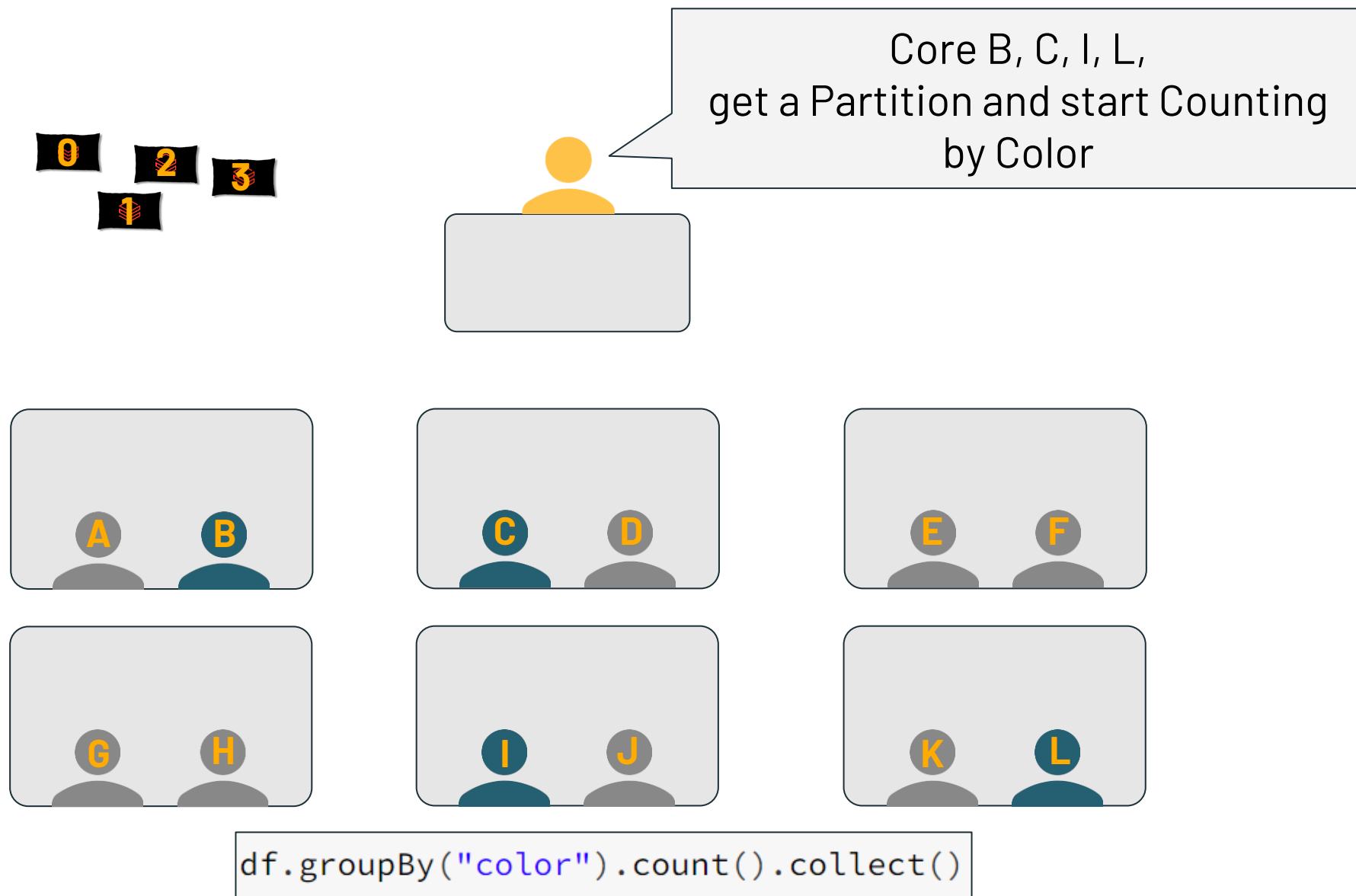
Creating new (Shuffle) Memory partitions in new Stage (and potentially in different Core)



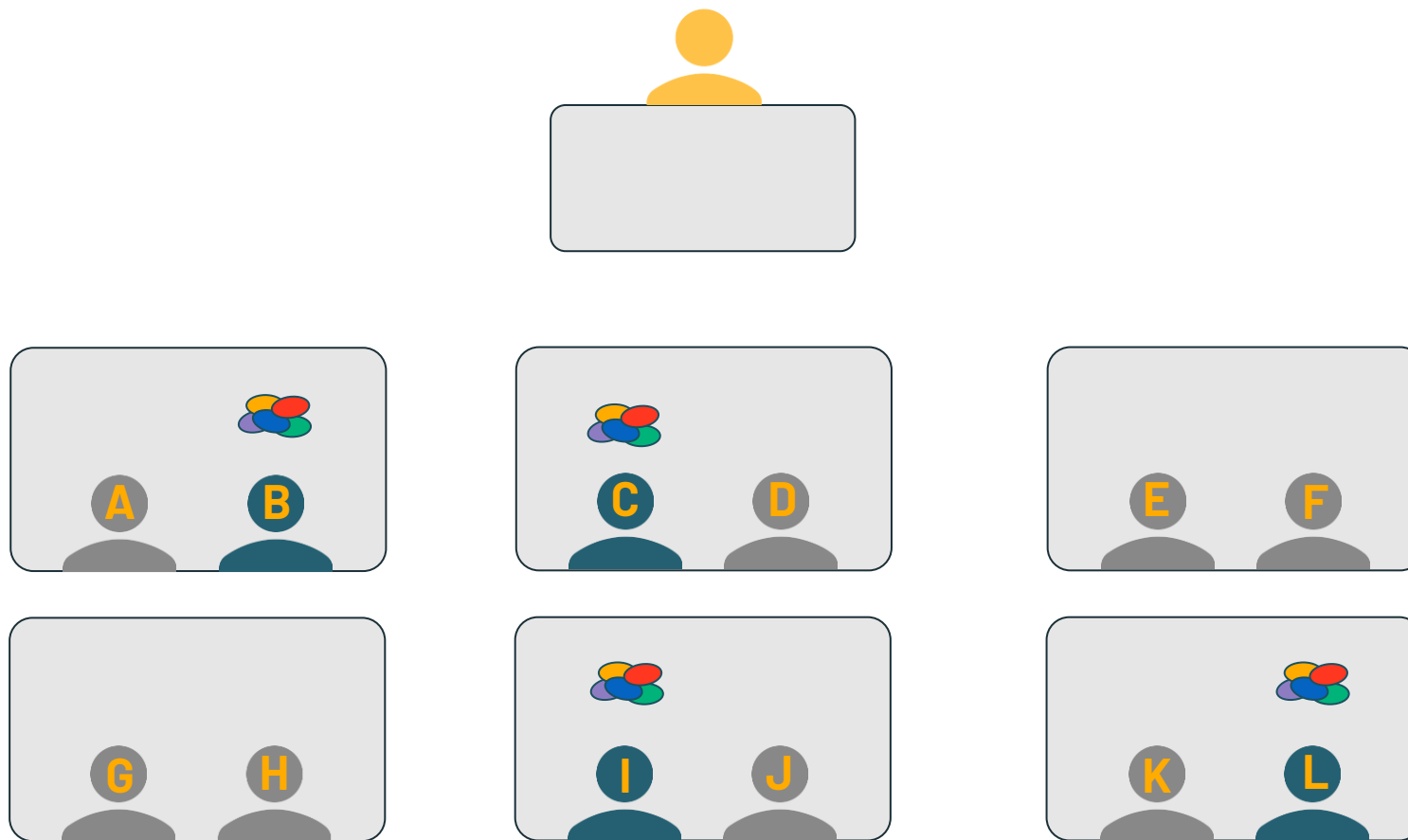
Stage 1: Local Count



Stage 1: Local Count

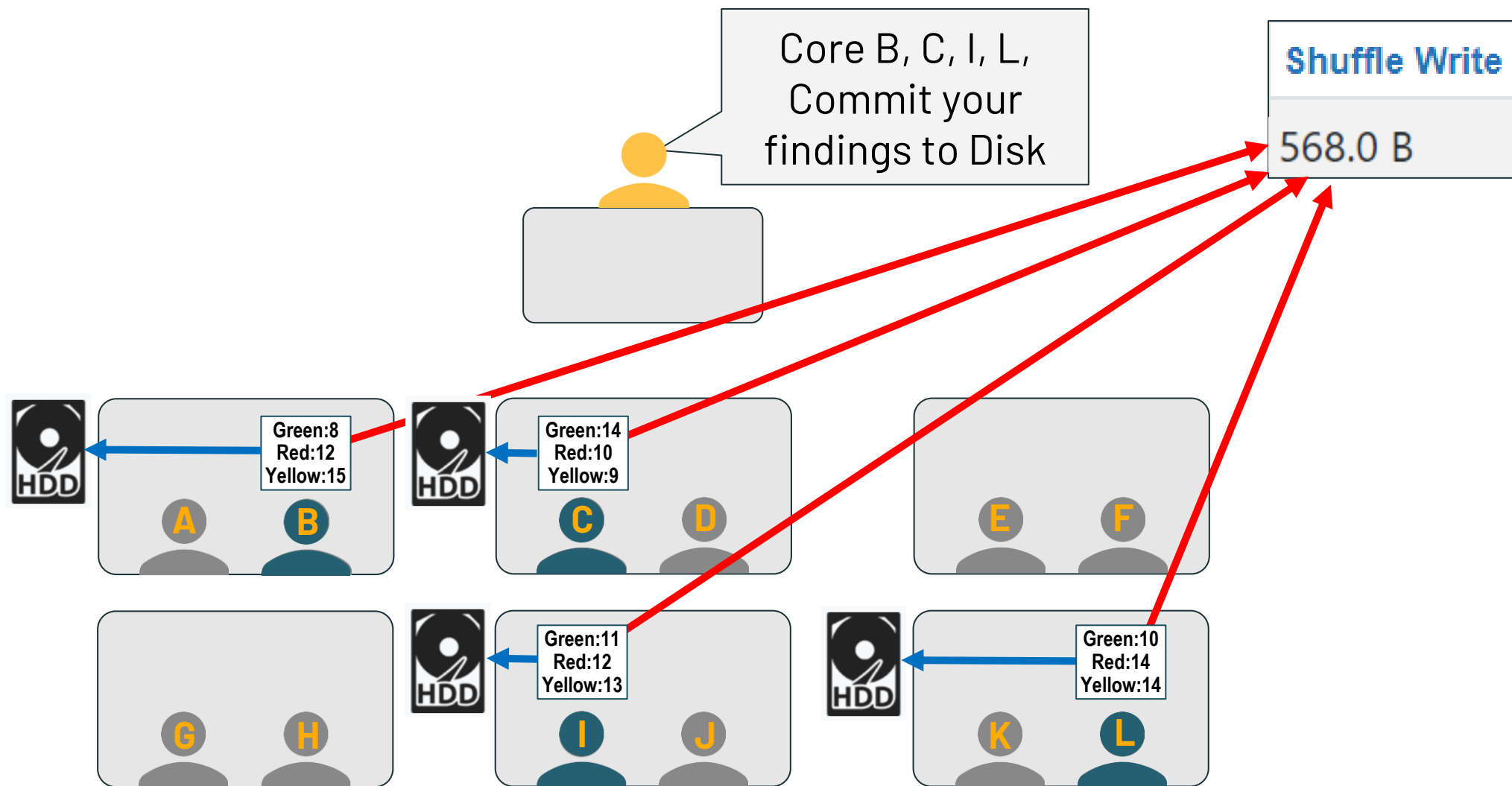


Stage 1: Local Count



```
df.groupBy("color").count().collect()
```

Stage 1: Local Count



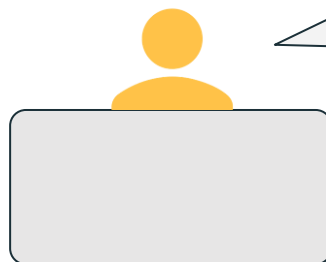
```
df.groupBy("color").count().collect()
```

Stage 1: Local Count

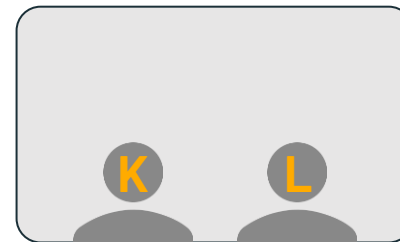
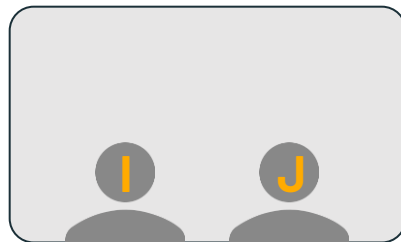
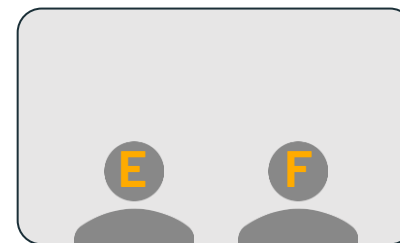
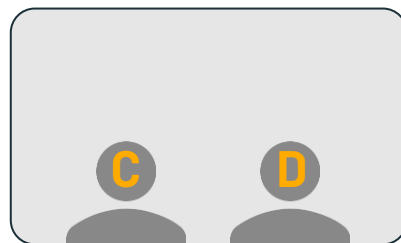
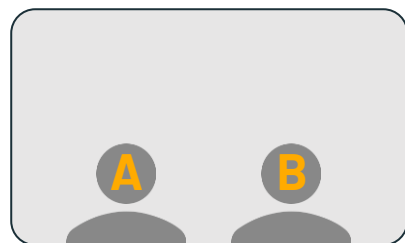
```
(2) HashAggregate [codegen id : 1]  
Input: []  
Keys: []  
Functions [1]: [partial_count(1) AS count#3363L]  
Aggregate Attributes [1]: [count#3362L]
```

Shuffle Write

568.0 B

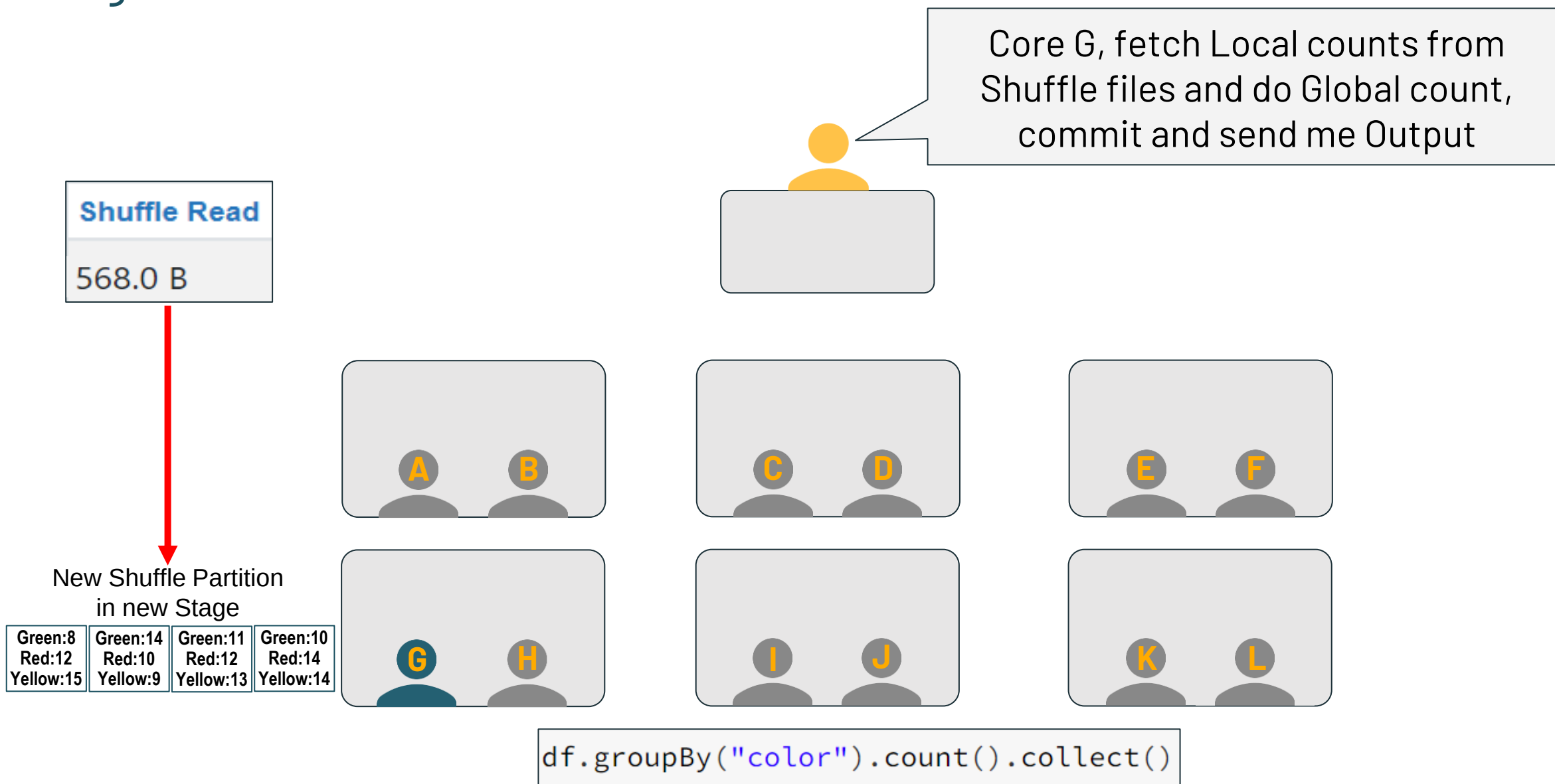


Stage 1 is complete!



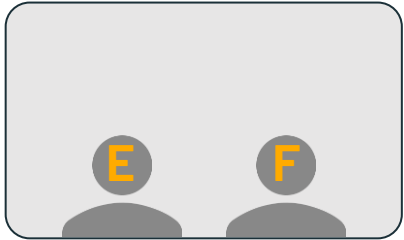
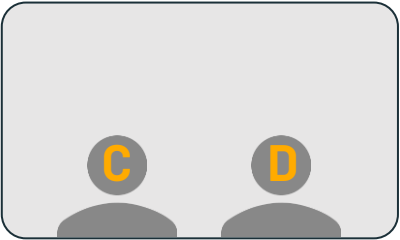
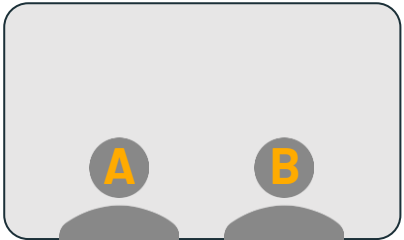
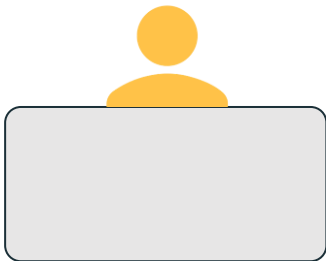
```
df.groupBy("color").count().collect()
```

Stage 2: Global Count

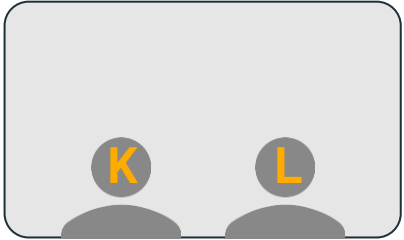
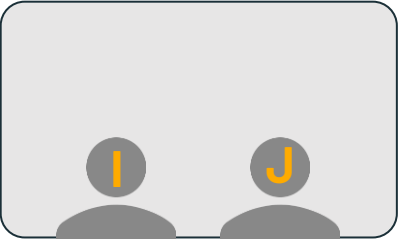
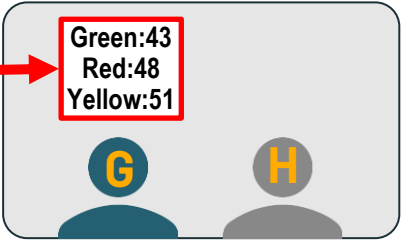


Stage 2: Global Count

```
(8) HashAggregate
Input [1]: [count#3363L]
Keys: []
Functions [1]: [finalmerge_count(m
```

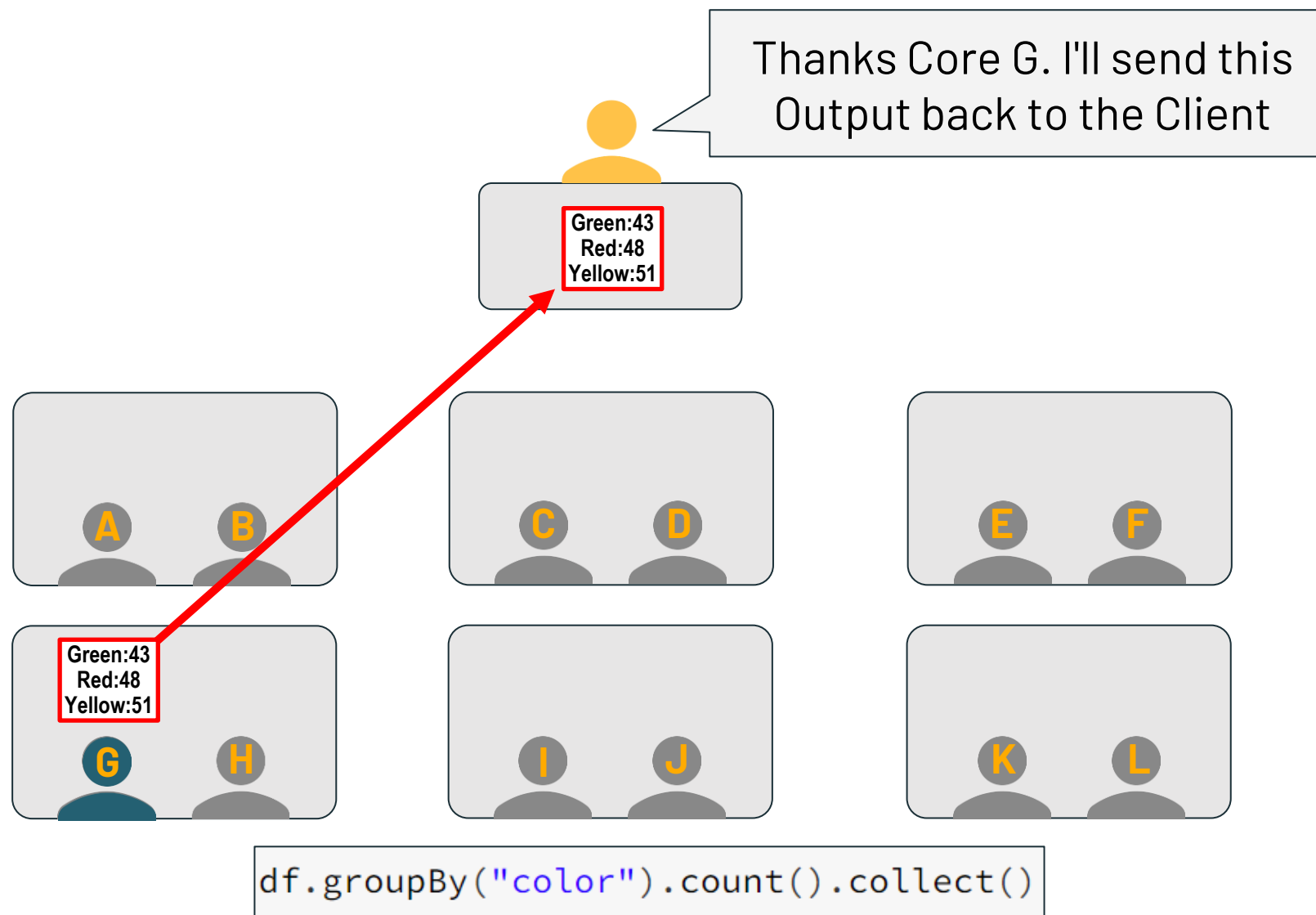


Green:8	Green:14	Green:11	Green:10
Red:12	Red:10	Red:12	Red:14
Yellow:15	Yellow:9	Yellow:13	Yellow:14



```
df.groupBy("color").count().collect()
```

Stage 2: Global Count



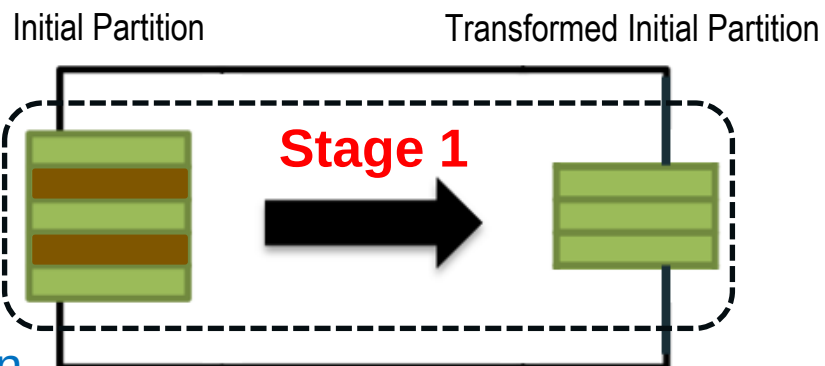


In Review: Narrow vs Wide Transformations

Narrow Transformations
Partitions stay in same Stage

`df.filter("color != 'brown').collect()`

select
union
filter
where
coalesce
withColumn



 = Stage

Wide Transformations spawn
new Stages/Shuffle Partitions

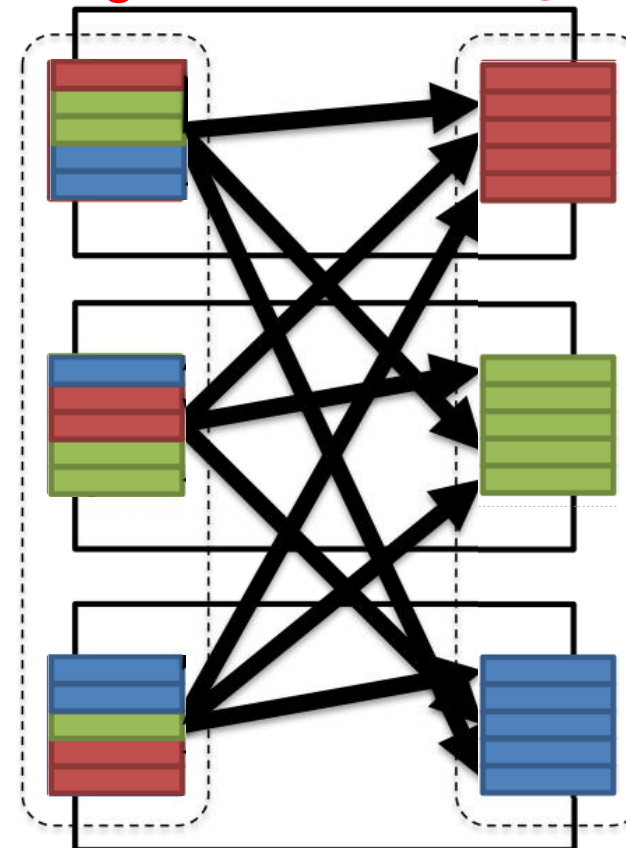
`df.groupBy("color").count().collect()`

Initial Partitions

Stage 1

New Shuffle Partitions

Stage 2



join
sort
distinct
groupBy
order by
count
repartition
dropDuplicates

Review: Partitions in Stages

There are two kinds of Memory Partitions

1. [Initial Memory Partitions](#) (roughly 125MB max)
2. [Shuffle Memory Partitions](#) (variable size)

```
display(initialDF
  .withColumn("first", upper(col("article").substr(0,1)) )
  .where( col("first").isin("A","B","C","D","E","F","G","H","I","J","K","L"))
  groupBy(col("project"), col("first")).sum()
  .drop("sum(bytes_served)")
  orderBy(col("first"), col("project"))
  .select(col("first"), col("project"), col("sum(requests)").alias("total"))
  .filter(col("total") > 10000))
```

Initial

Stage 54

Shuffle

Stage 55

Shuffle

Stage 56

▼ Job 43 [View](#) (Stages: 3/3)

Stage 54: 8/8 [i](#)

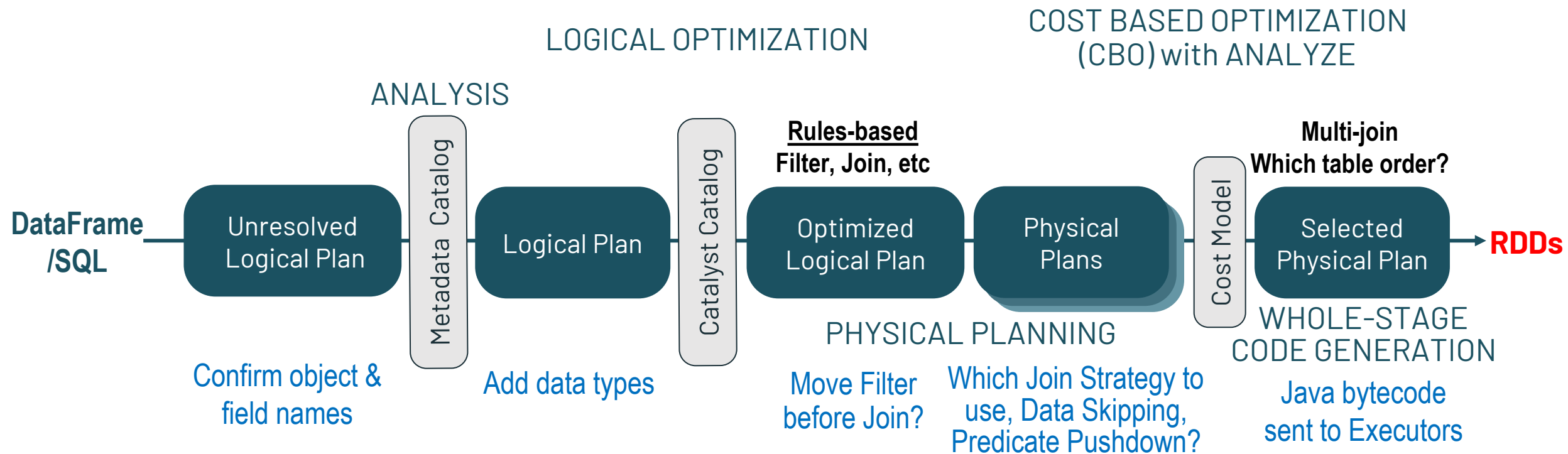
Stage 55: 900/900 [i](#)

Stage 56: 1/1 [i](#)

All partitions (Initial and Shuffle) run in Stage.
Initial Memory Partitions run in 1st Stage of Job

Wide transformations (ie: 'groupBy', 'orderBy') create new Shuffle Partitions
New Stages are created so these Partitions can run

DataFrame (DataSet, SQL) Optimization



RDD = Resilient Distributed Dataset
(ie: bag of data, collection of objects)

```
rdd1 = sc.textFile("dbfs:/FileStore/tables/input.txt")
rdd1.take(3)

Out[7]: ['id,firstname,age,city,profession,salary',
         '100,Brianna,32,SanFernando,doctor,1540',
         '101,Mariele,18,Cairo,developer,5800']
```

WholeStageCodeGen = Java byte code

```
1 df = spark.read.format("delta").load(events_path)
2 df.explain(mode = 'CODEGEN')
```

df: pyspark.sql.dataframe.DataFrame = [device: string, ecommerce: struct ... 8 more fields]

Found 1 WholeStageCodeGen subtrees.

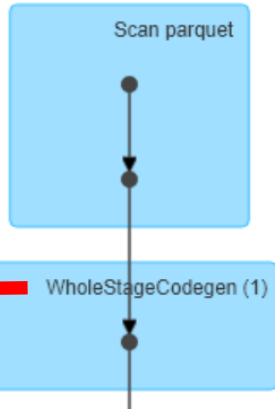
== Subtree 1 / 1 (maxMethodCodeSize:1526; maxConstantPoolSize:204(0.31% used); numInnerClasses:0) ==

*(1) ColumnarToRow

+ FileScan parquet [device#423,ecommerce#424,event_name#425,event_previous_timestamp#426L,event_timestamp#427L,geo#428,items#429,traffic_source#430,user_first_touch_timestamp#431L,user_id#432] Batched: true, DataFilters: [], Format: Parquet, Location: PreparedDeltaFileIndex[dbfs:/user/mark.ott@databricks.com/dbacademy/aspwd/datasets/events/events..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<device:string,ecommerce:struct<purchase_revenue_in_usd:double,total_item_quantity:bigint,u...

Generated code:

```
/* 001 */ public Object generate(Object[] references) {
/* 002 */   return new GeneratedIteratorForCodegenStage1(references);
/* 003 */ }
/* 004 */
/* 005 */ // codegenStageId=1
/* 006 */ final class GeneratedIteratorForCodegenStage1 extends org.apache.spark.sql.execution.BufferedRowIterator {
/* 007 */   private Object[] references;
/* 008 */   private scala.collection.Iterator[] inputs;
/* 009 */   private int columnartorow_batchIdx_0;
/* 010 */   private org.apache.spark.sql.catalyst.expressions.codegen.UnsafeRowWriter[] columnartorow_mutableStateArray_3 = new org.apache.spark.sql.catalyst.expressions.codegen.UnsafeRowWriter[4];
/* 011 */   private org.apache.spark.sql.vectorized.ColumnarBatch[] columnartorow_mutableStateArray_1 = new org.apache.spark.sql.vectorized.ColumnarBatch[4];
```



Note: this is Spark 2.x plan. Spark 3 looks different

Explain Plan

```
spark.sql("SELECT e.last_name, d.dept
          FROM emp_view e INNER JOIN dept_view d
          ON e.dept=d.dept
          WHERE e.dept = 301").explain(True)
```

1

```
== Parsed Logical Plan ==
'Project ['e.last_name, 'e.first_name, 'd.dept, 'e.dept, 'd.dept]
+- Filter ('e.dept = 301)
   +- 'Join Inner, ('e.dept = 'd.dept)
      :- 'UnresolvedRelation emp_view, e
      +- 'UnresolvedRelation dept_view, d
```

As you wrote it. Filter after JOIN

2

```
== Analyzed Logical Plan ==
last_name: string, first_name: string, dept: string, dept: string, dept: string
Project [last_name#81, first_name#78, dept#96, dept#76, dept#96]
+- Filter (cast(dept#76 as double) = cast(301 as double))
   +- Join Inner, (dept#76 = dept#96)
      :- SubqueryAlias e
      :   +- SubqueryAlias emp_view
      :     +- LogicalRDD [birth#75, dept#76, emp#77, first_name#78, hire#79, job#80, mgr#82, salary#83]
      :   +- SubqueryAlias d
      :     +- SubqueryAlias dept_view
      :       +- LogicalRDD [budget#95, dept#96, dept_name#97, mgr#98]
```

Add in data types and CAST as needed. Filter still after JOIN

3

```
== Optimized Logical Plan ==
Project [last_name#81, first_name#78, dept#96, dept#76, dept#96]
+- Join Inner, (dept#76 = dept#96)
   :- Project [dept#76, first_name#78, last_name#81]
   :   +- Filter (isNotNull(dept#76) && (cast(dept#76 as double) = 301.0))
   :     +- LogicalRDD [birth#75, dept#76, emp#77, first_name#78, hire#79, job#80, mgr#82, salary#83]
   +- Project [dept#96]
      +- Filter ((cast(dept#96 as double) = 301.0) && isNotNull(dept#96))
         +- LogicalRDD [budget#95, dept#96, dept_name#97, mgr#98]
```

Move FILTER before JOIN

4

```
== Physical Plan ==
*Project [last_name#81, first_name#78, dept#96, dept#76, dept#96]
+- *SortMergeJoin [dept#76], [dept#96], Inner
   :- *Sort [dept#76 ASC NULLS FIRST], false, 0
   :   +- Exchange hashpartitioning(dept#76, 200)
   :     +- *Project [dept#76, first_name#78, last_name#81]
   :       +- *Filter (isNotNull(dept#76) && (cast(dept#76 as double) = 301.0))
   :         +- Scan ExistingRDD[birth#75,dept#76,emp#77,first_name#78,hire#79]
```

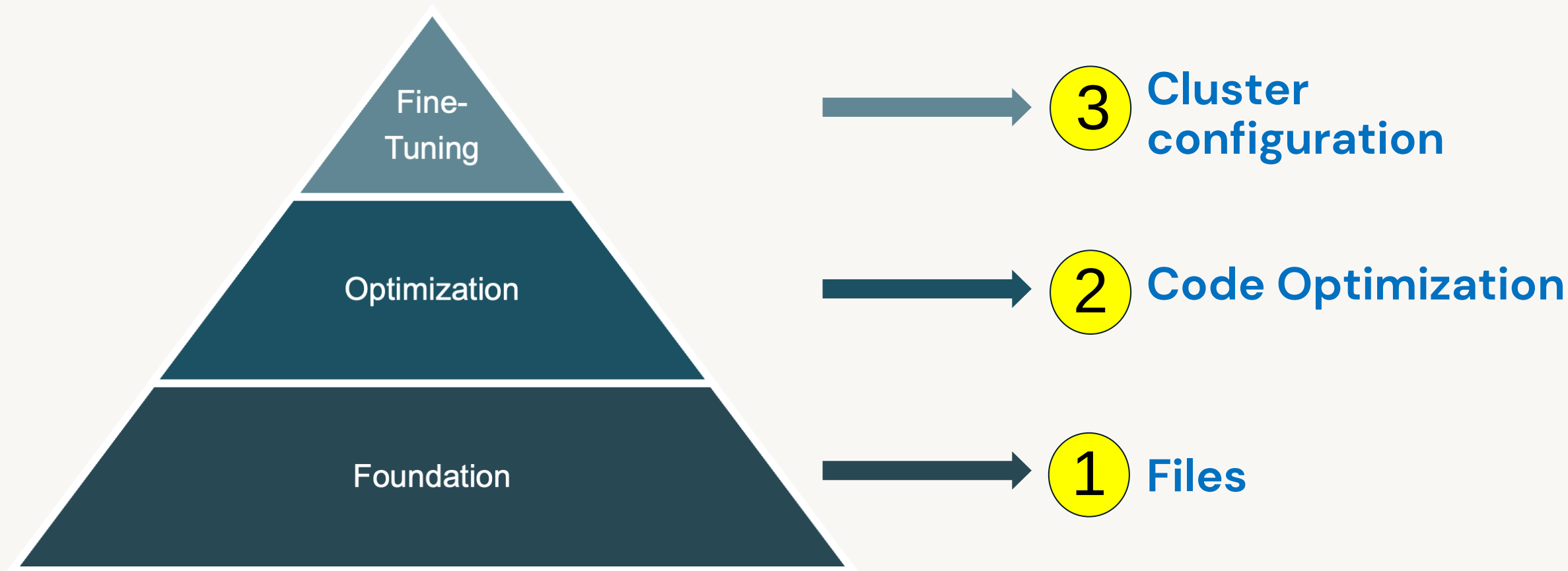
Determine Join strategy

Common Performance Bottlenecks

Encountered with any big data or MPP system

Bottleneck	Details
Small File Problem (Storage)	• Listing and metadata operation for too many small files can be expensive • Can also result in throttling from cloud storage I/O limits
Data Skew	• Large amounts of data skew can result in more work handled by a single executor • Even if data read is not skewed, certain transformations can lead to in-memory skew
Processing More Than Needed	• Traditional data lake platforms often require rewriting entire datasets or partitions
Resource Contention	• Processing large ingestion, ETL jobs at the same time as ad-hoc and BI queries results in slow query performance without cluster isolation

Building Performant Analytics



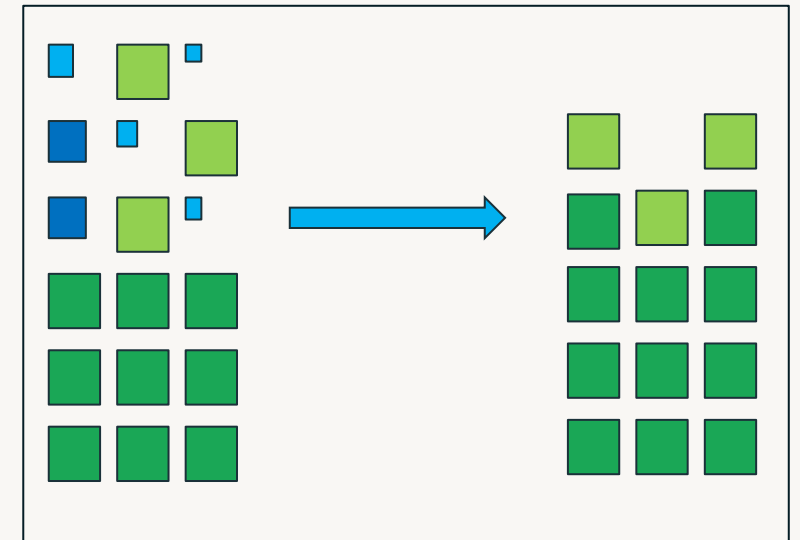
1 Files



Storage – Avoiding the small-size file problem

Automatically handle this common performance challenge in Data Lakes

- Too many small files greatly increases overhead for reads
- Too few large files reduces parallelism on reads
- Disk Over-partitioning is a common problem
- Databricks will automatically tune the size of Delta Lake tables
- Databricks will automatically compact small files (once configured) on write with auto-optimize



<https://learn.microsoft.com/en-us/azure/databricks/delta/tune-file-size>

Storage – Delta **Optimize** options for SQL tables

Delta **Optimize** and/or **Auto-Optimize** can afford the ability to focus on [potentially] bigger problems

OPTIMIZE TABLE student
ZORDER BY (id, ...)

Execute OptimizeTableCommandEdge (1) +	
Metric	Value
total bytes in files merged by ZOrderBy	3,535
total number of files merged by ZOrderBy	4

Creates up to **1GB** files from existing files (when run)
Optionally ZORDER by column value within file
Note: ZOrder columns must have Data Skipping enabled

<https://docs.databricks.com/spark/latest/spark-sql/language-manual/delta-optimize.html>

Can be enabled globally for all new SQL tables

CREATE TABLE student (id INT, name STRING, age INT)
TBLPROPERTIES (delta.autoOptimize.**optimizeWrite** =
true, delta.autoOptimize.**autoCompact** = **true**)

As new files written, files coalesced into **128MB** size

<https://docs.databricks.com/delta/optimizations/auto-optimize.html>

And you can use both. For example, suppose Streaming job ingests 1000 1MB file every hour. Use **Auto-Optimize** to create just 8 files (roughly 128 MB each) every hour. Then end of each week, **Optimize** those 1,344 files and coalesce into 172 1GB files

Data Skipping with Delta Lake and Partitioning tables

1. Via Delta Lake statistics
2. Via Disk Partitioning (compatible with any file format)
3. Via Delta Lake Z-Order
4. Via Delta Lake Liquid Clustering

```
▶ {"dataChange": true, "modificationTime": 1643559780000, "path": "part-00000-f446c24f-6b66-4953-94c5-e09a43819ff0-c000.snappy.parquet", "size": 1454, "stats": "{\"numRecords\":10,\"minValues\":{\"dept_num\":100,\"dept_name\":\"New\",\"budget\":0,\"mgr\":801},\"maxValues\":{\"dept_num\":501,\"dept_name\":\"technical operations\",\"budget\":\"New\",\"mgr\":1099},\"nullCount\":{\"dept_num\":2,\"dept_name\":0,\"budget\":0,\"mgr\":0}}\", \"tags\": {\"INSERTION_TIME\": \"1643559780000000\", \"OPTIMIZE_TARGET_SIZE\": \"268435456\"}}
```

.json file

Delta Lake Statistics

- Databricks Delta Lake collects stats about the first N columns
 - `dataSkippingNumIndexedCols = 32`
- These stats are used in queries
 - Metadata only queries: `select max(col) from table`
 - Queries just the Delta Log (JSON), doesn't need to look at the files if `col` has stats
 - Allows us to skip files
 - Partition Filters, **Data Filters**, Pushed Filters apply in that order
 - TimeStamp and String types aren't always very useful
 - Precision/Truncation prevent exact matches, have to fall back to files sometimes
- Avoid collecting stats on long strings
 - Put them outside first 32 columns or collect stats on fewer columns
 - `alter table change column col after col32`
 - `set spark.databricks.delta.properties.defaults.dataSkippingNumIndexedCols = 3`



```

▶ {"dataChange": true, "modificationTime": 1643559780000, "path": "part-00000-f446c24f-6b66-4953-94c5-e09a43819ff0-
c000.snappy.parquet", "size": 1454, "stats": "{\"numRecords\":10,\"minValues\":
{\\\"dept_num\\\":100,\\\"dept_name\\\":\\\"New\\\",\\\"budget\\\":\\\"0\\\",\\\"mgr\\\":801},\\\"maxValues\\\":{\\\"dept_num\\\":501,\\\"dept_name\\\":\\\"technical
operations\\\",\\\"budget\\\":\\\"New\\\",\\\"mgr\\\":1099},\\\"nullCount\\\":{\\\"dept_num\\\":2,\\\"dept_name\\\":0,\\\"budget\\\":0,\\\"mgr\\\":0}}\", \"tags\":
{\\\"INSERTION_TIME\\\": \\\"1643559780000000\\\", \\\"OPTIMIZE_TARGET_SIZE\\\": \\\"268435456\\\"}}

```

.json file



Delta Lake 'Data Skipping'

- Data Skipping stats collected automatically when you write into a Delta table
- Delta takes advantage of stats (`# numRecords`, `Min`, `Max`, `nullCount`) at query time to provide faster queries by skipping files

Output

id
25
290

```
SELECT id FROM wiki_view WHERE id < 300 ORDER BY id ASC
```

CSV

Scan csv +details	
Stages: 3257.0	192 files 1.28 min
file sorting by size time	0 ms
number of files read	192
filesystem read data size total (min, med, max)	4.4 GiB (45.8 MiB, 558.5 MiB, 567.2 MiB)
filesystem read data size (sampled) total (min, med, max)	4.4 GiB (45.8 MiB, 558.5 MiB, 567.2 MiB)
filesystem read time (sampled) total (min, med, max)	1.5 m (654 ms, 10.7 s, 13.4 s)
metadata time	0 ms
size of files read	4.4 GiB
rows output	2

Parquet

Scan parquet +details	
Stages: 3264.0	48 files 3.34 sec
file sorting by size time	0 ms
number of files read	48
filesystem read data size total (min, med, max)	566.8 KiB (9.9 KiB, 18.0 KiB, 383.1 KiB)
scan time total (min, med, max)	8.9 s (511 ms, 1.0 s, 1.2 s)
estimated repeated reads high size total (min, med, max)	566.8 KiB (9.9 KiB, 18.0 KiB, 383.1 KiB)
filesystem read data size (sampled) total (min, med, max)	566.8 KiB (9.9 KiB, 18.0 KiB, 383.1 KiB)
filesystem read time (sampled) total (min, med, max)	4.9 s (302 ms, 584 ms, 734 ms)
metadata time	0 ms
size of files read	6.8 GiB
estimated repeated reads low size total (min, med, max)	566.8 KiB (9.9 KiB, 18.0 KiB, 383.1 KiB)
rows output	85,449

Delta

PushedFilters: [IsNotNull(id), LessThan(id,300)]	
Scan parquet +details	
Stages: 3255.0	1 files .64 sec
file sorting by size time	0 ms
number of files read	1
filesystem read data size total (min, med, max)	371.8 KiB (1505.0 B, 370.3 KiB, 370.3 KiB)
scan time total (min, med, max)	610 ms (210 ms, 400 ms, 400 ms)
estimated repeated reads high size total (min, med, max)	0.0 B (0.0 B, 0.0 B, 0.0 B)
filesystem read data size (sampled) total (min, med, max)	371.8 KiB (1505.0 B, 370.3 KiB, 370.3 KiB)
filesystem read time (sampled) total (min, med, max)	364 ms (103 ms, 261 ms, 261 ms)
metadata time	0 ms
size of files read	143.2 MiB
estimated repeated reads low size total (min, med, max)	0.0 B (0.0 B, 0.0 B, 0.0 B)
rows output	74,820

<https://docs.databricks.com/delta/optimizations/file-mgmt.html#delta-data-skipping>

What about Disk Partitioning?

- Generally not recommended!
 - Partitioning is usually misused/overused
- Good use-cases for Partitioning
 - Isolating data for separate schemas (single-→multiplexing)
 - GDPR/CCP use cases where you commonly delete a partitions worth of data
 - Use cases requiring a physical boundary to isolate data
 - SCD Type 2, partition on current or not for better performance
- If you Partition:
 - Choose column with low cardinality
 - Try to keep each partition greater than 1GB
 - Table expected to grow TBs
 - Partition (usually) on date



Challenges with Disk Partitioning

Optimal file size range is typically between 128MB – 1 GB

partition by **customer_ID** and **date** + optimize

	2023-02-05	2023-02-06	2023-02-07
Customer A			
Customer B			
Customer C	 	  	  
Customer D			
Customer E			
Customer F			

- Many **small files** (small-sized files)
 - High metadata operations overhead
 - Slow read operations
- Many **large-sized** files
 - Inconsistent file sizes across partitions

By Partitioning by '**customer_id**' and '**date**', we end up with high cardinality rows which are better suited for Z-Order



```
DESCRIBE HISTORY iot_data;
```

⚙️ operationMetrics

```
> {"numFiles": "32", "numOutputRows": "50000000", "numOutputBytes": "809126357"}  
> {"numFiles": "2500", "numOutputRows": "2500", "numOutputBytes": "1614693"}
```

PO 1.1: Small File Explosion (Partitioned vs Non-Partitioned)

```
.partitionBy('id')
```

2 Queries

```
SELECT * FROM iot_data  
WHERE id = 519220707
```

```
SELECT avg(value) FROM iot_data  
WHERE time >= "2023-12-04 12:19:00"  
AND time <= "2023-12-04 13:01:20"
```

Partitioned (2500 rows)

3.82 seconds

files pruned	2,499
files read	1

7.24 seconds

files pruned	0
files read	2,500

Non-Partitioned (50m rows)

1.84 seconds

files pruned	0
files read	32

1.54 seconds

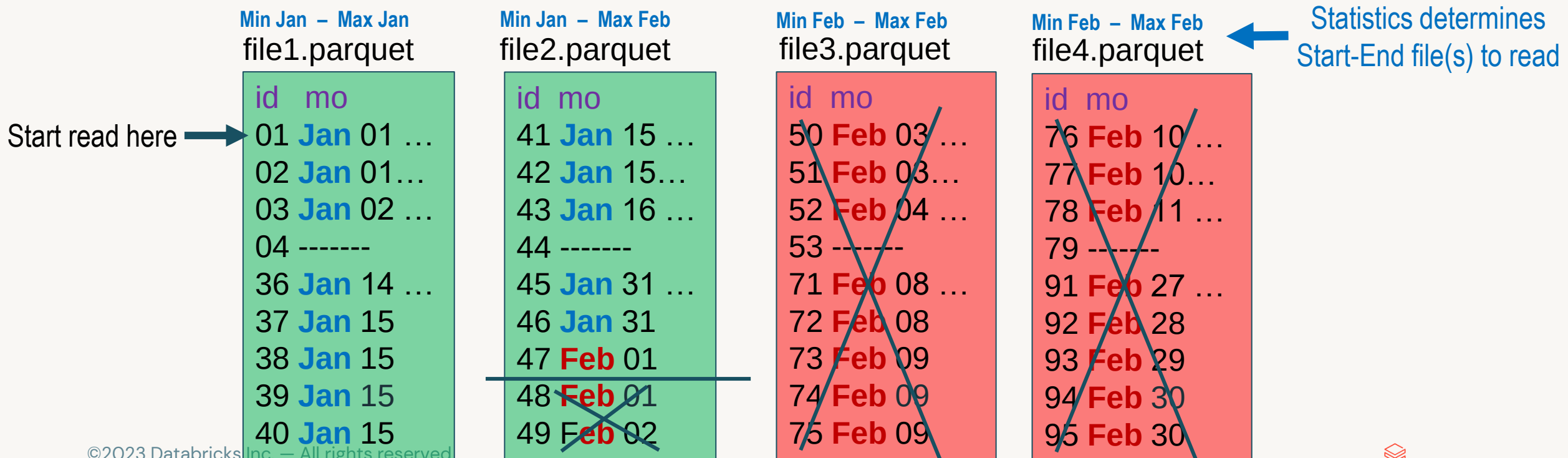
files pruned	31
files read	1

Z-Order rows by 'mo' column

OPTIMIZE tableX ZORDER BY (mo);

Order rows by first date on existing files. So instead of Partitioning by MONTH (where each directory only has file(s) with specific MONTH), have non-Partitioned table with files that can span more than 1 month

When query: **SELECT * FROM tableX WHERE mo = 'JAN';** Optimizer knows which files to read (via Stats) and it knows rows are contiguous in those file(s). When it hits a FEB row, it now has all the rows it needs



Introducing Liquid Clustering

```
CREATE TABLE flights_clstr_id  
(id BIGINT, year INT, FlightNum INT,  
UniqueCarrier STRING, TailNum STRING)  
USING delta  
CLUSTER BY (id)  
TBLPROPERTIES (parquet.compression.codec= 'zstd');
```

What it is?

- Technique to clustering data layout to support efficient query access and reduce data management and tuning overhead
- It's flexible and adaptive to data pattern changes, scaling, and skew
- Alternative to manually defining both **Partitioning** and **Z-Order**

Benefits

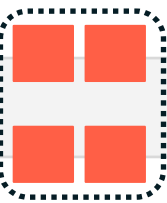
- Can redefine Clustering keys without re-writing existing data
- Most consistent data skipping
 - Immune to data skew
- Minimal write amplification on table maintenance
 - True incremental optimize
- Row Level Concurrency
 - Simplify logic of concurrent writers
- Reduced Cognitive Overhead
 - No worrying about cardinality



Liquid Clustering

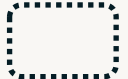
Consistent file size

Liquid cluster by `customer_ID` and `date`

	2023-02-05	2023-02-06	2023-02-07
Customer A			
Customer B			
Customer C			
Customer D			
Customer E			
Customer F			

Target File Size -> 64MB Mng table
256MB Ext table
Auto adjusted if source data is large

Target file size

- Liquid is not subject to rigid boundaries
 - Liquid intelligently decides which ranges of data to combine 
- Data skew is gone
 - **Files sizes are consistent**
- Liquid stores metadata
 - New data can be clustered into existing clusters on write **incrementally**



Non-Liquid Clustering table

351 files Min file size: 41 MB Max file size: 305 MB

Size difference between the 2 files: 7.41x

Total files size: 8.08 GB

1 %sql
2 LIST 'dbfs:/user/hive/warehouse/dbacademy.db/flights'

▸  _sqldf: pyspark.sql.dataframe.DataFrame = [path: string, name: string ... 2 more fields]

Table ▾ +

	 path	 name	 size	 modification_time
1	> dbfs/us...	_delta_log/	0	1734361145876
2	> dbfs/us...	part-00131-07f1a88d-dd24-4479-85bb-4f7b178d4688.c000.snappy.parquet	41352784	1734117874000
3	> dbfs/us...	part-00131-6fd65125-3248-444a-85ce-681cb181e6aa.c000.snappy.parquet	41352814	1734116075000
4	> dbfs/us...	part-00147-e843babe-239c-4f65-9008-f24eb6caa322.c000.snappy.parquet	41355876	1734118092000
5	> dbfs/us...	part-00147-41e2587a-be73-4b8e-9d90-5ab70532d7c7.c000.snappy.parquet	41355906	1734116276000
6	> dbfs/us...	part-00035-a312863d-b9ab-4bc4-b982-98794317048d.c000.snappy.parquet	41360842	1734116534000
7	> dbfs/us...	part-00035-281b98b1-228b-40b3-98ed-5ff0135d8a5a.c000.snappy.parquet	41360872	1734114870000
8	> dbfs/us...	part-00003-ac421f0d-459c-4245-826f-3394e19d9416.c000.snappy.parquet	41366259	1734116002000
9	> dbfs/us...	part-00003-954270b6-78a9-4fff-8edc-5d80044b286f.c000.snappy.parquet	41366289	1734114452000
10	> dbfs/us...	part-00115-1875ed94-69ed-4cf0-b318-1b836d307475.c000.snappy.parquet	41388048	1734117652000
11	> dbfs/us...	part-00115-fdd5eb6d-6921-470a-8804-94bc5ef49e57.c000.snappy.parquet	41388078	1734115878000
12	> dbfs/us...	part-00066-4cd07e8-9744-4fb8-9043-df22e61fa75a.c000.snappy.parquet	41389432	1734116981000
13	> dbfs/us...	part-00066-77296a21-41a9-407a-a2ca-0c3ba1a0d99f.c000.snappy.parquet	41389462	1734115274000
14	> dbfs/us...	part-00083-310553f0-f44a-4e10-914a-513cd6009f64.c000.snappy.parquet	41389649	1734117208000
15	> dbfs/us...	part-00083-4876ead-bd74-4838-84cb-ac4204d4238e.c000.snappy.parquet	41389679	1734115477000

345	> dbfs/us...	part-00000-cbe47b52-74aa-4e1b-a55e-9156f3b93b0d.c000.snappy.parquet	272115845	1734116991000
346	> dbfs/us...	part-00027-98584834-c178-4cb2-852b-d83a81697659.c000.snappy.parquet	272647458	1734116944000
347	> dbfs/us...	part-00025-55fb9b57-8b12-4261-be77-2be45e432ff.c000.snappy.parquet	278703172	1734116944000
348	> dbfs/us...	part-00026-f97bb82a-a217-4a72-b848-4ddd21b0bdd0.c000.snappy.parquet	285170514	1734116892000
349	> dbfs/us...	part-00028-7d77bee6-15d7-485a-a392-3c8d09172b54.c000.snappy.parquet	294655924	1734116940000
350	> dbfs/us...	part-00019-60056bce-035a-4de2-9e7b-40cd25a04695.c000.snappy.parquet	295208734	1734116893000
351	> dbfs/us...	part-00029-1ef19938-5b50-4898-b168-08945e96dfde.c000.snappy.parquet	303897211	1734116894000
352	> dbfs/us...	part-00030-f3918b82-04b0-467f-b55c-d0f5ffc02acc.c000.snappy.parquet	305565556	1734116892000

Liquid Clustering table

32 files Min file size: 207 MB Max file size: 320 MB

Size difference between the 2 files: 1.54x

Total files size: 8.3 GB

1 %sql
2 LIST 'dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id'

▸  _sqldf: pyspark.sql.dataframe.DataFrame = [path: string, name: string ... 2 more fields]

Table ▾ +

	 path	 name	 size
2	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00023-baf68c86-490b-41a2-ba42-b57b8639b5c0.c000.snappy.parquet	part-00023-baf68c86-490b-41a2-ba42-b57b8639b5c0.c000.snappy.parquet	206974747
3	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00026-8e67c7e2-2b6a-47db-b183-af6839f8ecd9.c000.snappy.parquet	part-00026-8e67c7e2-2b6a-47db-b183-af6839f8ecd9.c000.snappy.parquet	209863641
4	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00005-7dd1246d-aff6-4033-9983-c133e4014bac.c000.snappy.parquet	part-00005-7dd1246d-aff6-4033-9983-c133e4014bac.c000.snappy.parquet	210121724
5	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00030-de6c3816-e284-4271-8725-fc460d52d03c.c000.snappy.parquet	part-00030-de6c3816-e284-4271-8725-fc460d52d03c.c000.snappy.parquet	215050336
6	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00029-d41bf39a-a0da-4ade-9f71-0771ffdb7b26.c000.snappy.parquet	part-00029-d41bf39a-a0da-4ade-9f71-0771ffdb7b26.c000.snappy.parquet	218545766
7	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00014-9ecfd7c3-7b5f-49da-8e44-0d2140e441b1.c000.snappy.parquet	part-00014-9ecfd7c3-7b5f-49da-8e44-0d2140e441b1.c000.snappy.parquet	219360271
8	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00011-77b70404-ebc3-452b-94e0-beda62c3d7.c000.snappy.parquet	part-00011-77b70404-ebc3-452b-94e0-beda62c3d7.c000.snappy.parquet	226649209
9	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00015-c59fb9c4-7186-4ae5-8a32-9bba58e3ef02.c000.snappy.parquet	part-00015-c59fb9c4-7186-4ae5-8a32-9bba58e3ef02.c000.snappy.parquet	227240013
10	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00007-8f2c121e-afda-4565-b69b-cd1db6089d59.c000.snappy.parquet	part-00007-8f2c121e-afda-4565-b69b-cd1db6089d59.c000.snappy.parquet	233534968
11	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00002-59f08a64-cb62-4db6-a850-52a2836bd3b2.c000.snappy.parquet	part-00002-59f08a64-cb62-4db6-a850-52a2836bd3b2.c000.snappy.parquet	236520122
12	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00000-8512963c-2900-4463-88ae-b66a0b90a647.c000.snappy.parquet	part-00000-8512963c-2900-4463-88ae-b66a0b90a647.c000.snappy.parquet	244374109
13	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00013-c80d5038-44e2-46a8-9a27-a920e07e6b14.c000.snappy.parquet	part-00013-c80d5038-44e2-46a8-9a27-a920e07e6b14.c000.snappy.parquet	247146702
14	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00009-e2d50f09-2979-43b9-a112-449fd427dc30.c000.snappy.parquet	part-00009-e2d50f09-2979-43b9-a112-449fd427dc30.c000.snappy.parquet	255226042
15	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00025-90f0671a-5bac-4cd9-8e5e-4277a0dfd0b3.c000.snappy.parquet	part-00025-90f0671a-5bac-4cd9-8e5e-4277a0dfd0b3.c000.snappy.parquet	257655321
16	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00001-76be910a-9373-4f0a-8edf-ebd9c5103732.c000.snappy.parquet	part-00001-76be910a-9373-4f0a-8edf-ebd9c5103732.c000.snappy.parquet	257809332
17	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00022-b0748e34-3a02-4e80-bf85-8709830490f4.c000.snappy.parquet	part-00022-b0748e34-3a02-4e80-bf85-8709830490f4.c000.snappy.parquet	262561660
18	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00028-cadea3ff-5f7b-460d-95a1-e21b74965659.c000.snappy.parquet	part-00028-cadea3ff-5f7b-460d-95a1-e21b74965659.c000.snappy.parquet	262783259
19	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00021-2fe752be-d1a3-4800-8d10-02b3e43c07df.c000.snappy.parquet	part-00021-2fe752be-d1a3-4800-8d10-02b3e43c07df.c000.snappy.parquet	264105306
20	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00018-f591d5b1-8efd-406a-8b6f-29c3e0a73920.c000.snappy.parquet	part-00018-f591d5b1-8efd-406a-8b6f-29c3e0a73920.c000.snappy.parquet	268049692
21	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00010-135529b0-e082-4943-8c4c-3d16d96f3e8.c000.snappy.parquet	part-00010-135529b0-e082-4943-8c4c-3d16d96f3e8.c000.snappy.parquet	268909191
22	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00016-e75a1d8a-d494-44a0-8df8-85b4336b36a7.c000.snappy.parquet	part-00016-e75a1d8a-d494-44a0-8df8-85b4336b36a7.c000.snappy.parquet	278704113
23	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00027-3348b7eb-0b44-4b98-9f6c-5963f156a5d2.c000.snappy.parquet	part-00027-3348b7eb-0b44-4b98-9f6c-5963f156a5d2.c000.snappy.parquet	275219177
24	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00006-6509712c-0dd6-4bd5-adb4-7995059fc70e.c000.snappy.parquet	part-00006-6509712c-0dd6-4bd5-adb4-7995059fc70e.c000.snappy.parquet	280166287
25	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00019-20bb3c1e-1c16-46d4-813a-0c5b45662e32.c000.snappy.parquet	part-00019-20bb3c1e-1c16-46d4-813a-0c5b45662e32.c000.snappy.parquet	281918142
26	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00017-a1b29144-6bd3-446e-b0f6-1a70904da023.c000.snappy.parquet	part-00017-a1b29144-6bd3-446e-b0f6-1a70904da023.c000.snappy.parquet	282494422
27	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00024-9aa1b294-0dd6-49ad-a066-b3a2e8fc4a2.c000.snappy.parquet	part-00024-9aa1b294-0dd6-49ad-a066-b3a2e8fc4a2.c000.snappy.parquet	287025880
28	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00012-5f6b8dee-5e6d-4471-bcbe-4e508da73bb7.c000.snappy.parquet	part-00012-5f6b8dee-5e6d-4471-bcbe-4e508da73bb7.c000.snappy.parquet	289720984
29	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00031-4a3031f2-e495-48bd-8e34-664acd6d77e6.c000.snappy.parquet	part-00031-4a3031f2-e495-48bd-8e34-664acd6d77e6.c000.snappy.parquet	295407343
30	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00008-37e50c3e-b42b-4475-ab0d-71f95f693930.c000.snappy.parquet	part-00008-37e50c3e-b42b-4475-ab0d-71f95f693930.c000.snappy.parquet	295827773
31	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00020-257cdeb3-2cf0-4a1a-900c-584498a0bce5.c000.snappy.parquet	part-00020-257cdeb3-2cf0-4a1a-900c-584498a0bce5.c000.snappy.parquet	306354176
32	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00003-83e3ecd6-0192-4d5e-92c1-3649cd88510.c000.snappy.parquet	part-00003-83e3ecd6-0192-4d5e-92c1-3649cd88510.c000.snappy.parquet	318700587
33	dbfs:/user/hive/warehouse/dbacademy.db/flights_cluster_id/part-00004-90f53d16-3e6c-4439-8b68-a4b364d32141.c000.snappy.parquet	part-00004-90f53d16-3e6c-4439-8b68-a4b364d32141.c000.snappy.parquet	320844244



Hive Partitioning vs Delta Lake Partitioning

Hive Partitioning

- Uses physical directory structure to organize data
- Partitions are represented as subdirectories within the main table directory
- Requires manual management of partition boundaries
- Can lead to small files and increased storage costs
- Works well for columns with low cardinality
- Query performance depends on partition column being used

Delta Lake Partitioning

- Uses logical partitioning based on metadata in the transaction log
- Does not require physical directory structure
- Partitions are defined in the table schema, not physical location
- More flexible and easier to manage than Hive-style partitions
- Supports dynamic partitioning and partition evolution
- Better performance for large tables (>100TB)
- ACID transactions are not bound to partition boundaries

Key Differences

- Delta Lake uses logical partitioning, while Hive uses physical partitioning
- Delta Lake does not require small files or physical directory structure
- Delta Lake is more flexible and easier to manage than Hive-style partitions
- Delta Lake partitions are defined in the table schema, not physical location
- Delta Lake supports dynamic partitioning and partition evolution

When to Use Each

- Hive partitioning may be useful for very large tables (>100TB) or when migrating from traditional data warehouses
- Delta Lake partitioning is recommended for most modern data lake architectures
- Consider using Delta Lake's automatic ingestion time clustering for smaller tables (<1TB)



Summary

	Hive-style Partitioning	Z-Ordering	Liquid Clustering
Many small files			
Data skew			
Slow writes			
Requires tuning			
Inconsistent performance			
Flexible			
Immediate good skipping			
Incremental			
Risk of conflict with maintenance operations			



```
df = spark.table('hive_metastore.dbacademy.flights')
df.summary().display()
```

PO 1.2 Data skipping
OPTIMIZE hive_metastore.dbacademy.flights ZORDER BY id, FlightNum;

```
SQL table (id by id, flight_num)  rows  Data read
SQL (WHERE Carrier)              - 13.88 100MB
SQL (WHERE flight_num)           - 4.30 31MB
SQL (WHERE id)                   - 4.70 34MB

CL table (Cluster by id)
CL (WHERE Carrier)              - 14.41 114MB
CL (WHERE flight_num)           - 17.15 201MB
CL (WHERE id)                   - 4.37 33MB

CL table (Cluster by id, flight_num)
CL (WHERE Carrier)              - 15.96 111MB
CL (WHERE flight_num)           - 4.44 302MB
CL (WHERE id)                   - 4.30 422MB
```

PO 1.2: Data Skipping (Liquid Clustering vs Partition + ZOrder)

Run with SQL Whse to get stats (row read, bytes read)
Go to SQL Whse > Spark UI > Files pruned, files read

P-Z table: 8161 files 7.68GB
LC table : 144 files 8.24GB

Compare LC to Part+ZOrder (rows read, bytes read, files pruned/read)

		Rows read	Bytes read	files pruned	files read
SELECT * FROM fly_lc_id_num WHERE id = 1125281431554	4.21 s	1	431.12 MB	132	12
SELECT * FROM fly_part_num_z_id WHERE id = 1125281431554	11.12 s	8,161	4.44 GB	1	8,160
SELECT * FROM fly_lc_id_num WHERE FlightNum = 1890	3.28 s	100,302	598.99 MB	132	12
SELECT * FROM fly_part_num_z_id WHERE FlightNum = 1890	1.71 s	12,289	2.77 MB	8,160	1
SELECT * FROM fly_lc_id_num WHERE UniqueCarrier = 'TW'	3.33 s	53,836	297.12 MB	0	144
SELECT * FROM fly_part_num_z_id WHERE UniqueCarrier = 'TW'	3.92 s	107,250	44.16 MB	2,637	5,524

Table Statistics

Keeping table statistics to date for best results with **Cost Based Optimizer**

- Collects statistics on all columns in table
- Helps **Adaptive Query Execution**
 - Choose proper join type
 - Select correct build side in a hash-join
 - Calibrating the join order in a multi-way join

```
ANALYZE TABLE mytable COMPUTE STATISTICS FOR ALL COLUMNS;
```

<https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-aux-analyze-table.html>

Without CBO, Largest Tables joined 1st (1 of 2)

```
spark.conf.set("spark.sql.cbo.enabled", False)
spark.conf.set("spark.sql.cbo.joinReorder.enabled", False)
spark.conf.set("spark.sql.statistics.histogram.enabled", False)
```

```
-- With CBO turned off/no Stats, follows JOIN order

SELECT l.id, m.location, s.author
FROM nocbo_large_wiki AS l INNER JOIN nocbo_medium_wiki AS m
ON l.id = m.id
INNER JOIN nocbo_small_wiki AS s
ON s.id = m.id;
```

Oh no, we are joining 2 largest tables 1st

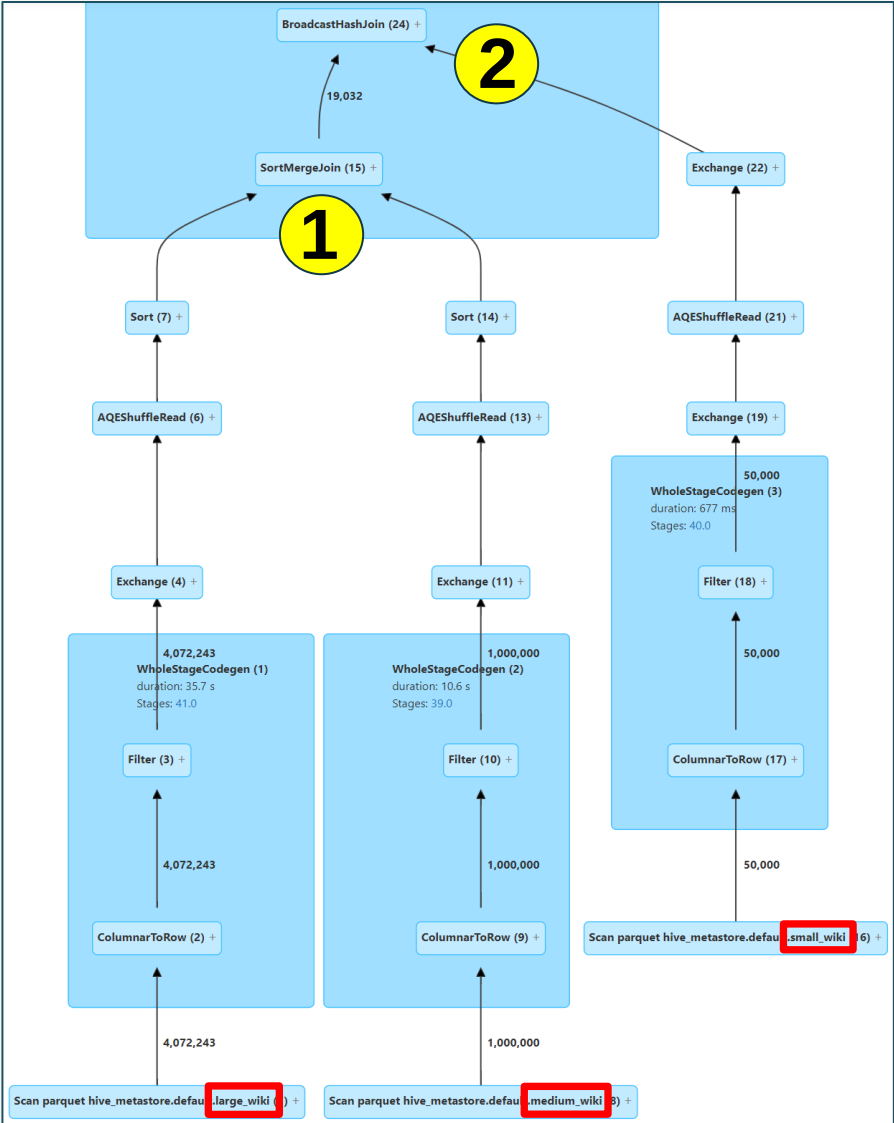
```
EXPLAIN COST SELECT * FROM nocbo_large_wiki;
```

_sqldf: pyspark.sql.dataframe.DataFrame = [plan: string]

Table +

	plan
1	<pre>== Optimized Logical Plan == Relation default.nocbo_large_wiki[id#48313L,location#48314,ts#48315,comment#48316,author#48317] parquet GiB) == Physical Plan == *(1) ColumnarToRow +- FileScan parquet default.nocbo_large_wiki[id#48313L,location#48314,ts#48315,comment#48316,author#48317] Batched: true, DataFilters: [], Format: Parquet, Location: PreparedDeltaFileIndex(1 paths)[dbfs:/user/hive/warehouse/nocbo_large_wiki], PartitionFilters: [], PushedFilters: [], ReadSchema...</pre>

Even if CBO = On, if don't ANALYZE, same Plan as this



20.14 seconds

Before

```
spark.sql("ANALYZE TABLE largewiki1 COMPUTE STATISTICS FOR COLUMNS id, location")
spark.sql("ANALYZE TABLE medwiki1 COMPUTE STATISTICS FOR COLUMNS id, location")
spark.sql("ANALYZE TABLE smallwiki1 COMPUTE STATISTICS FOR COLUMNS id, location")
```

6.86 seconds

11.71 seconds

After

With CBO: Smaller Tables joined 1st (2 of 2)

```
spark.conf.set("spark.sql.cbo.enabled", True)
spark.conf.set("spark.sql.cbo.joinReorder.enabled", True)
spark.conf.set("spark.sql.statistics.histogram.enabled", True)
```

```
spark.sql("ANALYZE TABLE large_wiki COMPUTE STATISTICS FOR COLUMNS id, location")
spark.sql("ANALYZE TABLE med_wiki COMPUTE STATISTICS FOR COLUMNS id, location")
spark.sql("ANALYZE TABLE small_wiki COMPUTE STATISTICS FOR COLUMNS id, location")
```

-- Optimizer JOIN 2 Smallest tables 1st

```
SELECT l.id, m.location, s.author
FROM large_wiki AS l INNER JOIN med_wiki AS m
ON l.id = m.id
INNER JOIN small_wiki AS s
ON s.id = m.id;
```

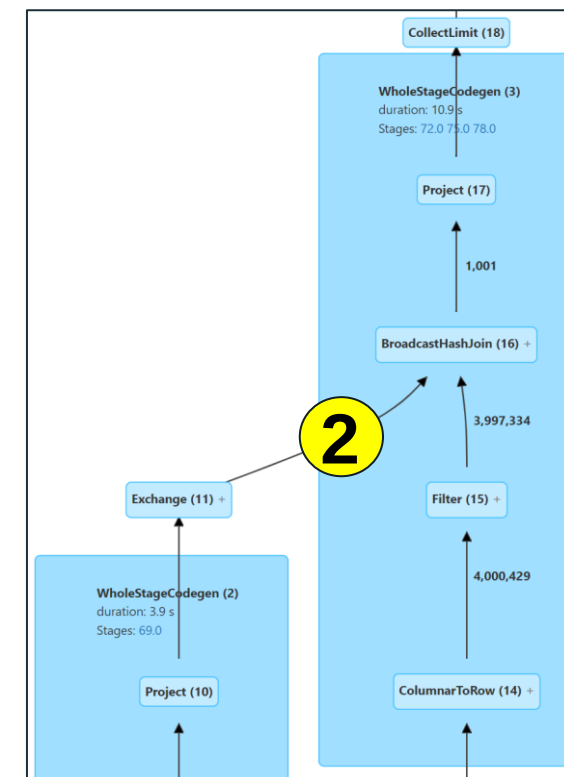
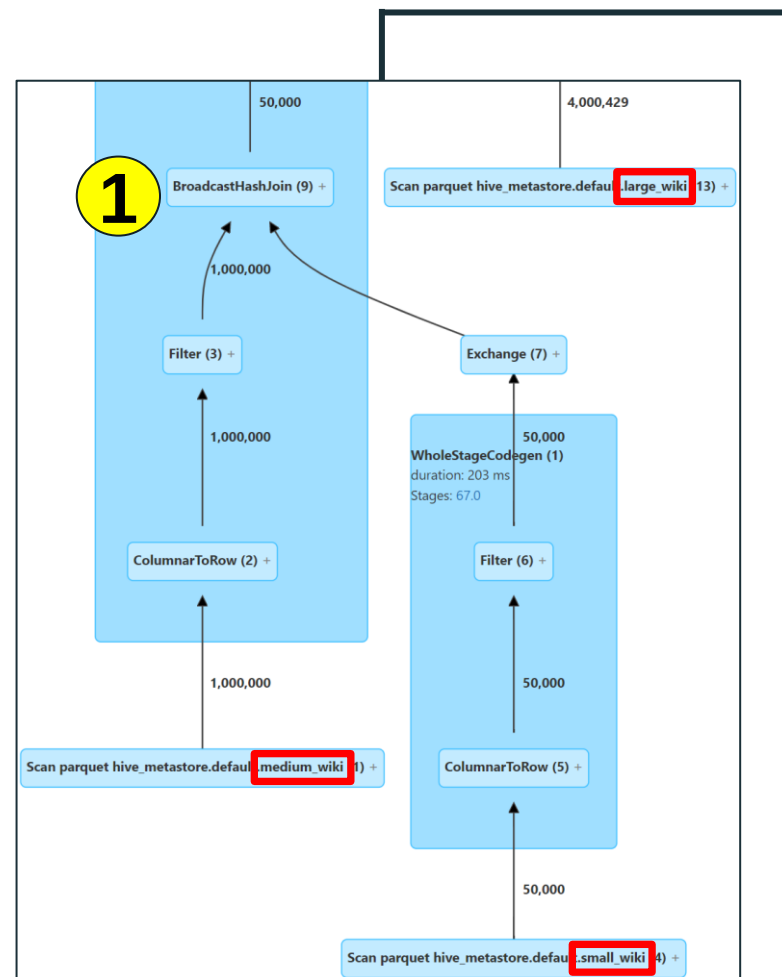
```
1 %sql
2 EXPLAIN COST SELECT * FROM large_wiki;
```

```
_sqldf: pyspark.sql.dataframe.DataFrame = [plan: string]
```

Table +

plan

```
== Optimized Logical Plan == Relation default.large_wiki[id#45317L,location#45318,ts#45319,comment#45320,author#45321]
parquet, Statistics(sizeInBytes=372.8 MiB, rowCount=4.07E+6, [id#45317L ->
ColumnStat(Some(3896095),Some(25),Some(42648837),Some(0),Some(8),Some(8),None,2),location#45318 ->
ColumnStat(Some(4072243),None,None,Some(0),Some(20),Some(245),None,2)]) == Physical Plan == *(1) ColumnarToRow +-
FileScan parquet default.large_wiki[id#45317L,location#45318,ts#45319,comment#45320,author#...
```



Predictive I/O

Settings

Auto-Enable Deletion Vectors

Default (All new tables) ▾

- Predictive I/O is exclusive to the Photon engine on Databricks
- Predictive I/O is a collection of Databricks optimizations that improve performance for data interactions
- Capabilities are grouped into the following categories:
 1. Accelerated **reads** reduce the time it takes to scan and read data (via Deep Learning algorithms)
 2. Accelerated **writes** reduce the amount of data that needs to be rewritten during **updates**, **deletes**, and **merge** (via **Deletion Vectors**)

<https://docs.databricks.com/en/admin/workspace-settings/deletion-vectors.html>



Bonus: Deletion Vectors (part of Predictive I/O)★

Databricks Runtime 14.2 now has row-level concurrency generally available and enabled by default for Delta tables with deletion vectors. This feature dramatically reduces conflicts between concurrent write operations.

```
DELETE FROM wiki_nondv WHERE id = 670;  
UPDATE wiki_nondv SET id = 1 WHERE id = 1192;
```

Write amplification = 38.3 MB

ORIGINAL
DEL
UPD

A ^B _C name	1 ² ₃	size
part-00000-a9870aaa-c6fc-4836-8935-327b65d631aa-c000.snappy.parquet		19164491
part-00000-c6857282-5d3a-4f9e-aadd-101012207f6a-c000.snappy.parquet		19144425
part-00000-e9758fcd-b745-4617-9105-01ac205a0476-c000.snappy.parquet		19144425
_delta_log/		0

Without

```
DELETE FROM wiki_dv WHERE id = 670;  
UPDATE wiki_dv SET id = 1 WHERE id = 1192;
```

Write amplification = .01 MB

ORIGINAL
DEL
UPD

A ^B _C name	1 ² ₃	size
part-00000-e2e063ba-f4ed-4b09-af63-154abdfe00f1-c000.snappy.parquet		19164491
deletion_vector_6083ee0a-5e21-4813-a417-1bffa3972b3e.bin		43
deletion_vector_446ff2c4-b2c2-442a-914b-cb5956f92fcb.bin		87
part-00000-4686a287-a4e0-44a8-919e-287bde0dd1dd-c000.snappy.parquet		10115
_delta_log/		0

With

Before we talk about: Predictive Optimization

OPTIMIZE and VACUUM and ANALYZE

OPTIMIZE

```
OPTIMIZE students  
ZORDER BY id;
```

```
numFilesAdded: 1  
numFilesRemoved: 4
```

- Combine Small-sized Data Files into Larger-sized Data Files
- In above example, 4 small files coalesced into 1 larger file
- The 4 files 'removed' are 'Tombstone' files that still exist

VACUUM

```
SET spark.databricks.delta.retentionDurationCheck.enabled = false;  
SET spark.databricks.delta.vacuum.logging.enabled = true;  
VACUUM students RETAIN 0 HOURS
```

- Deletes the Tombstone (non-current) files from Cloud storage
- In above example, we retain No Tombstone files for table 'students'

ANALYZE

- Triggers incremental update of statistics to improve query performance



Predictive Optimization (Optimize, Vacuum)

- Predictive Optimization refers to using predictive analytics techniques to automatically optimize and enhance the performance of systems, processes, or workflow
- Only works on Unity Catalog Managed tables that have Deletion Vectors enabled

Operation	Description
OPTIMIZE	Triggers incremental clustering for enabled tables. See Use liquid clustering for Delta tables. Improves query performance by optimizing file sizes. See Optimize data file layout.
VACUUM	Reduces storage costs by deleting data files no longer referenced by the table. See Remove unused data files with vacuum.

https://www.databricks.com/blog/introducing-predictive-optimization-faster-queries-cheaper-storage?utm_source=linkedin&utm_medium=organic-social



Predictive Optimization

It figures out:

- Which optimizations to run?
- Which tables should be optimized?
- How often to run these optimizations?

Key Features

Automatic Maintenance

- It automates the execution of background maintenance tasks on Delta tables



Support Maintenance Operations

- It supports maintenance operations, including **OPTIMIZE** to improve query performance by optimizing file sizes and **VACUUM** to reduce storage costs by deleting unused data



Set and Forget Approach

- It intelligently and automatically runs maintenance jobs without requiring ongoing user supervision



Serverless Computing

- It utilizes serverless compute, eliminating the need for users to manually manage compute cluster



How Predictive Optimization works

Account console

Settings

Feature enablement

Predictive Optimization

Disabled ▼

Databricks intelligently optimizes your Unity Catalog managed tables for performance and cost effectiveness.

[Learn more](#)

Schedule

Databricks intelligently determines the best schedule of optimizations based on table usage and data layouts

Optimize

Databricks automatically runs optimizations using purpose-built serverless infrastructure

Log

Operations and their impact are logged in a system table for easy observability

2 Code Optimization



Code Optimization

Four commonly seen performance problems associate with Spark

1. Skew Shuffle (Solution: Adaptive Query Execution)
2. Reduce Shuffle (One Solution: Broadcast Hash Join)
3. Spill
4. Serialization



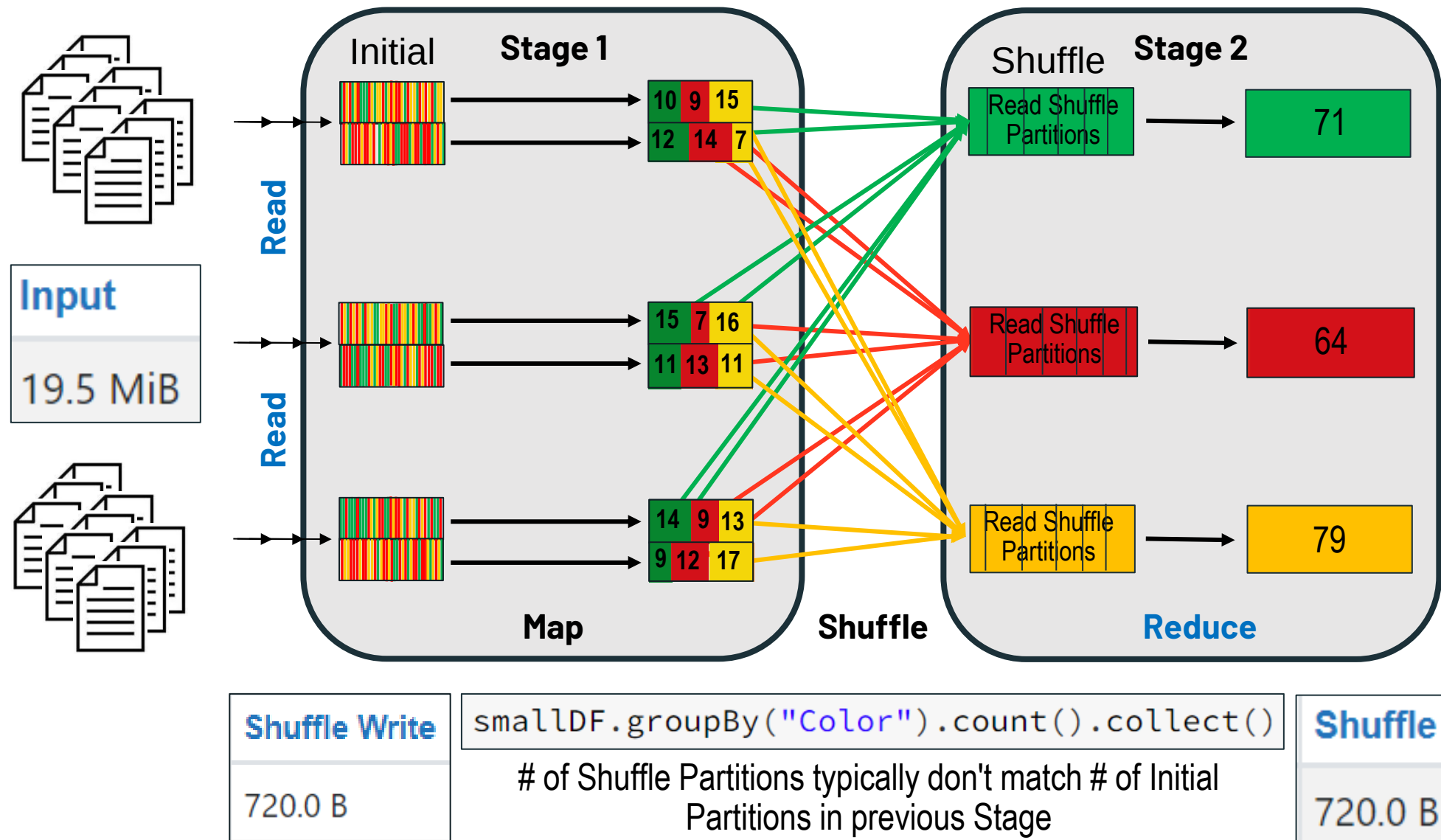
1. Skew (Shuffle Partitions)

Skewed Shuffle Partitions can now be handled **automatically** by
Adaptive Query Execution

Although there are older ways to minimize Skewed Shuffle Partitions, ('**Skew Hints**' or '**Salting Join keys**'), this requires **manually** intervention



Review: Shuffle Partitions



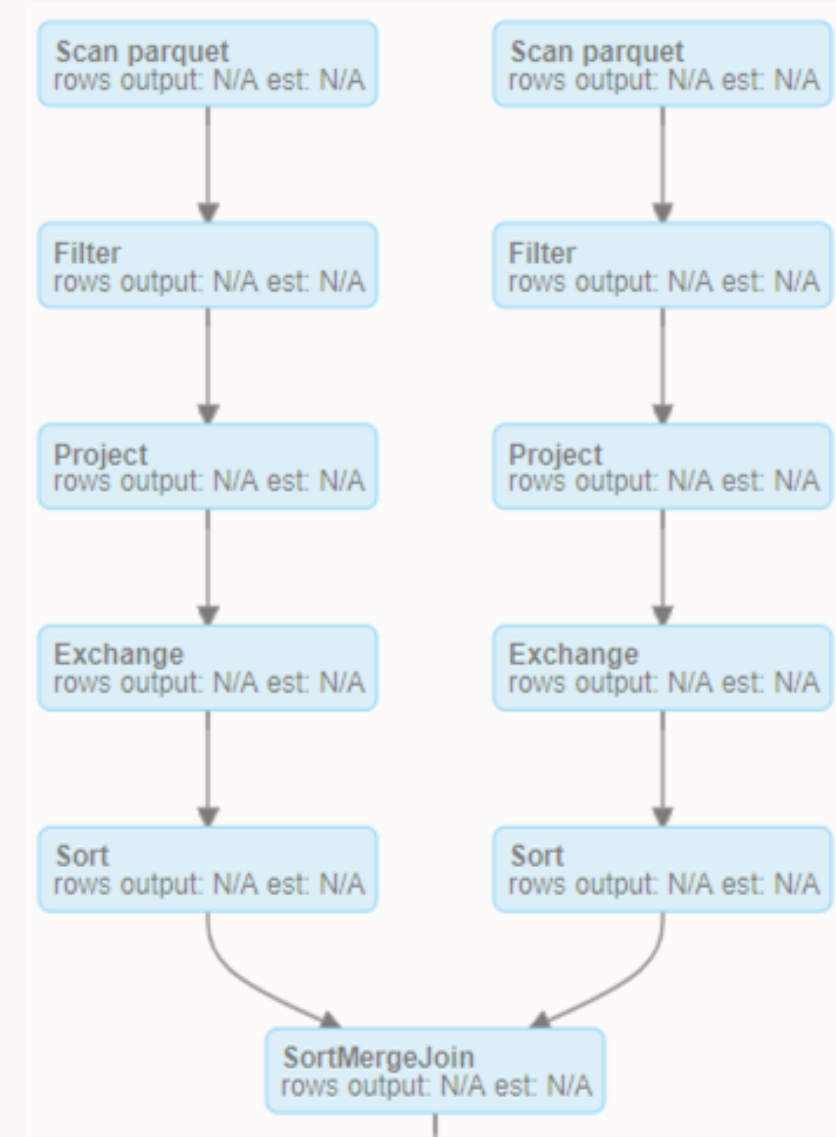
Shuffles

As seen earlier, wide transformations cause new Shuffle Partitions to spawn

This incurs 3 costs:

1. Disk I/O
2. Network traffic (if > 1 worker)
3. DeSerialization/Serialization

- join
- sort
- distinct
- group By
- order by
- count
- repartition
- dropDuplicates

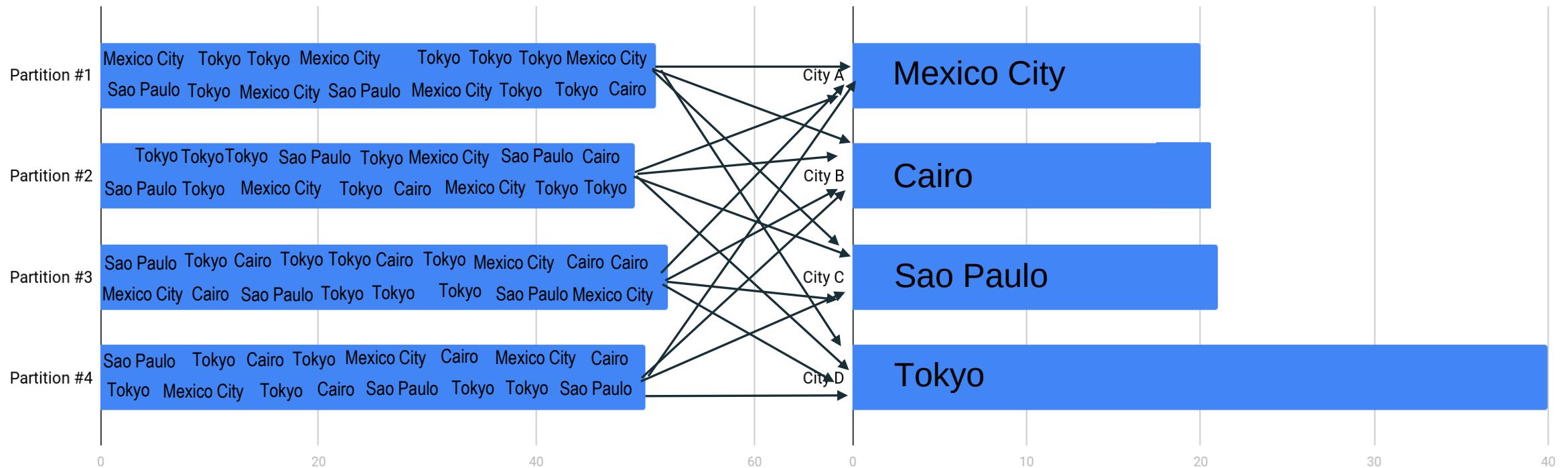


Skewed Shuffle Partitions – Before & After

```
trxDF.join(ctyDF, ctyDF("city_id") == trxDF("city_id")).write.format("noop").mode("overwrite").save()
```

Before Join
(Initial Memory Partition – 128 MB Max size)

After Join
(Shuffle Memory Partition – Variable size)



Handling Skew Shuffle Partitions

number of skewed partitions	9
number of skewed partition splits	36

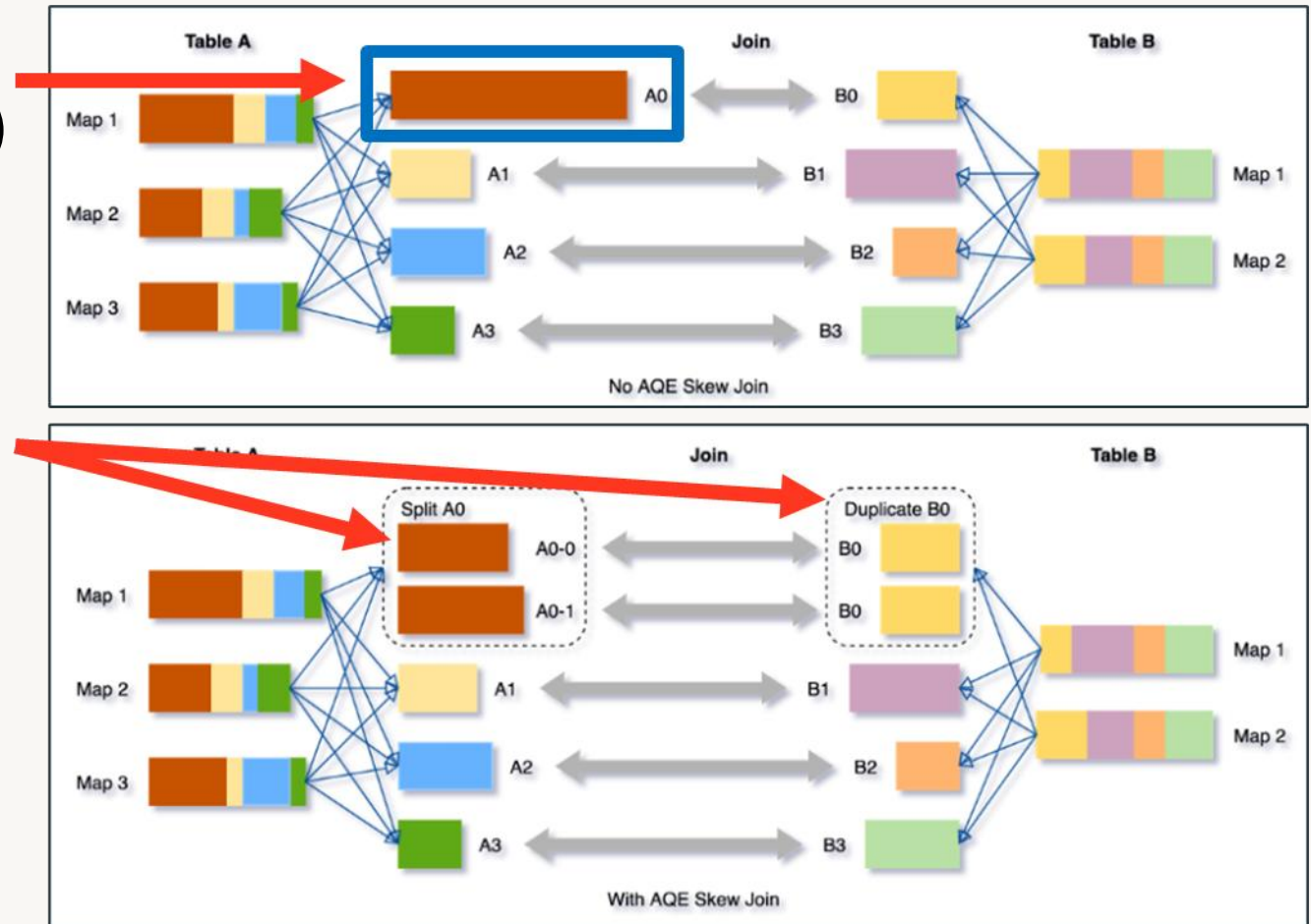
Data skew unavoidable, **Adaptive Query Execution** handles it automatically

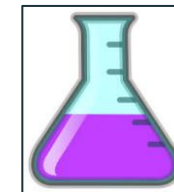
Have Skewed Partition A0
(when compared to A1-A3 Partitions)

AQE is Solution

Over-sized Partition gets split into 2

Typically, a Shuffle Partition is considered Skewed if it is 5x size of median Partition size



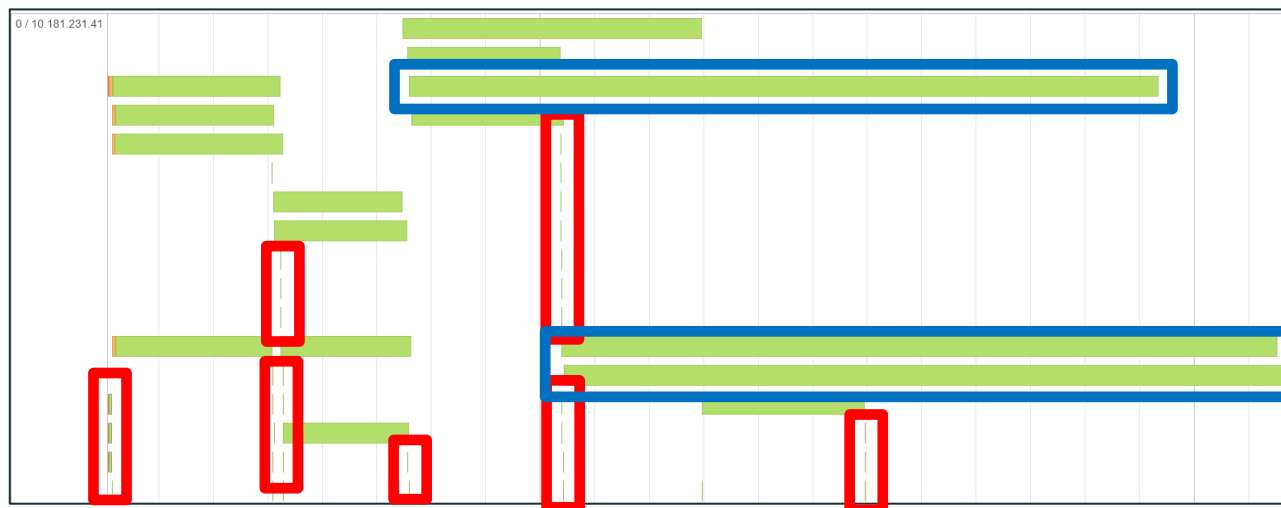


Skew - Baseline vs AQE

See [Experiment #1596](#)

```
== Physical Plan ==
OverwriteByExpression org.apache.spark.sql.execution.datasources.noop.NoopTable$@74741dfa, [AlwaysTrue()]
+- AdaptiveSparkPlan isFinalPlan=true
   +- *(3) SortMergeJoin(skew=true) [city_id#888], [city_id#873], Inner
      :- Sort [city_id#888 ASC NULLS FIRST], false, 0
      :  +- CustomShuffleReader coalesced and skewed
      :    +- ShuffleQueryStage 0, Statistics(sizeInBytes=203.5 GiB, rowCount=2.69E+9, isRuntime=true)
      :      +- Exchange hashpartitioning(city_id#888, 832), true, [id=#1185]
```

Before (super-small and super-large Shuffle partitions)



After (even sized)





2. Reduce Shuffles via Broadcast Hash Join



PO 1.3 Shuffles

Jobs	Stages	Storage
Stages for All Jobs		

Shuffle Write
10.7 KiB
1454.6 MiB
749.0 B
1412.9 MiB

These are the 2 (excessive) Shuffles from the 150M row table (transaction). I should not have coded this on Left table. Should have coded the smaller table as the Left table

Stop at Broadcast Join (next Lab)



Spark Join Strategies

Spark Join Strategies are named after their associated distribution and join strategies

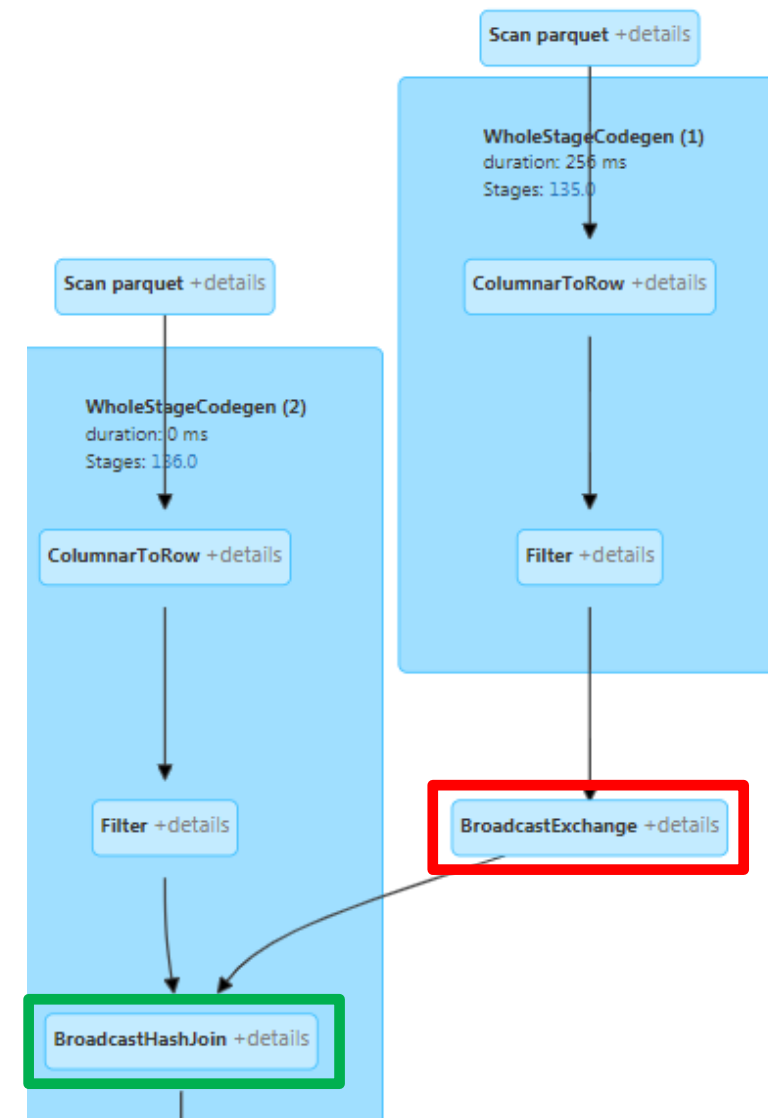
Distribution Strategy	Join Type	Join Strategy Name
Broadcast	Hash Join	<u>Broadcast-Hash Join</u>
Shuffle	Hash Join	<u>Shuffle-Hash Join</u>
Shuffle	Sort Merge Join	<u>Shuffle-Sort Merge Join</u>
Broadcast	Nested Loop	<u>Broadcast-Nested Loop Join</u>

<https://forrestmcdaniel.com/2019/02/10/the-three-physical-joins-visualized/>



4 BroadcastHashJoins

- **BroadcastHashJoins** are not a magic bullet
- The use cases are **typically** limited to small tables (under 10 MB by default)
- They can put undue pressure on the Driver resulting in OOMs* (not case anymore)
- In some cases, the alternative **SortMergeJoin** might be faster (even with **Exchange** and **Sort** step
- In general, Spark's automatic behavior might be your best bet (turn on AQE)



Let's review how the BroadcastHashJoin works...

Assume you are using Photon.

If not, then Driver (and not the Executors) hands out the complete smaller table to the Executors

TheDriver plans Partitions our two Datasets (and hands them to Executors to be processed as Tasks)

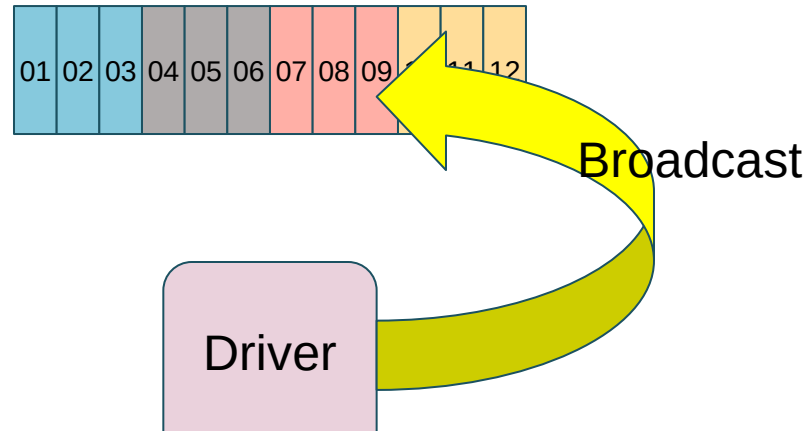
01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

01	02	03	04	05	06	07	08	09	10	11	12
----	----	----	----	----	----	----	----	----	----	----	----

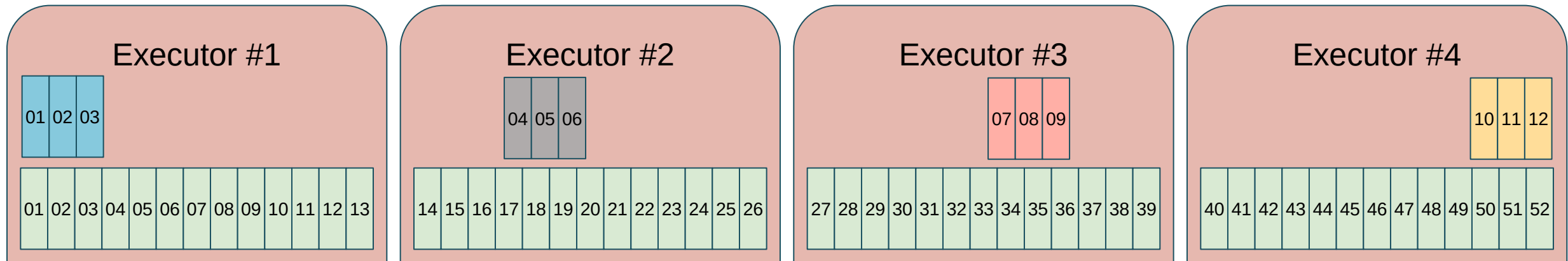
Driver

Because the Cities table is ≤ 10 MB,
the Driver plans a **BroadcastHashJoin**

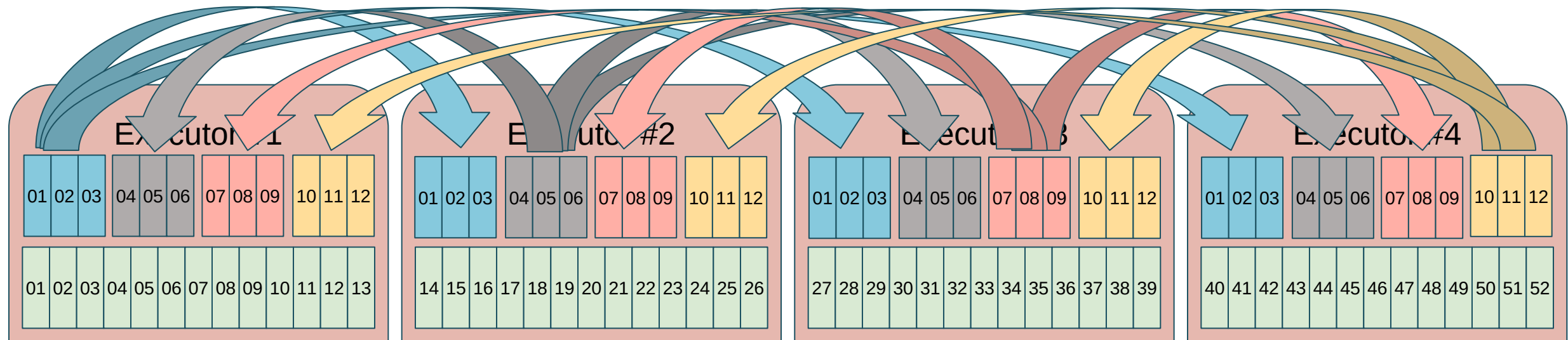
01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----



Each Executor reads in their assigned Partitions

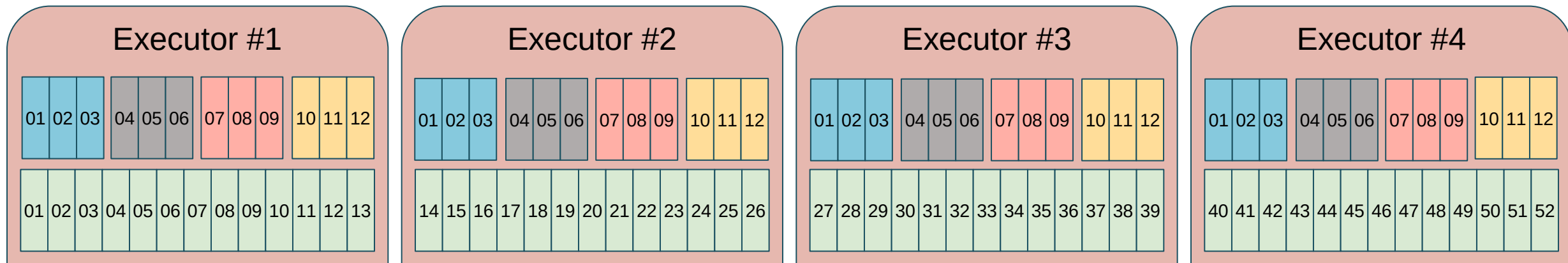


Each Executor then passes their Partitions to other Executors for the smaller Broadcasted table



Lastly, each Executor can now Join any two of its matched records because it has a complete copy of the Broadcasted table

No Shuffle needed for larger Table since each Executor has complete contents of smaller Table



Shuffles – Mitigation

1. Reduce network IO via fewer, larger Workers
2. Speed shuffle reads/writes via NVMe & SSDs



3. Reduce amount of Shuffled data
 - Remove unnecessary columns (SELECT)
 - Filter out unnecessary records (WHERE)
4. Denormalize datasets, esp when shuffle is rooted in a join
5. Employ Cost based Optimizer (CBO)

Reevaluate join strategy:

1. Reordering the join (vis CBO)
2. Dynamically Switching Join Strategies
 - Broadcast Hash Join (AQE)
 - Shuffle Hash Joins (default for Photon)
 - Sort-Merge Join (default for OS Spark)

PO 1.3 (con't)

Broadcast Join Lab

Start at Broadcast Join

Shuffle Write	
7.5 KiB	
4.6 KiB	
356.0 B	

These are the Shuffles from the 99 row and 12 row tables (stores + countries). We didn't need to Shuffle the 150m row table. Cool!

3. Spill

Executor JVM Heap Memory

`spark.executor.memory = <size>`

Reserved
Memory:
300 MB

User Memory:
1 -
`spark.memory
.fraction`

On-Heap Unified Memory Pool
`spark.memory.fraction = 0.6`

Storage:



Execution

`spark.memory.storageFraction = 0.5`

```
1 # Old way - This function is deprecated
2 #You spill when size of RDD partitions at end of stage > RAM amount available for shuffle buffer
3 #Increase the shuffle buffer by increasing the fraction of executor memory allocated to it
4
5 #spark.conf.get("spark.shuffle.memoryFraction")
6 spark.conf.set("spark.shuffle.memoryFraction", '0.5')
7 spark.conf.get("spark.shuffle.memoryFraction")
```

```
%scala
// Return the total amount of memory shared between
// execution and storage, in Bytes.
def getMaxMemory(memoryFraction: Double): Long = {
  val systemMemory =
    java.lang.Runtime.getRuntime.maxMemory()

  // Fixed amount of memory for non-storage, non-execution
  // purposes.
  val reservedMemory = 300 * 1024 * 1024
  val usableMemory = systemMemory - reservedMemory
  (usableMemory * memoryFraction).toLong
}

// Call the function and assign the result to a variable
val memoryFraction = 0.6 // Example memory fraction
val usableMemory = getMaxMemory(memoryFraction)

usableMemory
```

<https://michaelheil.medium.com/understanding-common-performance-issues-in-apache-spark-deep-dive-data-spill-7cdba81e697e>

3. Spill

```
%scala
// Create a large partition by mismanaging shuffle partitions
spark.conf.set("spark.sql.shuffle.partitions", 60)

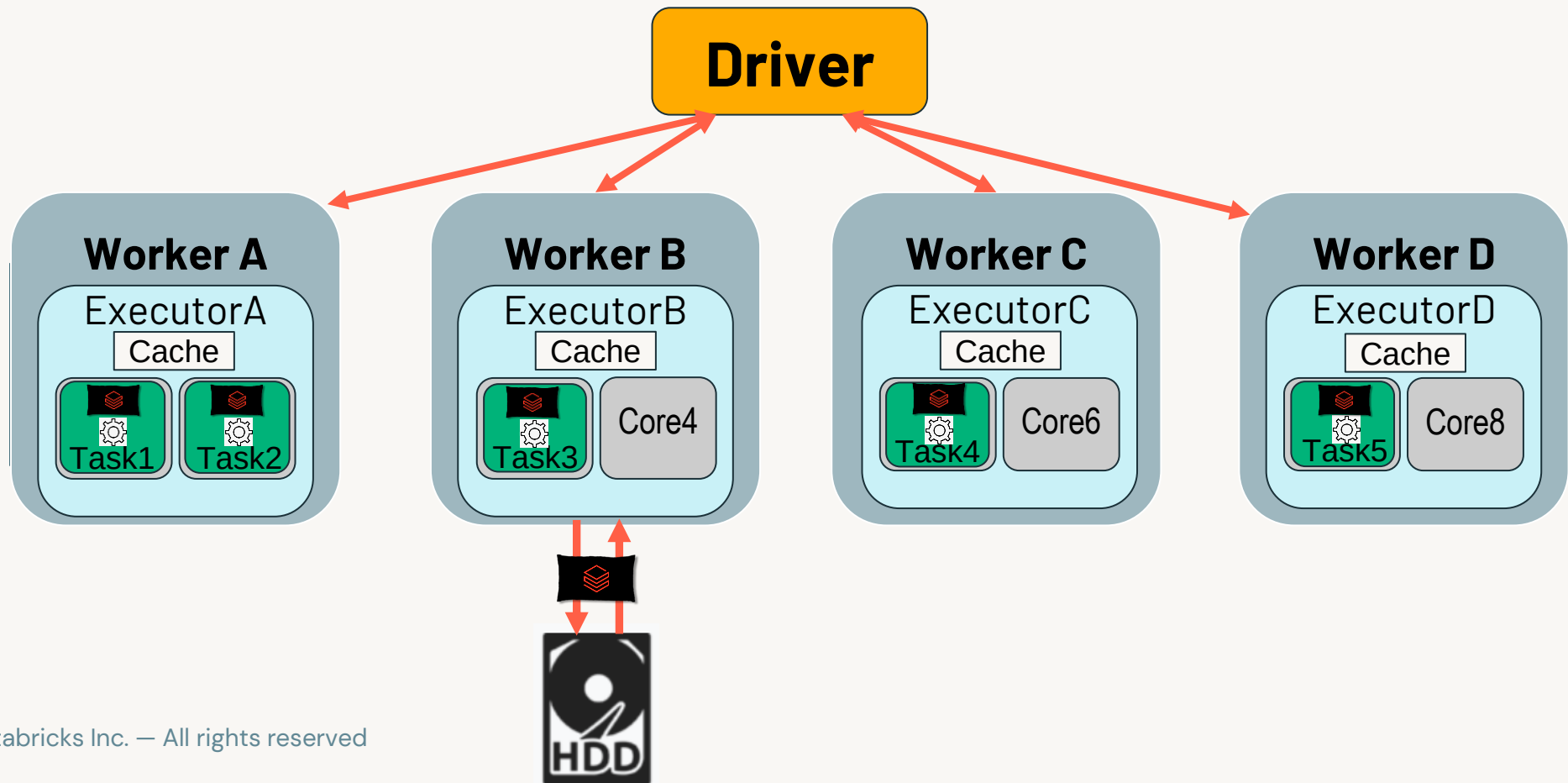
// The location of our non-skewed set of transactions
val trxPath = "wasbs://spark-ui-simulator@dbacademy.blob.core.windows.net/global-sales/transactions/2011-to-2018_int-only_60gb.delta"

spark.read.load(trxPath).orderBy("trx_id").write.format("noop").mode("overwrite").save()

//spark.read.format("delta").load(trxPath).orderBy("trx_id").write.format("noop").mode("overwrite").save()
```

How Spill from Memory to Disk occurs

Each Worker has 10GB RAM. Worker B already has a 5GB Task it's working on in Core 3. Driver hands off new 6GB Memory Partition to Worker B. But only 5GB free RAM is available. Worker B spills 6GB Memory Partition to Disk and tries later once Core 3 goes idle



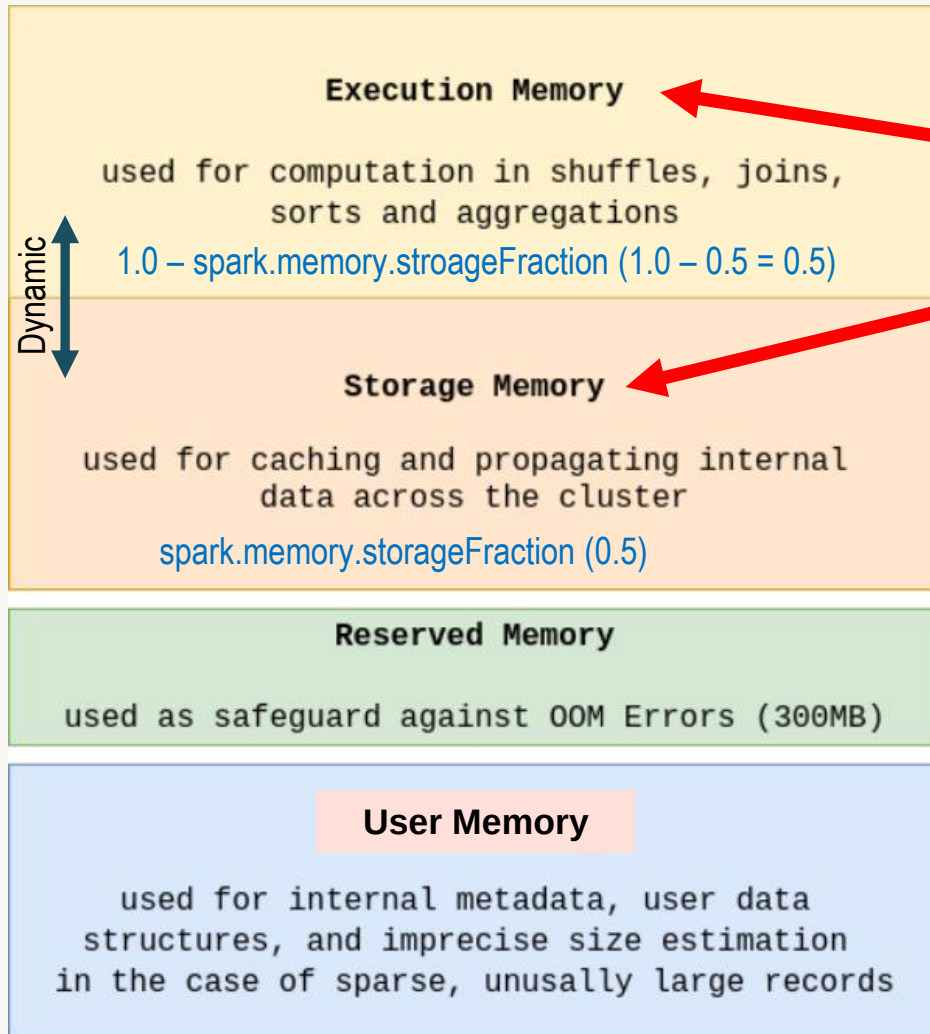
Spill

- Spill is the term used to refer to the act of moving data from RAM to disk, and later back into RAM again
- This occurs when a given partition is simply too large to fit into RAM
- In this case, Spark is forced into [potentially] expensive disk reads and writes to free up local RAM
- All of this just to avoid the dreaded OOM Error

```
java.lang.OutOfMemoryError: Java heap space
```



Bonus: How much RAM do I get?



Worker Type ?	Min Workers	Max Workers	Current
d3.xlarge	2	8	2

spark.memory.fraction: Fraction of (heap space - 300MB) used for **Execution** and **Storage**. The lower this is, the more frequently spills and cached data eviction occur. Default = .6 RAM. **38GB across 8 Cores/2 Wrks**

• **spark.memory.storageFraction:** Amount of storage memory immune to eviction, expressed as a fraction of the size of the region set aside by **spark.memory.fraction**. The higher this is, the less working memory available to Execution and tasks may spill to disk more often. Default = .5 of spark.memory.fraction. **Have up to 19GB across 8 Cores/2 Wrks**

<https://michaelheil.medium.com/understanding-common-performance-issues-in-apache-spark-deep-dive-data-spill-7cdba81e697e>
<https://spark.apache.org/docs/latest/configuration.html> <https://medium.com/swlh/spark-oom-error-closeup-462c7a01709d>

Spill – Examples

- Set `spark.sql.files.maxPartitionBytes` too high (default is 128 MB)
- The `explode()` of even a small array
- The `join()` or `crossJoin()` of two tables which generates lots of new rows
- The `join()` or `crossJoin()` of two tables by a skewed key
- The `groupBy()` where the column has low cardinality
- The `countDistinct()` and `size(collect_set())`
- Setting `spark.sql.shuffle.partitions` too low or wrong use of `repartition()`



Spill – Memory & Disk

Details for Stage 12 (Attempt 0)	
Total Time Across All Tasks: 14.9 h	
Locality Level Summary: Process local: 480	
Shuffle Read Size / Records: 726.5 GiB / 12239301032	
Spill (Memory): 990.1 GiB	
Spill (Disk): 59.5 GiB	

In the Spark UI, Spill is represented by two values:

- **Spill (Memory):** For the Partition that was spilled, this is size of that data as it existed in memory (**Deserialized**)
- **Spill (Disk):** For the Partition that was spilled (**will be Serialized**), this is the size of the data as it existed on disk

The two values are always presented together

The size on disk will always be smaller due to the natural **compression** gained in the act of Serializing that data before writing it to disk



Spill – Mitigations

- Allocate cluster with more RAM per Core (**don't > 128GB RAM due to JGC**)
- Address data Skew (if Skew causing Spill)
- Manage size of Spark partitions (via repartition() function)
- Avoid expensive operations like `explode()`
- Reduce amount data preemptively whenever possible (SELECT, WHERE)
- Use Spill Listener to detect spill in Cell output of notebook `Spilled Stages: 1`



Countries: 2.4KB
Stores: .36MB
Transactions: 18.42MB

```
FROM transactions
JOIN stores
  ON transactions.store_id = stores.id
JOIN countries
  ON transactions.country_id = countries.id
```

Metric	Min	25th percentile	Median	75th percentile	Max
Shuffle Write Size / Records	314.4 MiB / 48000000	326.6 MiB / 48000000	344 MiB / 50000000	349.8 MiB / 53980000	349.8 MiB / 53980000

PO 1.4L (Optional) Exploding Join

The transaction table has duplicate 'store_id'. So when do this 3-table Join, I'm going to have a large result set returned

	i_3^2 id	1.2 amount	i_3^2 country_id	i_3^2 store_id
97	96	8581.64	6	96
98	97	9934.39	7	97
99	98	4239.26	8	98
100	99	4854.99	9	99
101	100	7532.56	0	0
102	101	5154.71	1	1
103	102	9962.42	2	2
104	103	9354.87	3	3

See PO 1.4L (Solutions) Cell 18
Find Max Shuffle Write, then open that Stage to see spill

Shuffle Write 1334.9 MiB

Spill (Memory): 0.0 B
Spill (Disk): 927.6 MiB

(8) Spark Jobs

- ▶ Job 59 [View](#) (Stages: 1/1)
- ▶ Job 60 [View](#) (Stages: 1/1)
- ▶ Job 61 [View](#) (Stages: 1/1)
- ▼ Job 62 [View](#) (Stages: 1/1, 2 skipped)
 - Stage 68 0/4 succeeded [View](#) skipped
 - Stage 69 0/4 succeeded [View](#) skipped
 - Stage 70 4/4 succeeded [View](#)
- ▶ Job 63 [View](#) (Stages: 1/1, 4 skipped)
- ▶ Job 64 [View](#) (Stages: 1/1)
- ▶ Job 66 [View](#) (Stages: 1/1)
- ▶ Job 67 [View](#) (Stages: 1/1, 1 skipped)

4/4	Shuffle Read	Shuffle Write
	391.2 MiB	1335.0 MiB

6/6	Shuffle Read	Shuffle Write
	18.8 MiB	679.9 MiB

Shuffle Write Size /	314.5 MiB /	326.7 MiB /	343.9 MiB /	349.8 MiB /	349.8 MiB /
----------------------	-------------	-------------	-------------	-------------	-------------

Shuffle Write Size /	69.5 MiB /	74 MiB /	101.1 MiB /	173.6 MiB /	174.8 MiB /
----------------------	------------	----------	-------------	-------------	-------------

Join smaller table first

4	Shuffle Read	Shuffle Write
	388.7 KiB	386.9 KiB

	Shuffle Read	Shuffle Write
	460.0 B	460.0 B

4. Serialization



Performance problems with Serialization

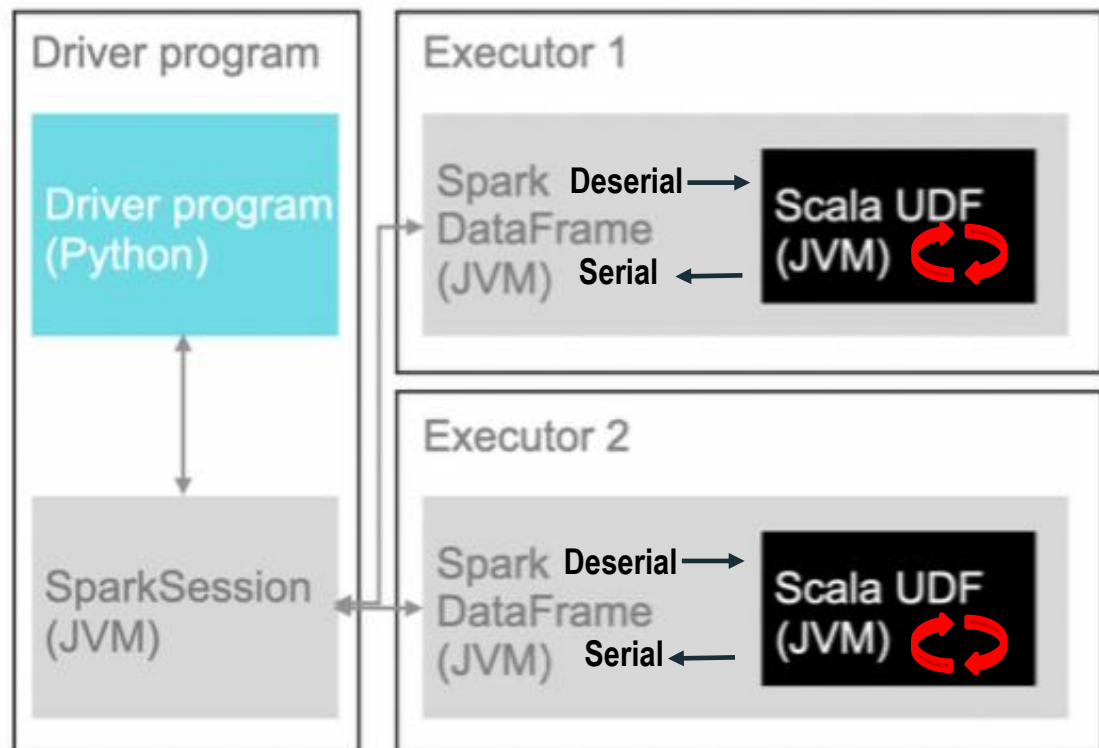
- Spark SQL functions and built-in functions are highly optimized
- All UDFs must be Deserialized and distributed to each Executor
- The parameters and return value of each UDF must be converted for each row of data before distributing to executors
- Python UDFs takes an even harder hit
 - The Python code has to be Pickled
 - Spark must instantiate a Python interpreter in each and every Executor
 - The Deserialization/Serialization of each row from Python to DataFrame costs even more

<https://spark.apache.org/docs/latest/api/sql/index.html>

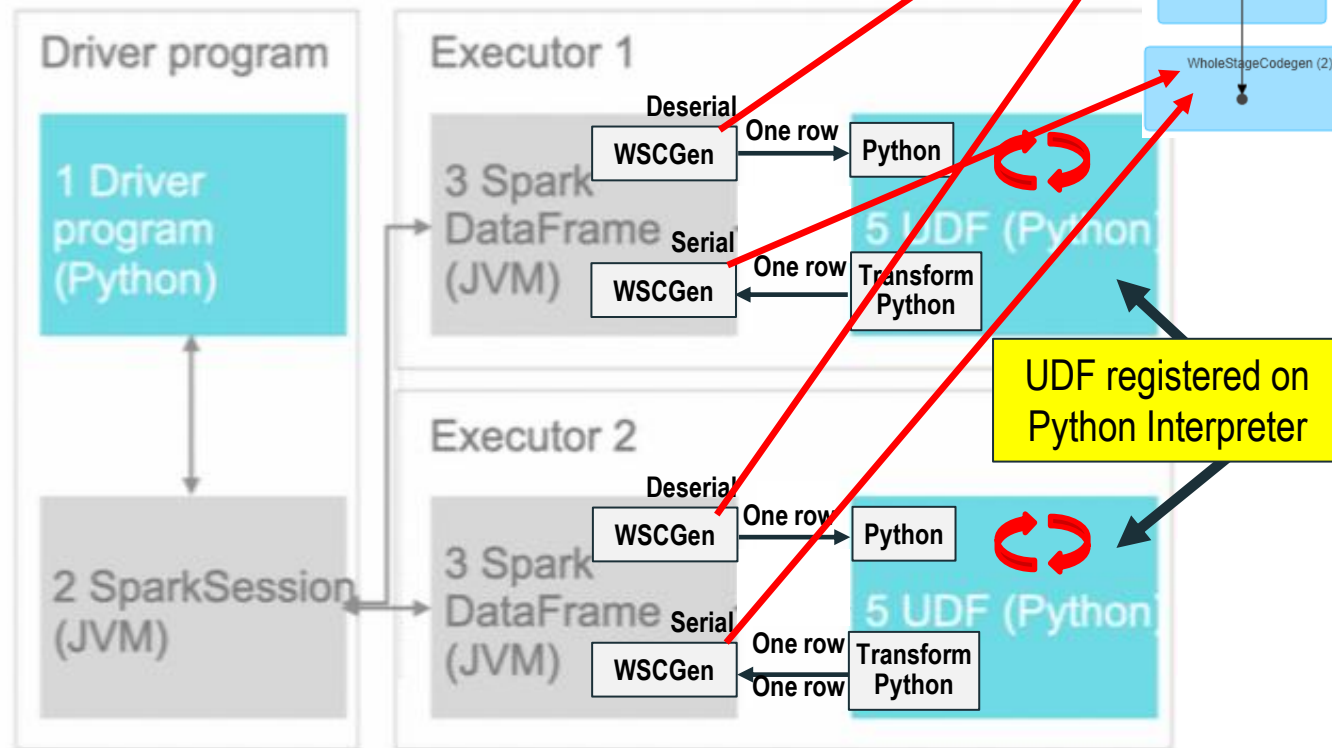


Scala and Python UDFs

Scala UDF



Python UDF



<https://medium.com/quantumblack/spark-udf-deep-insights-in-performance-f0a95a4d8c62>



Bonus: SQL functions beats Python UDF (1 of 2)

No need for additional Serialization plus Catalyst optimizer is operational with SQL UDFs

SQL

```
DROP FUNCTION IF EXISTS farh_to_cels;
CREATE FUNCTION farh_to_cels (farh DOUBLE)
RETURNS DOUBLE RETURN ((farh - 32) * 5/9);
```

```
SELECT farh_to_cels(temperature_F) as Farh_to_cels_convert
FROM temp_data;
```

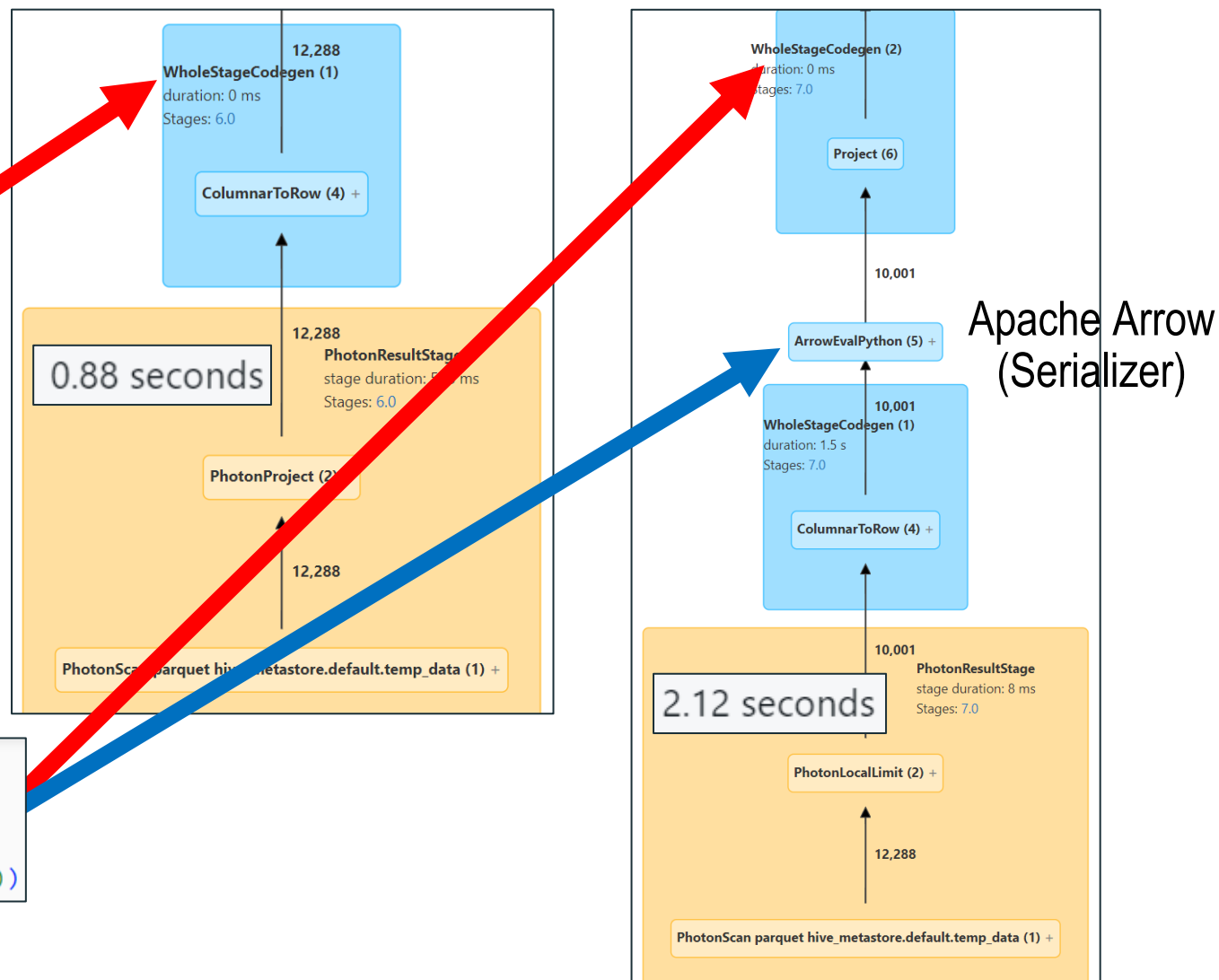
Python

```
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def F_to_Celcius_pandas(f: pd.Series) -> pd.Series:
    return (f - 32) * (5/9)
```

```
from pyspark.sql.functions import col

temp_df = spark.table("temp_data")
display(temp_df.select(F_to_Celcius_pandas(col("temperature_F"))))
```





Bonus: SQL functions beats Scala UDF (2 of 2)

Scala UDF has less overhead than Python UDF, but still (typically) doesn't beat SQL UDF

SQL

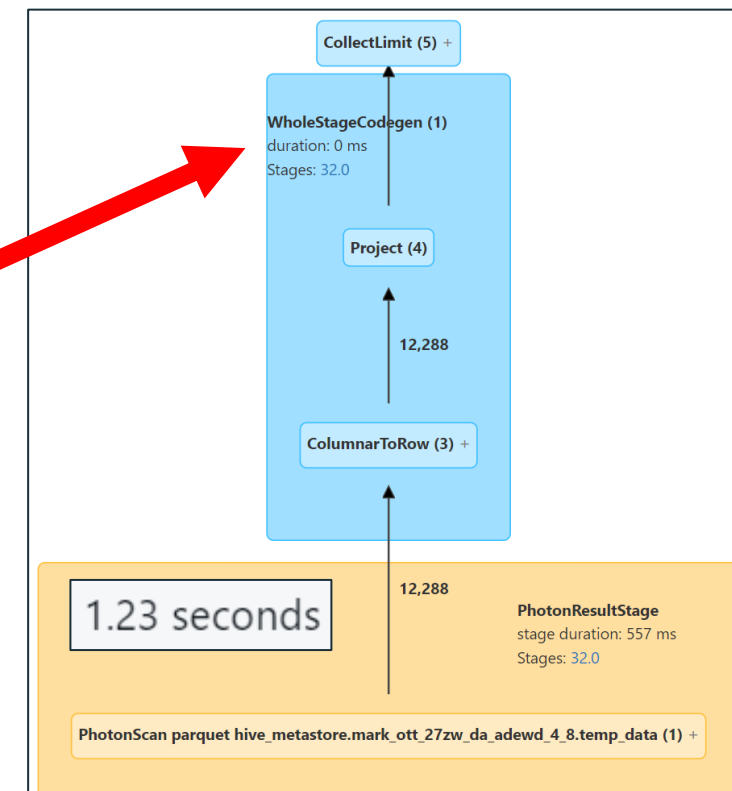
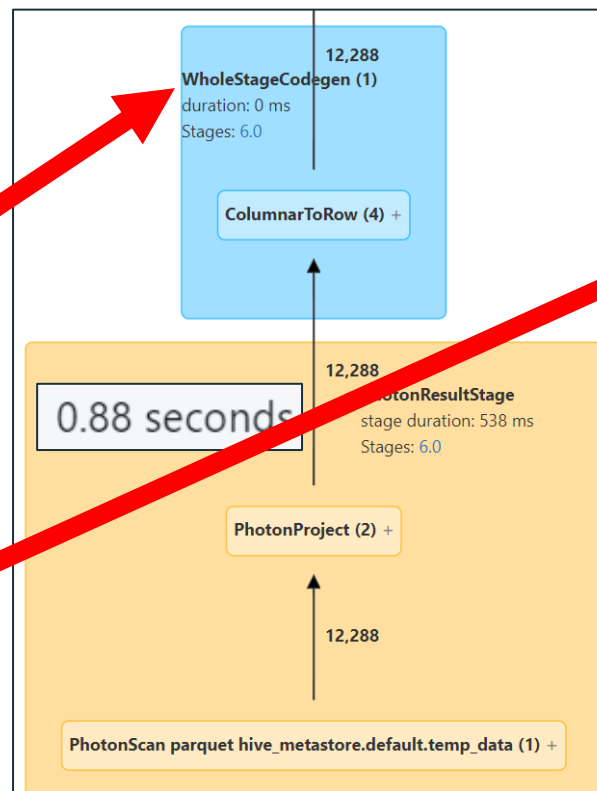
```
DROP FUNCTION IF EXISTS farh_to_cels;  
CREATE FUNCTION farh_to_cels (farh DOUBLE)  
RETURNS DOUBLE RETURN ((farh - 32) * 5/9);
```

```
SELECT farh_to_cels(temperature_F) as Farh_to_cels_convert  
FROM temp_data;
```

Scala

```
import org.apache.spark.sql.functions.udf  
import org.apache.spark.sql.types.DoubleType  
import org.apache.spark.sql.functions.col  
val df = spark.table("temp_data")  
  
// Define the UDF  
val tempUDF = udf((i: Double) => ((i * 9.0 / 5.0) + 32.0))  
  
// Apply the UDF to the column  
val df1 = df.withColumn("temperature_F", tempUDF(col("temperature_F")))
```

```
display(df1)
```



Mitigating Serialization issues

Order of performance from best to worst:

1. Built-in functions/SQL functions
2. Scala typed UDF
3. Panda Vectorized UDF
4. Python UDF

- Don't use UDFs (**with exception of SQL Functions which are Optimized**)
 - I challenge you to find a set of transformations that cannot be done with the built-in, continuously optimized, community supported, higher-order functions
- If you have to use UDFs in **Python** (common for Data Scientist) use the Panda Vectorized UDFs as opposed to the stock Python UDFs or Apache Arrow Optimised Python UDFs
- If you have to use UDFs in **Scala**, use Typed Transformations as opposed to the stock Scala UDFs



PO 1.5

User-Defined Functions



③ Cluster configuration

Cluster Types

Compute

All-purpose compute

Job compute

SQL warehouses

ALL PURPOSE COMPUTE

- Designed to handle **interactive** workloads, including streaming workloads
- Enable Auto-Scale to add capacity when needed and reduce time to answer
- Security considerations must be considered as auto-scaling can introduce additional risks

JOBS COMPUTE

- Run on ephemeral clusters that are created for the job, and terminate on completion
- Pre-scheduled or submitted via API
- Single-user
- Great for isolation and debugging
- Production and repeat **pipelines**
- Lower cost

SQL WAREHOUSE

- Built for high concurrency ad-hoc **SQL** analytics and **BI** serving
- **Photon** included
- Recommended shared warehouse for ad-hoc SQL analytics, isolated warehouse for specific workloads
- Serverless available for instant startup and lower TCO



Cluster Autoscaling



Worker type ⓘ

r6id.xlarge 32 GB Memory, 4 Cores | v

Min workers 2 Max workers 8

Driver type

Same as worker 32 GB Memory, 4 Cores | v

☒ Enable autoscaling ⓘ

☐ Enable autoscaling local storage ⓘ

- Dynamically resizes cluster based on Workload
 - Can run faster than a statically-sized, under-provisioned cluster
 - Can reduce overall costs compared to a statically-sized cluster
- Setting range for the number of workers requires some experimenting

Use Case	Auto Scaling Range
Ad-hoc usage or business analytics	Large variance
Production batch jobs	Not needed or buffer on upper limit
Streaming	Available in Delta Live Tables





World record achieving query engine



Save on compute costs

- ETL customers are saving up to 40% on compute cost



Fast query performance

- Built for modern hardware with up to 12x better price/perf compared to other cloud data warehouses



No code changes

- Spark APIs that can do exploration, ETL, big data, small data, low latency, high concurrency, batch, and streaming



Broad language support

- Support for SQL, Python, Scala, R, and Java

Performance

Databricks runtime version ⓘ

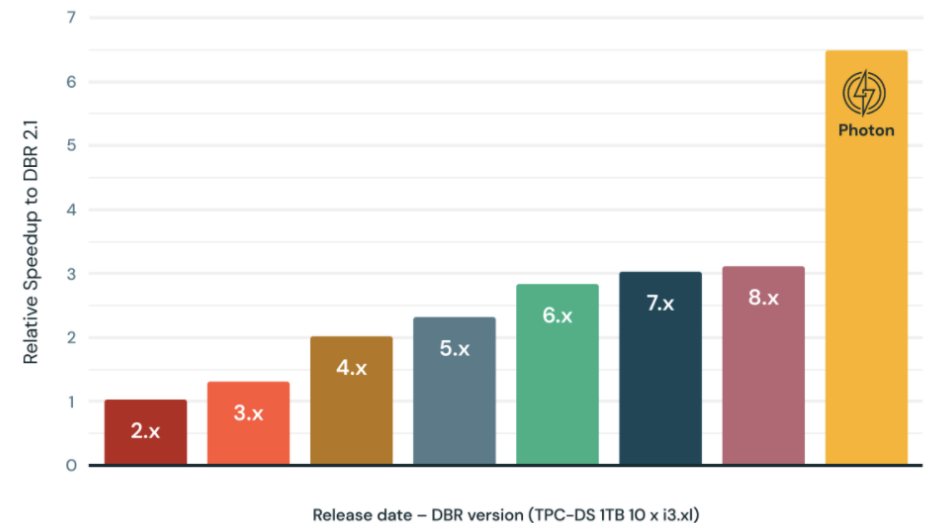
Runtime: 14.3 LTS (Scala 2.12, Spark 3.5.0)

☒ Use Photon Acceleration ⓘ

Databricks Sets Official Data Warehousing Performance Record

Relative Speedup to DBR 2.1 by DBR version

Higher is better



Cluster Optimization Recommendations

1. **DS & DE development:** all-purpose compute, auto-scale and auto-stop enabled, develop & test on a subset of the data
2. **Ingestion & ETL jobs:** jobs compute, size accordingly to job SLA
3. **Ad-hoc SQL analytics:** (serverless) SQL warehouse, auto-scale and auto-stop enabled
4. **BI Reporting:** isolated SQL warehouse, sized according to BI SLAs
5. **Best practices:**
 - a. Enable spot instances on Worker nodes
 - b. Use the latest LTS Databricks Runtime when possible
 - c. Use Photon for best TCO when applicable
 - d. Use latest gen VM, start with general purpose, then test memory/compute optimized



Bonus: Cluster configuration

What will the Cluster be used for?

- | | |
|---------------------------------------|------------------------------|
| 1. Ad Hoc Data Analysis | 4. Structured Streaming Jobs |
| 2. Reporting Generation | 5. (Scheduled) Batch ETL |
| 3. Training ML & Deep Learning Models | 6. Data Pipelines |

How a cluster is used often follows the persona of the person using it

What – Ad Hoc Data Analysis

- **Level:** The amount of memory and compute is difficult to estimate
- **Shuffles:** Often employs arbitrary Wide operations (e.g. joins).
Tuning needs to be done in the general case, not query specific
- **Caching:** Repetitive queries can benefit from [Delta caching](#) (automatic).
[Spark caching](#) should only be used to temporarily store aggregates
- **Opportunity:** Little opportunity for specific tuning
- **SLAs:** Generally not applicable

The dynamic nature of this activity makes planning and tuning hard

What – Reporting Generation

- **Level:** The amount of memory and compute can be fine tuned
- **Shuffles:** Shuffles can be mitigated with use-case specific datasets
- **Caching:** Generally does not require repetitive reads
- **Opportunity:** More opportunity for job-specific tuning
- **SLAs:** May be imposed in that the report is a job in and of itself

Presumably 'lighter' than full batch ETL, this activity is still nothing more than periodically running a job

Canned queries are candidates for Tuning

What – Training ML & Deep Learning Models

- **Level – first run:** Memory and compute is difficult to estimate
 - **Level – secondary runs:** Memory and compute can be fine tuned
 - **SLAs – first run:** Difficult to impose requirements on first runs
 - **SLAs – secondary runs:** May be imposed when retraining a model
- **Shuffles:** Shuffles can be mitigated with use-case specific datasets
 - **Caching:** Entire datasets are often cached to achieve maximum performance via Spark and Delta caches ([add RAM](#))
 - In some cases [GPUs](#) may be warranted to maximize memory and/or compute

What – Structured Streaming Jobs

- **Level:** The amount of memory and compute are often **Over-provisioned** to enable a stopped stream to catch up
- **Shuffles:** Shuffles **are** mitigated by AQE and when joining datasets, with use case specific datasets
- **Caching:** Generally, not employed in a Streaming job
- **Opportunity:** High opportunity for specific tuning
- **SLAs:** Dictated by the throughput of the stream – possibly schedule & terminate short-running streams (e.g. 2 hr, every 1 wk)

Dedicate Streaming jobs to its own Cluster. Don't share

What – Batch ETL

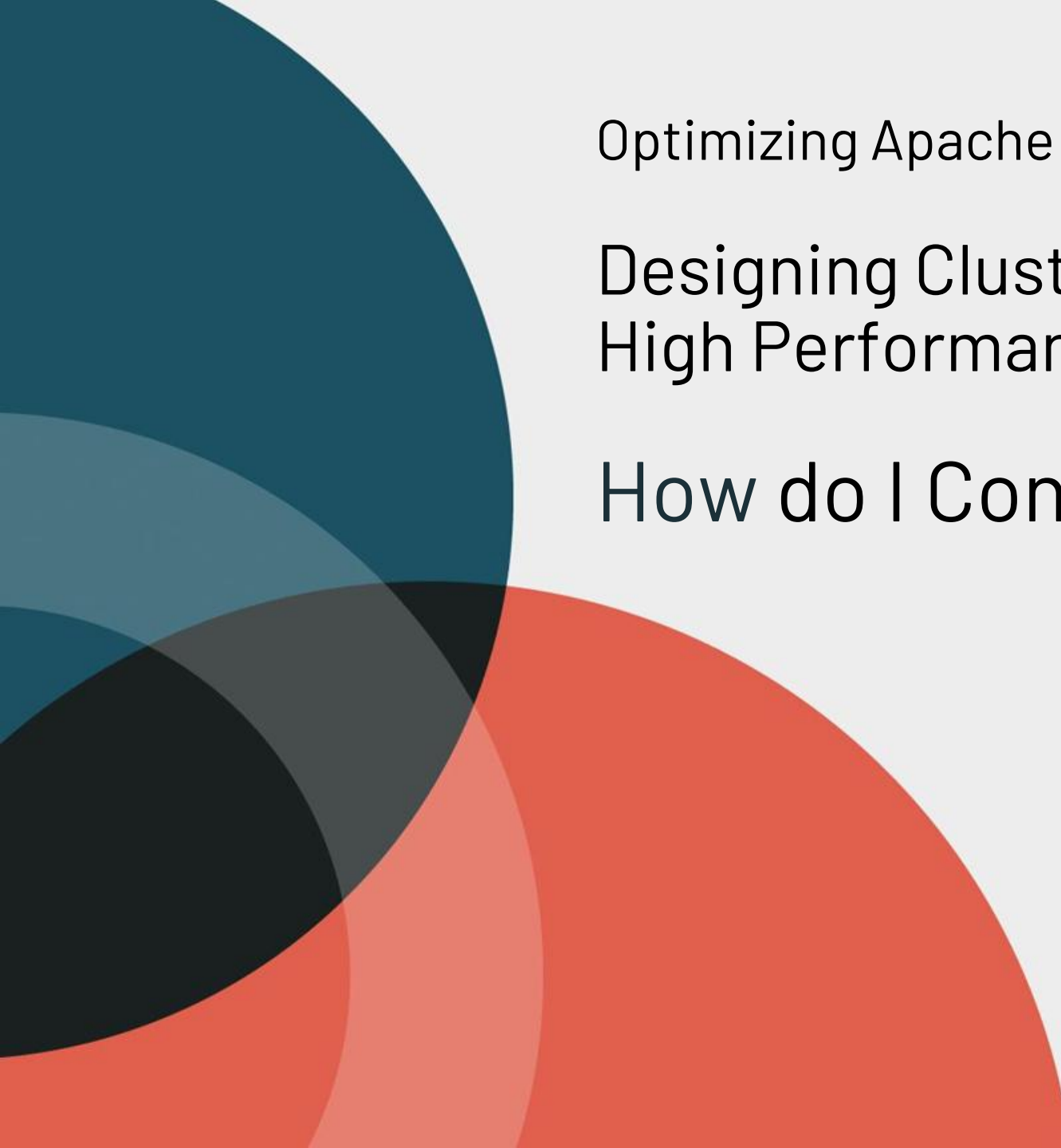
- **Level:** The amount of memory and compute can be fine tuned
- **Shuffles:** Shuffles 'cannot' be mitigated with use-case specific datasets – should be mitigated by fine tuning
- **Caching:** Generally does not require repetitive reads
(opt for **Delta caching** over **Spark caching** if needed)
- **Opportunity:** High tuning opportunity as it pertains to a specific job
- **SLAs:** Post tuning, different SLAs levels are met by adjusting compute levels (e.g. size of and number of executors and thus cores)

Scheduled Jobs. Raw data > Clean > Aggr
Dedicated Cluster so have Deterministic conditions

What – Data Pipelines

- Batch ETL can be thought of as one instance of data-in and data-out
- Data Pipelines can be thought of as many batches orchestrated to work together
- Do you use one general-purpose cluster for the entire pipeline?
Or does each 'job' get a highly-tuned configuration?
- SLAs will most likely decide between a shared-cluster design VS a cluster-per-batch design
 - For example, low-latency pipelines might require a cluster-per-batch design where higher latency pipelines might use a shared cluster

ELT workloads



Optimizing Apache Spark

Designing Clusters for
High Performance

How do I Control & Predict the **Costs**?

Costs – Compute Levels

The price between a **Level-N** VM and a **Level-N+1** VM is 2x the cost,
with 2x the resources

Level	Cores	Size	Cloud Provider – A		Cloud Provider – B	
1	4	Small	30.5 GB	\$0.266 / hour	28 GB	\$0.299 / hour
2	8	Medium	61.0 GB	\$0.532 / hour	56 GB	\$0.598 / hour
3	16	Large	122.0 GB	\$1.064 / hour	112 GB	\$1.196 / hour

Costs – Actual Consumption Cost

Assume you have a Job with 256 partitions and each Partition takes 2 minute to process



Level	Cores	VMs	Max Compute (cores * VMs)	Iterations (max/part)	Actual Duration (iterations * min)	~Price/Hour (level \$ * VMs)	VM Costs (VMs * dur * price / 60)
1	4	1	4	64	128 minutes	\$0.283	60¢
1	4	64	256	1	2 minutes	\$0.283	60¢
2	8	1	8	32	64 minutes	\$0.565	60¢
3	16	1	16	16	32 minutes	\$1.130	60¢
3	16	8	128	2	4 minutes	\$1.130	60¢

Costs – Developer (Human) Costs

Another factor to consider is the cost of the Developers:

- What are they doing when the job is running?
- How much time does it take to tune?



Level	Cores	VMs	Max Compute (cores * VMs)	Iterations (max/part)	Actual Durations (iterations * min)	~Price/Hour (level \$ * VMs)	VM Costs	Dev \$/Hour (\$50 * dur / 60)
1	4	1	4	64	128 min	\$0.283	60¢	\$106.66
1	4	64	256	1	2 min	\$0.283	60¢	\$1.66
2	8	1	8	32	64 min	\$0.565	60¢	\$53.33
3	16	1	16	16	32 min	\$1.130	60¢	\$26.66
3	16	64	1024	1	4 min	\$1.130	60¢	\$22.22



Optimizing Apache Spark

Designing Clusters for
High Performance

VM Selection

Have open mind when picking VM

- For AWS. i3's aren't always the best. Explore m5d and r5d
- Enable caching if needed
 - This will probably change immediately once graviton are supported
 - M5d and r5d have better processors, similar (albeit smaller) local disk and much more stable spot markets
- For Azure, the L-series are kind of slow, try the eav4, dav4 and f-series
 - The ACU is very useful
- GCP defaults are pretty good
- Probably don't need network optimized instance types unless using Photon



How to choose right VM is simple...

- Just a series of IFTTT questions and rules of thumb!
 - Side note – if you 2x the cluster and it runs in 1/2 time, it costs the same
- Rules of thumb
 - First run: `set spark.sql.shuffle.partitions = 2x # of cores`
 - Keep total memory available to the machine less than 128gb
 - Number of cores should be a ratio of 1 core to 128mb → 2gb of *reads*
 - Some caveats may apply
 - Avoid setting any other configs at first (don't carry over configs from legacy platforms unless absolutely necessary)

Scan parquet +details	
Stages: 655.0	
file sorting by size time	2 ms
cache writes size (uncompressed) total (min, med, max)	262.4 MiB (576.8 KiB, 1031.5 KiB, 1085.7 KiB)
time spent in the cache locality manager in milliseconds total (min, med, max)	37 ms (0 ms, 0 ms, 37 ms)
number of files read	1,027
filesystem read data size total (min, med, max)	279.8 MiB (620.5 KiB, 1101.1 KiB, 1155.4 KiB)
cache async file status fetch waiting time total (min, med, max)	0 ms (0 ms, 0 ms, 0 ms)
scan time total (min, med, max)	14.6 m (1.9 s, 3.2 s, 7.1 s)
filesystem read data size (sampled) total (min, med, max)	279.8 MiB (620.5 KiB, 1101.1 KiB, 1155.4 KiB)
filesystem read time (sampled) total (min, med, max)	12.4 m (1.7 s, 2.8 s, 5.5 s)
metadata time	8 ms
size of files read	236.7 GiB

VM Selection: Effect on Shuffles

Assume 256
Shuffle Partitions

If cost is NOT a primary factor, what about the effect on Performance?

Level	Cores	VMs	Max Compute (cores * VMs)	Iterations (max/part)	Actual Durations (iterations * min)	Notes
1	4	1	4	64	128 minutes	<u>No network IO</u>
1	4	64	256	1	2 minutes	<u>Heavy network IO between 64 VMs</u>
2	8	1	8	32	64 minutes	No network IO
3	16	1	16	16	32 minutes	No network IO
3	16	8	128	2	4 minutes	Reasonable(?) network IO
7	256	1	256	1	2 minutes	<u>Most optimal Shuffle experience</u>

VM Selection: Worker Categories

Worker Type ?

i3.xlarge 30.5 GB Mem

Storage Optimized (Delta Cache Accelerated)

So which VM should we use? Start by breaking them down by Category:

Worker Type Categorization	Amazon	GBs	Cores	MS Azure	GBs	Cores
Memory Optimized (+ RAM)	r4.xlarge	30.5	4	DS12_v2	28.0	4
Compute Optimized (+ Cores)	c5.xlarge	8.0	4	F4s	8.0	4
Storage Optimized Delta Cache	i3.xlarge**	30.5	4	L4s**	32.0	4
GPU Optimized	p2.xlarge	61.0	1	NC6s_v3	112.0	1
General Purpose	m5.xlarge	16.0	4	DS3_v2	14.0	4

Only a sample of VMs are shown here. Each type is represented by N different levels of memory and cores. Availability varies by Cloud

VM Categories

Memory Optimized

- ML workload with data caching
- If shuffle-spill remains a problem (no other mitigation strategy)
- When spark-caching is a requirement

Compute Optimized

- ETL with full file scans and no data reuse
- Structured Streaming Jobs

General Purpose

- Used in absence of specific requirements

Storage Optimized

- Optimized with **Delta IO Caching** !!
- ML & DL workloads with data caching
- Data Analysis / Analytics
- If shuffle-spill remains a problem (no other mitigation strategy)
- When spark-caching is a requirement

GPU Optimized

- ML & Deep Learning workloads with exceptionally large memory and compute requirements (presumes caching)

Memory and Storage Optimized VMs will help with Shuffle/Spill but can ignore an underlying problem such as Skew

Guessing Compute Level

Use Ganglia to find out

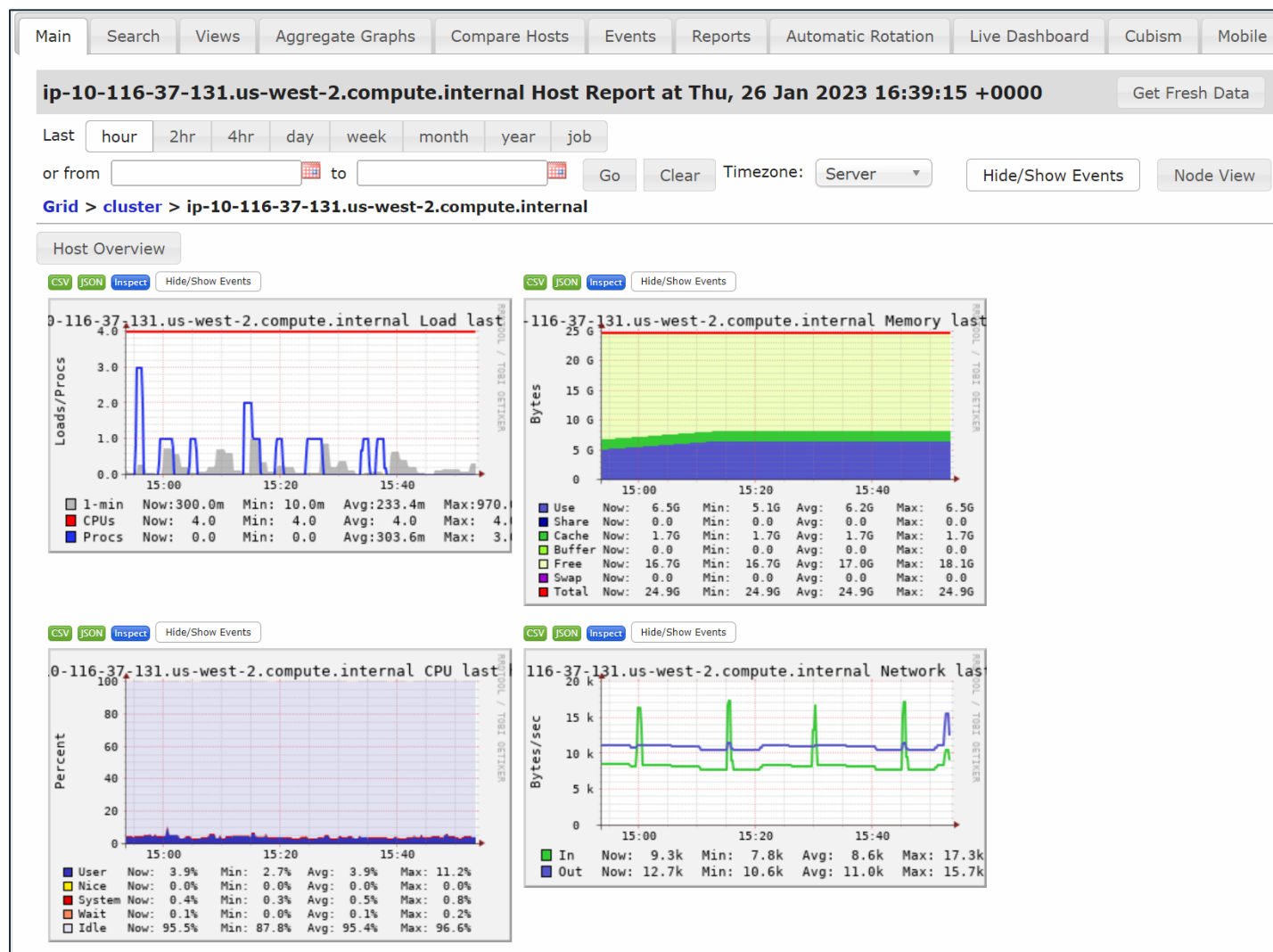
Experimentation is easy...

- Make a guess
- If you are Spilling, assume you need more RAM (unless you have Skew)
- If your Shuffles are slow, increase VM Level while decreasing the number of VMs ([Slide 37](#))
- How many iterations did it take? Increasing the VM Level or number of VMs for more cores
- **Is your cluster underutilized?** Reduce the VM level or number of VMs for fewer cores
- Expect this processes to take a fair amount of trial and error (aka time, aka money)

Ganglia

<https://www.youtube.com/watch?v=9fa8dnKbfsU&t=541s>

Great way to see if you have under-utilized or over-utilized Worker nodes



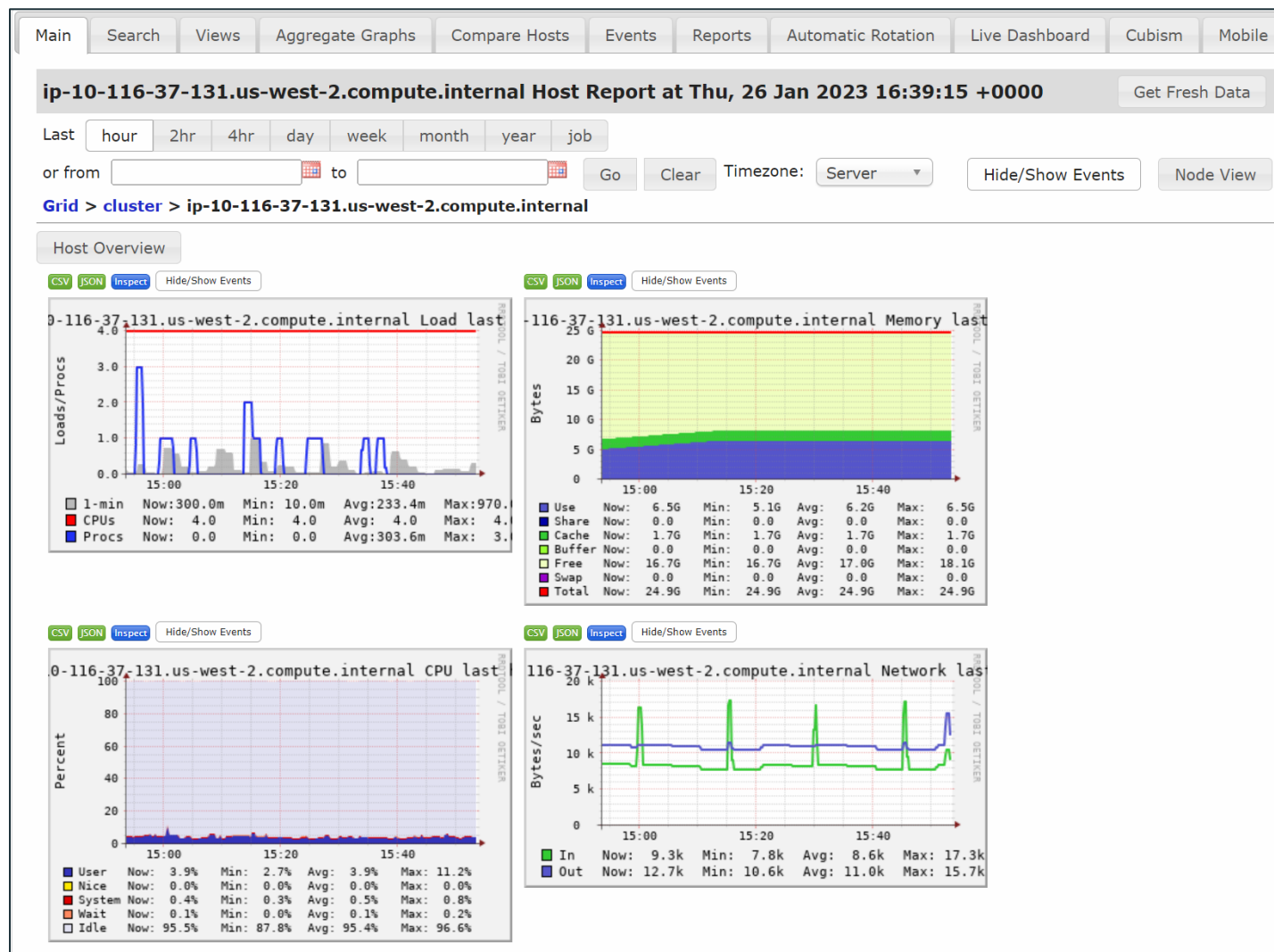
Estimate Number of VM's

1. Calculate the data's size on disk
 2. **spark.sql.files.maxPartitionBytes?**
 3. Compute # of partitions or cheat and run **df.rdd.getNumPartitions()**
 4. Decide which VM Worker Type Category
 5. Based on the SLA, quota, and budget, select the type and level of VM
 6. Select the number of iterations
 7. **Compute the number of VMs**
 8. Adjust/experiment/retest - Time (and money) saved with semi-reasonable configuration
1. Assume we have 100 GB or **102,400 MB**
 2. Assume **maxPartitionBytes** is **128 MB**
 3. **102,400 MB / 128 MB = 800 partitions**
 4. **Compute Optimized**
 5. **Level 5, 144 GB, 72 cores** each
 6. **2 iterations**
 7. **800 par / 72 cores / 2 iterations = 6 VMs**

Ganglia

<https://www.youtube.com/watch?v=9fa8dnKbfsU&t=541s>

Great way to see if you have under-utilized or over-utilized Worker nodes



<https://docs.databricks.com/clusters/cluster-config-best-practices.html>

<https://subhamkharwal.medium.com/pyspark-the-cluster-configuration-ed802db3e7cf>

Cluster Configurations Scenarios

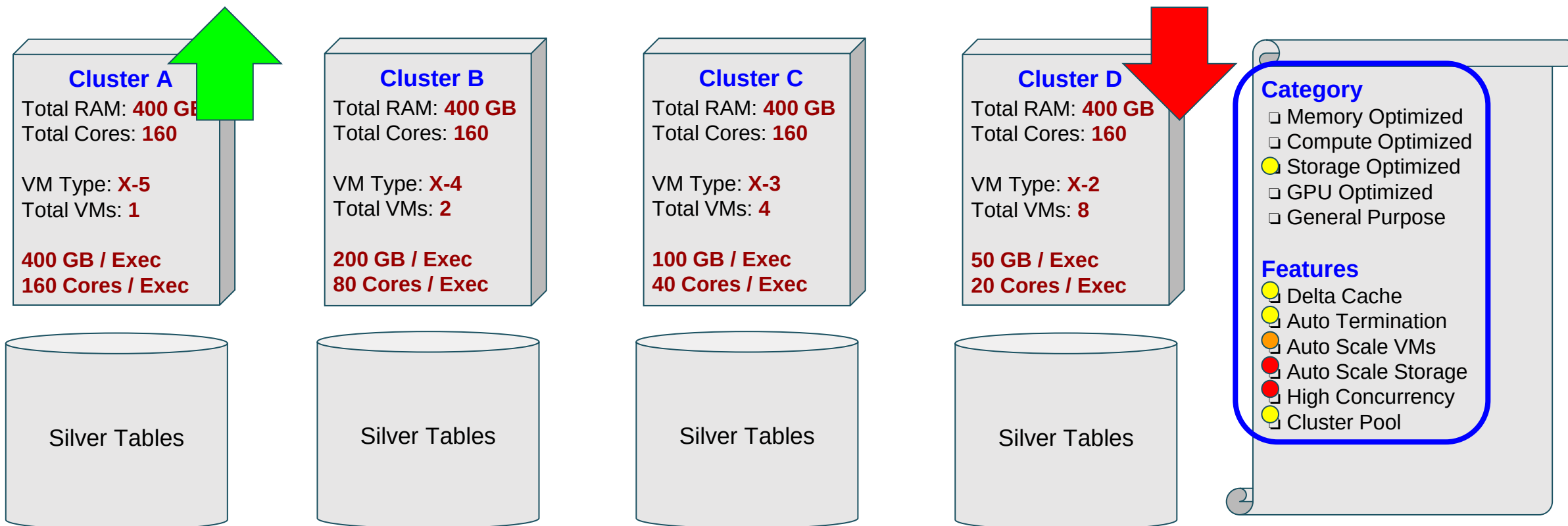
Should I run Feature Enhancement on one Cluster type, and then Model on different type?

1. You might want std cluster for ETL and GPU cluster for DL model building?
2. Or perhaps photon-enabled for ETL, and not for the training which doesn't benefit.
3. Or the ML workloads might better use CPU- or memory-heavy instances

Cluster Configurations Scenarios

Typical Analyst

Which of the following cluster configurations is best / least suited for a **single SQL** or **Data Analyst**?

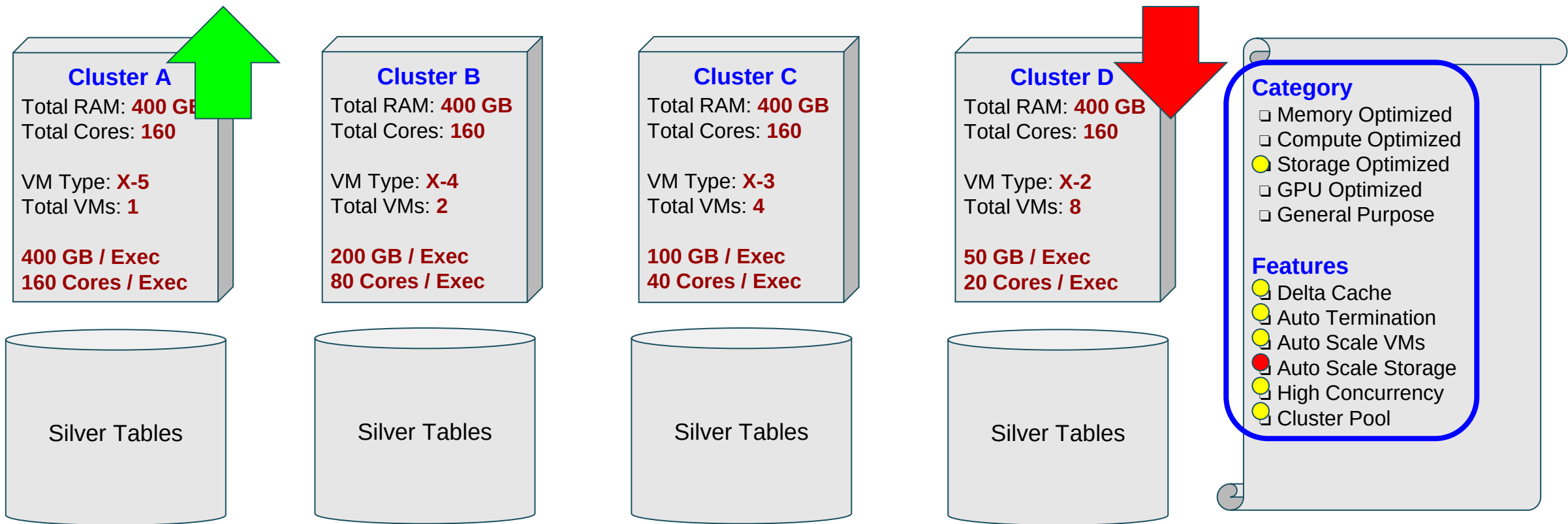


For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Cluster Configurations Scenarios

Team of Analyst (20 users)

Which of the following cluster configurations is best / least suited for a team of SQL and/or Data Analysts?

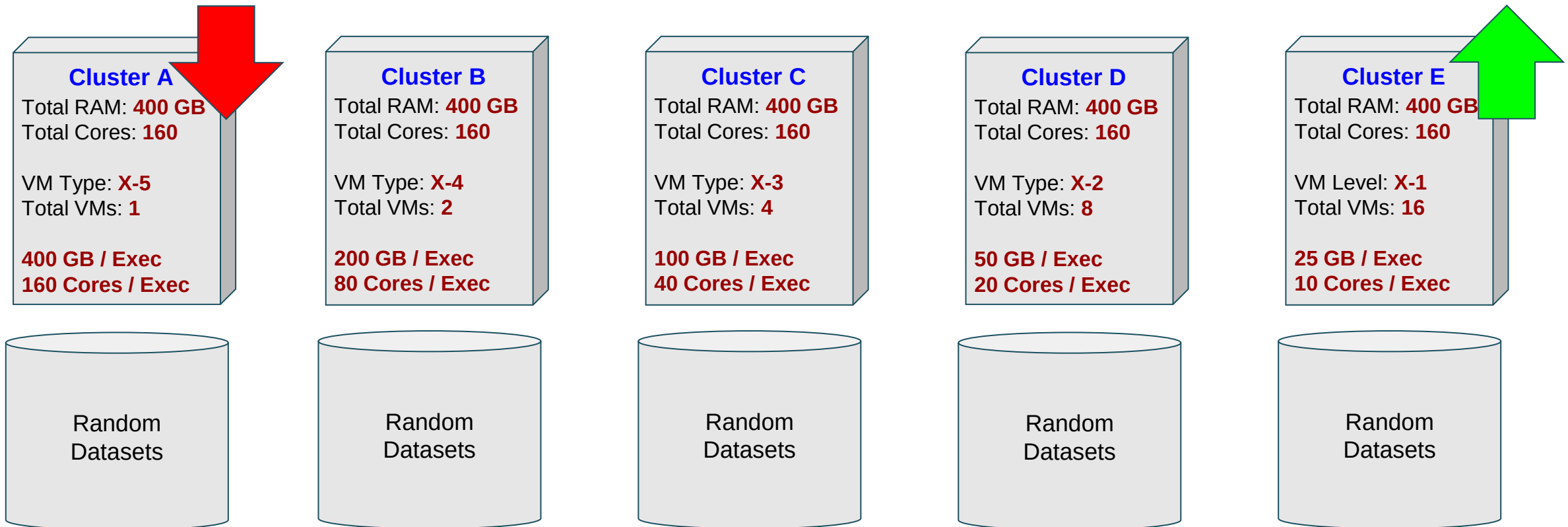


For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Cluster Configurations Scenarios

Cluster Stability

Which of the following cluster configurations is most / least likely to survive a [random] executor failure during a long-running job?

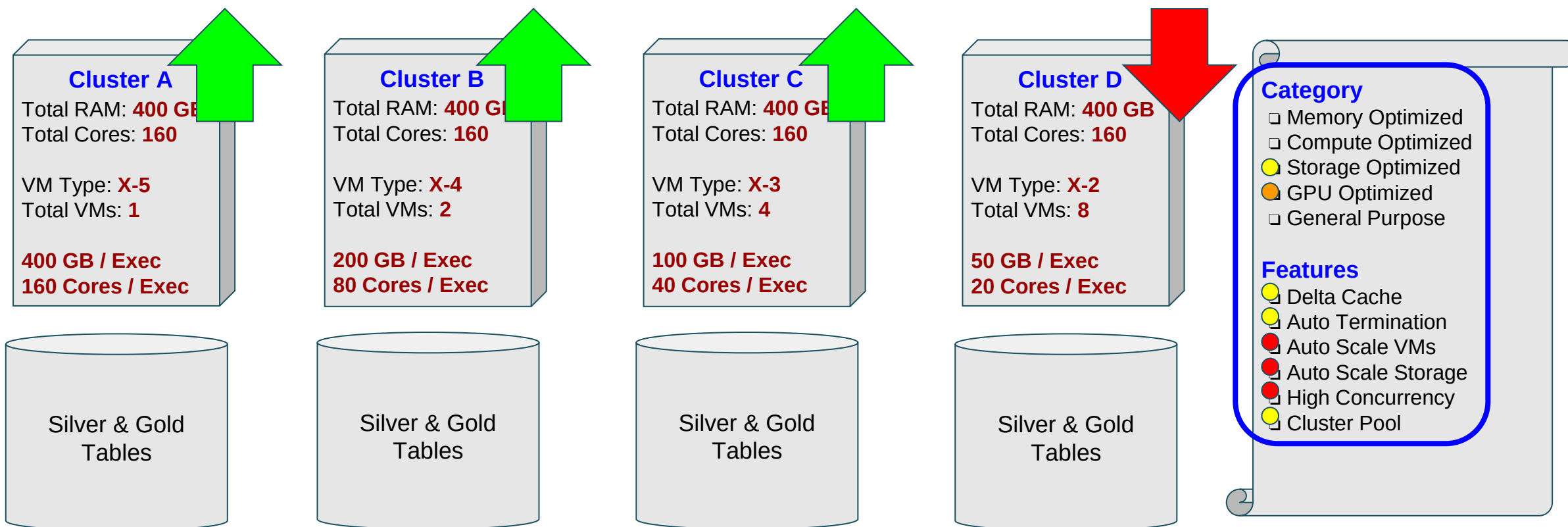


For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Cluster Configurations Scenarios

Training ML Models

Which of the following cluster configurations is best / least suited for training the **first iteration of an ML or DL Model**?



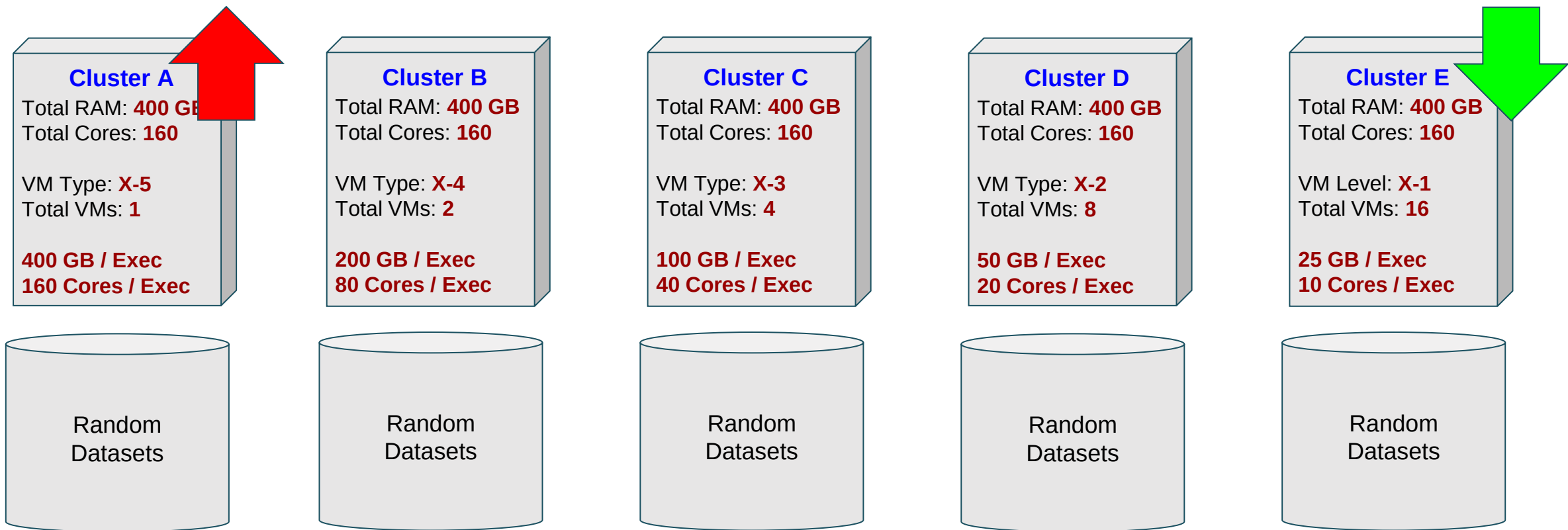
For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Cluster Configurations Scenarios

Java Garbage Collection

It will create a lot of Garbage Collection (GC) issues leading to slow performance

Which of the following cluster configurations is most / least likely to be adversely impacted by a long **garbage collection** sweep?



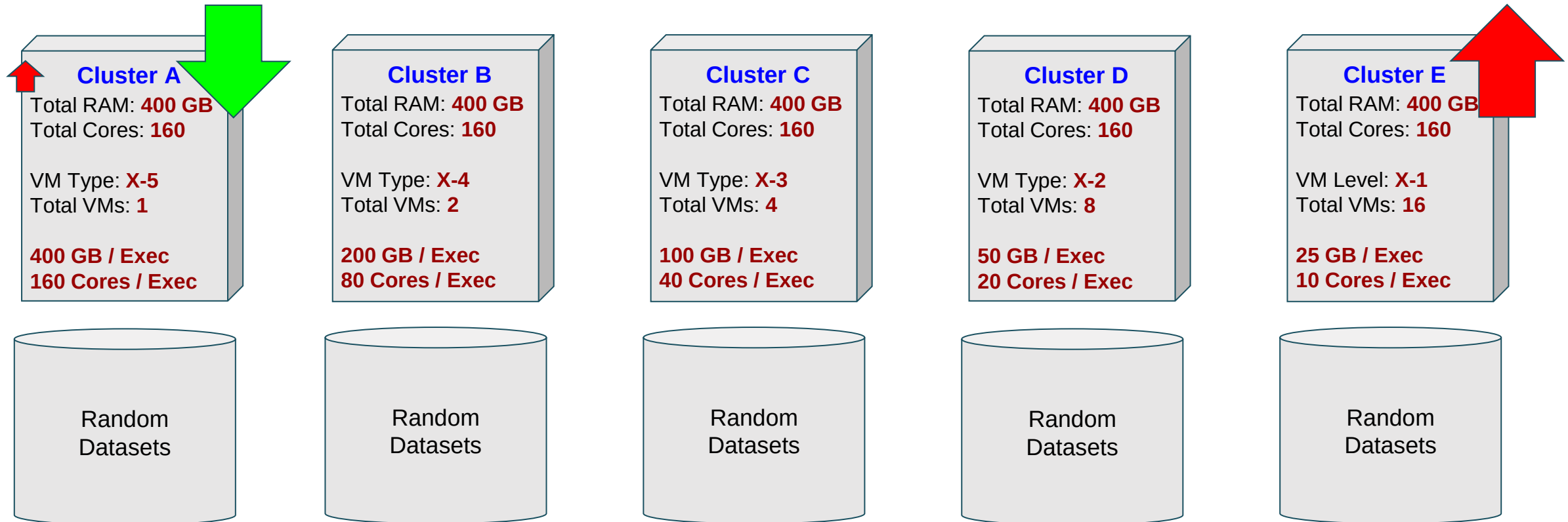
For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Cluster Configurations Scenarios

General OOM Error

If Driver overloads one VM in Cluster E (while other VM's remain idle), that one VM could OOM if amount of RAM exceeds $.60 \times 25\text{GB}$. But Cluster A only has 1 VM so it has $.60 \times 400\text{GB}$ to work with. Cluster A has less chance of OOM.

Which of the following cluster configurations is most / least likely to encounter an OOM Error?



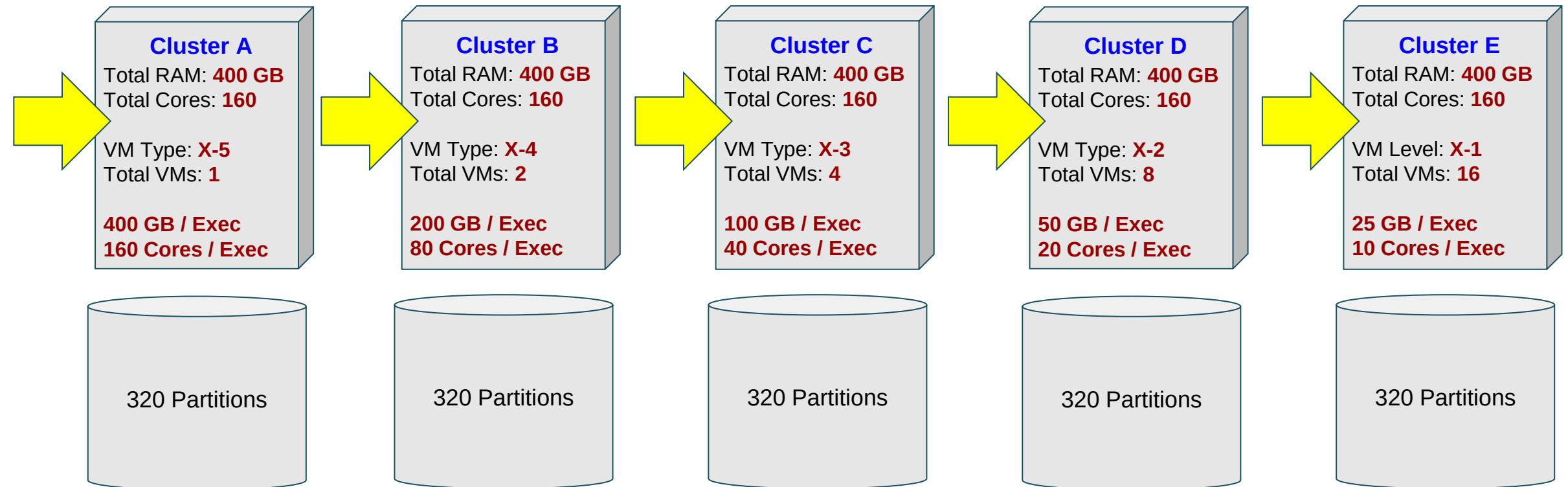
For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Cluster Configurations Scenarios

More Cores == More Money

No Cost Difference

Assuming the data in 320 partitions is equally distributed, which cluster configuration will cost the most / least amount of money for this job?

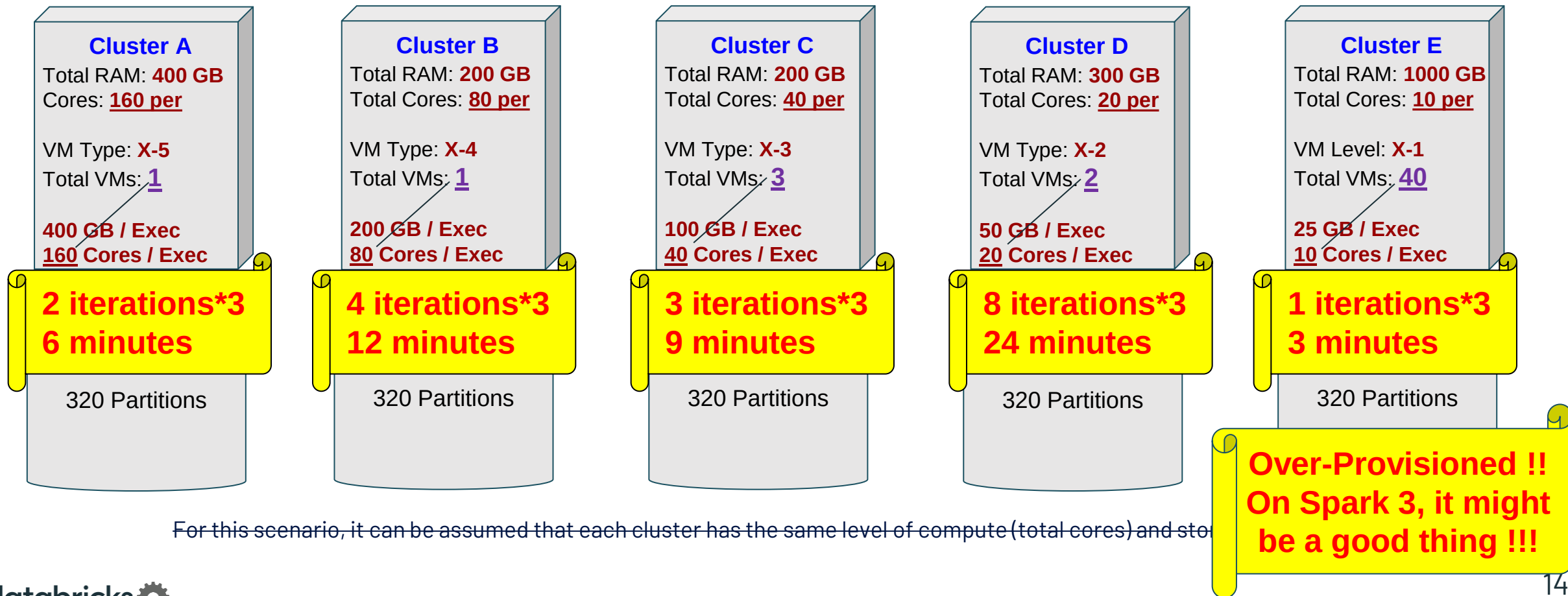


For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Cluster Configurations Scenarios

More Cores == More Money

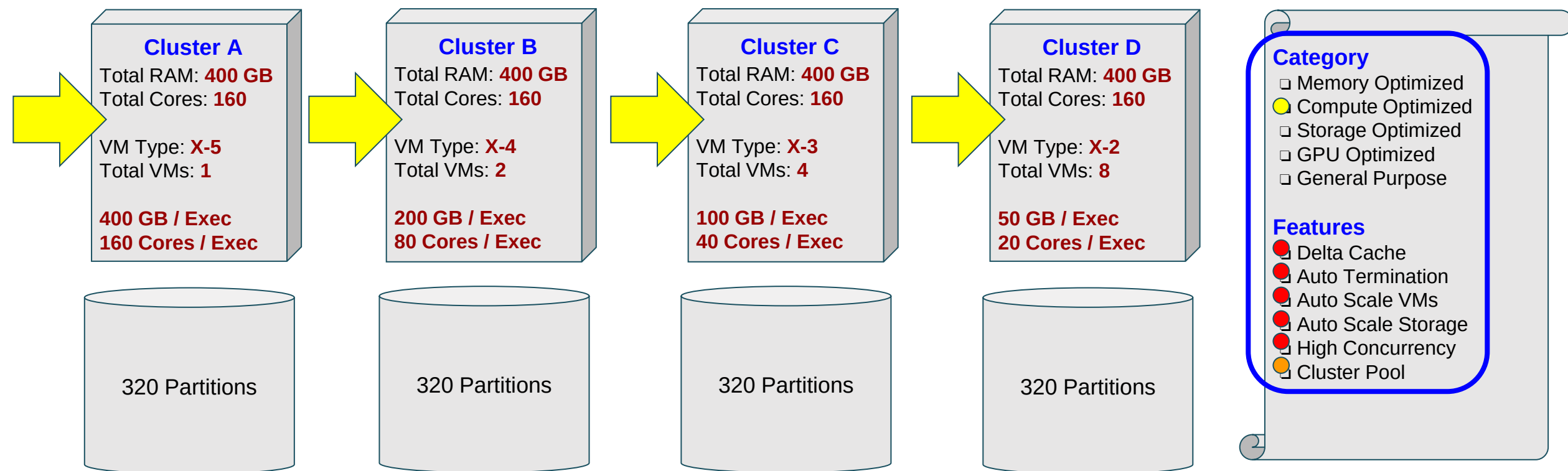
Assuming each Partition takes 3 minutes to process... Calculate the **compute-time**, **number of iterations** and **run-time** for each scenario:



Cluster Configurations Scenarios

Batch ETL: Raw -> Bronze

Which of the following cluster configurations is best / least suited for a simple ETL job that does not employ wide transformations (no joins)?

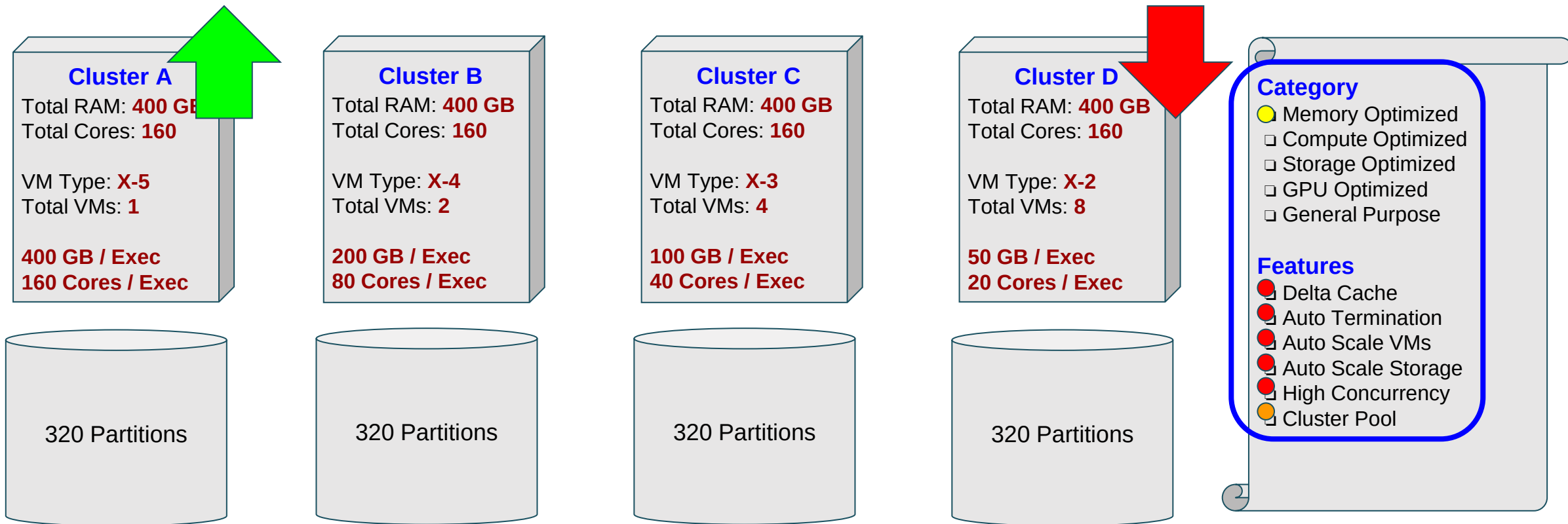


For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Cluster Configurations Scenarios

Batch ETL: Silver -> Gold

Which of the following cluster configurations is best / least suited for an ETL job that unions and joins multiple tables into a single, new table?



For this scenario, it can be assumed that each cluster has the same level of compute (total cores) and storage (total RAM)

Countries: 2.4KB
Stores: .36MB
Transactions: 18.42MB

```
FROM transactions
JOIN stores
  ON transactions.store_id = stores.id
JOIN countries
  ON transactions.country_id = countries.id
```

PO 1.5L (Optional) Exploding Join

(8) Spark Jobs

- ▶ Job 59 [View](#) (Stages: 1/1)
- ▶ Job 60 [View](#) (Stages: 1/1)
- ▶ Job 61 [View](#) (Stages: 1/1)
- ▼ Job 62 [View](#) (Stages: 1/1, 2 skipped)
 - Stage 68 0/4 succeeded [View](#) skipped
 - Stage 69 0/4 succeeded [View](#) skipped
 - Stage 70 4/4 succeeded [View](#)**
- ▶ Job 63 [View](#) (Stages: 1/1, 4 skipped)
- ▶ Job 64 [View](#) (Stages: 1/1)
- ▶ Job 66 [View](#) (Stages: 1/1)
- ▶ Job 67 [View](#) (Stages: 1/1, 1 skipped)

Shuffle Write Size /	314.5 MiB /	326.7 MiB /	343.9 MiB /	349.8 MiB /	349.8 MiB /
Shuffle Write Size /	69.5 MiB /	74 MiB /	101.1 MiB /	173.6 MiB /	174.8 MiB /

4/4	Shuffle Read	Shuffle Write
	391.2 MiB	1335.0 MiB

6/6	Shuffle Read	Shuffle Write
	18.8 MiB	679.9 MiB

Join smaller table first

4/4	Shuffle Read	Shuffle Write
	388.7 KiB	386.9 KiB

Note you asked for 8
Shuffle Partitions but
Adaptive Query
Execution (AQE)
overrode it and said 6

Shuffle Read	Shuffle Write
460.0 B	460.0 B



This is me guessing what is happening. I'm going to submit to Databricks's developers for clarification.

Why 3 Jobs?

```
1 %sql
2 SELECT * FROM mott_cat.mott_db.sales
3
```

▼ (3) Spark Jobs

- ▼ Job 0 [View](#) (Stages: 1/1)
Stage 0 8/8 succeeded [View](#)
- ▼ Job 1 [View](#) (Stages: 1/1)
Stage 1 1/1 succeeded [View](#)
- ▼ Job 2 [View](#) (Stages: 1/1)
Stage 2 1/1 succeeded [View](#)

Job 0, Stage 0
Here is where it says the RAM cache is being populated

Stage Id ▾	Pool Name	Description
0	1734176909919	SELECT * FROM mott_cat.mott_db.sales executeCollect at DatasetRefCache.scala:73 +details com.databricks.spark.util.FrameProfiler\$.record(FrameProfiler\$.scala:225) com.databricks.sql.transaction.tahoe.DeltaFileProvider\$.cachedDeltaStats(DeltaFileProvider.scala:224) com.databricks.sql.transaction.tahoe.DeltaFileProvider\$.cachedDeltaStats(DeltaFileProvider.scala:222) com.databricks.sql.transaction.tahoe.Snapshot.cachedDeltaStats(Snapshot.scala:86)

Job 0, Stage 0
Here's the RAM cache. This was initially empty prior to running the query. But now it's populated. I surmise this cache got populated during Job 0 Stage 0
Job 2, Stage 2
I surmise this is where the Parquet IO Cache (Disk Cache is populated)

Storage

Parquet IO Cache											
Host	Disk Usage	Max Disk Usage Limit	Percent Disk Usage	Metadata Cache Size	Max Metadata Cache Size Limit	Percent Metadata Usage	Data Read from IO Cache (Cache Hits, Compressed)	Data Written to IO Cache (Compressed)	Cache Misses (Compressed)	True Cache Misses	Rescheduling Cache Misses
10.200.129.145	18.1 KiB	118.0 GiB	0 %	3.5 KiB	2.4 GiB	0 %	0.0 B	16.6 KiB	16.6 KiB	16.6 KiB	0.0 B
10.200.153.177	0.0 B	135.1 GiB	0 %	0.0 B	2.4 GiB	0 %	0.0 B	0.0 B	0.0 B	0.0 B	0.0 B
Total	18.1 KiB	253.1 GiB	0 %	3.5 KiB	4.7 GiB	0 %	0.0 B	16.6 KiB	16.6 KiB	16.6 KiB	0.0 B
Data Read from External Filesystem (All Formats)		Data Read from IO Cache (Cache Hits, Compressed)		Data Written to IO Cache (Compressed)		Cache Misses (Compressed)	True Cache Misses	Rescheduling Cache Misses	Cache Hit Ratio	Number of Local Scan Tasks	Number of Scans
36.8 KiB		0.0 B		16.6 KiB		16.6 KiB	16.6 KiB	0.0 B	0 %	2	0



This is me guessing what is happening. I'm going to submit to Databrick's developers for clarification.

Why 3 Jobs?

```
1 %sql
2 SELECT * FROM mott_cat.mott_db.sales
3
```

(3) Spark Jobs

Job 0 [View](#) (Stages: 1/1)

Stage 0 8/8 succeeded [View](#)

Job 1 [View](#) (Stages: 1/1)

Stage 1 1/1 succeeded [View](#)

Job 2 [View](#) (Stages: 1/1)

Stage 2 1/1 succeeded [View](#)

Details for Stage 0 (Attempt 0)

Resource Profile Id: 0

Total Time Across All Tasks: 21 s

Locality Level Summary: Process local: 8

Associated Job Ids: 0

DAG Visualization

Show Additional Metrics

☒ Select All

☒ Scheduler Delay

☒ Task Deserialization Time

☒ Result Serialization Time

☒ Getting Result Time

☒ Peak Execution Memory

☒ Peak JVM Memory OnHeap / OffHeap

☒ Peak Tracked Execution Memory OnHeap / OffHeap

☒ Peak Storage Memory OnHeap / OffHeap

☒ Peak Pool Memory Direct / Mapped

☒ Peak Untracked Netty Memory Direct

☒ Peak Net OffHeap Memory

Event Timeline

Summary Metrics for 8 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	2 s	2 s	2 s	4 s	4 s
GC Time	91.0 ms	91.0 ms	0.1 s	0.1 s	0.1 s
Scheduler Delay	0.4 s	0.4 s	0.4 s	0.5 s	0.5 s
Task Deserialization Time	1 s	1 s	1 s	1 s	1 s
Result Serialization Time	0.0 ms	0.0 ms	8.0 ms	11.0 ms	14.0 ms
Getting Result Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Peak Execution Memory	0.0 B	0.0 B	0.0 B	0.0 B	0.0 B

Stage 0: Note there is no 'Input Record Size'. So it's not reading data files at all. Since it's not reading data, that tells me its doing some sort of housekeeping. It appears to me it's inserting the Delta Lake's JSON files into RAM cache so it can quickly pull that information the next time the query is run

Job 1, Stage 1

I was originally thining this is where the query reads the RAM cache? But since it's 31KB of data, I'm thinking that's too large to be Metadata. So now I'm thinking this is where Spark reaches out to Cloud storage and reads the data into Memory. It only needs 1 Memory Partition (1 Task) since data is so small.

Stage Id	Pool Name	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	1734176909919	SELECT * FROM mott_cat.mott_db.sales collectResult at OutputAggregator.scala:325 + details	2024/12/14 11:54:33	6 s	1/1	31.6 KiB			



This is me guessing what is happening. I'm going to submit to Databricks's developers for clarification.

Why 3 Jobs?

```
1 %sql
2 SELECT * FROM mott_cat.mott_db.sales
3
```

(3) Spark Jobs

▼ Job 0 [View](#) (Stages: 1/1)

Stage 0 8/8 succeeded [View](#)

▼ Job 1 [View](#) (Stages: 1/1)

Stage 1 1/1 succeeded [View](#)

▼ Job 2 [View](#) (Stages: 1/1)

Stage 2 1/1 succeeded [View](#)

Note there is no 'Input' for Stage 0. I suspect this is because it's not reading any data. Instead it's putting the Delta Lake JSON metadata for this table into RAM cache.

Stage Id	Pool Name	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
2	1734176909919	SELECT * FROM mott_cat.mott_db.sales collectResult at OutputAggregator.scala:325	2024/12/14 11:54:39	0.2 s	1/1	5.2 KiB			
1	1734176909919	SELECT * FROM mott_cat.mott_db.sales collectResult at OutputAggregator.scala:325	2024/12/14 11:54:33	6 s	1/1	31.6 KiB			
0	1734176909919	SELECT * FROM mott_cat.mott_db.sales executeCollect at DatasetRefCache.scala:73	2024/12/14 11:54:25	6 s	8/8				



This is me guessing what is happening. I'm going to submit to Databrick's developers for clarification.

Why 3 Jobs?

If run query again, I only have 2 Jobs, not 3.
That's because I don't have to read the Metadata into RAM cache, since it's already there

```
1 %sql
2 SELECT * FROM mott_cat.mott_db.sales
3
```

▼ (3) Spark Jobs

- ▼ Job 0 [View](#) (Stages: 1/1)
Stage 0 8/8 succeeded [View](#)
- ▼ Job 1 [View](#) (Stages: 1/1)
Stage 1 1/1 succeeded [View](#)
- ▼ Job 2 [View](#) (Stages: 1/1)
Stage 2 1/1 succeeded [View](#)

```
1 %sql
2 -- Run again, only 2 Jobs now
3 SELECT * FROM mott_cat.mott_db.sales;
4
```

▼ (2) Spark Jobs

- ▼ Job 3 [View](#) (Stages: 1/1)
Stage 3 1/1 succeeded [View](#)
- ▼ Job 4 [View](#) (Stages: 1/1)
Stage 4 1/1 succeeded [View](#)

2nd query (Stage 3 and 4) doesn't have any Input. That's because it's pulling everything out of the RAM Cache (where's the data) and the Disk Cache (have answer set there so don't need to touch any files)

Stage Id ▼	Pool Name	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
4	1734176909919	SELECT * FROM mott_cat.mott_db.sales; collectResult at OutputAggregator.scala:325 + details	2024/12/14 12:43:49	51 ms	1/1				
3	1734176909919	SELECT * FROM mott_cat.mott_db.sales; collectResult at OutputAggregator.scala:325 + details	2024/12/14 12:43:49	89 ms	1/1				
2	1734176909919	SELECT * FROM mott_cat.mott_db.sales collectResult at OutputAggregator.scala:325 + details	2024/12/14 11:54:39	0.2 s	1/1	5.2 KiB			
1	1734176909919	SELECT * FROM mott_cat.mott_db.sales collectResult at OutputAggregator.scala:325 + details	2024/12/14 11:54:33	6 s	1/1	31.6 KiB			
0	1734176909919	SELECT * FROM mott_cat.mott_db.sales executeCollect at DatasetRefCache.scala:73 + details	2024/12/14 11:54:25	6 s	8/8				



How Ingestion Time Clustering works

Ingestion time clustering ensures that clustering for tables is always maintained by ingestion time, enabling performance gains through data skipping for queries that filter by date or time



Resulting Delta Table

ID	Name	Time	File 001
1	Bilbo	T	
2	Sam	T+1	File 002
3	Gimli	T+2	
4	Aragorn	T+3	File 003
5	Gandalf	T+4	
6	Frodo	T+5	

Ingestion time clustering ensures data is maintained in the order of ingestion, significantly improving clustering

Ingestion Time Clustering is enabled by default on Databricks Runtime 11.2 and Databricks SQL (version 2022.35 and above)